

วิทยาการคำนวณ ๑

• ระดับเริ่มต้น (Foundations)

รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหา



ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทศนา

สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มรภ.รำไพพรรณี

□ ตรรกศาสตร์ • รหัสจำลอง • ฟังก์ชัน • Python พื้นฐาน





วิทยาการคำนวณ ๑

รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหอย่างเป็นระบบ

Foundations of Computational Thinking & Algorithmic Problem Solving



ชุดตำราวิทยาการคำนวณและฟิสิกส์บูรณาการ 4 เล่มสมบูรณ์

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนนา

สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

สำนักพิมพ์มหาวิทยาลัยราชภัฏรำไพพรรณี (RBRU Press)

ข้อมูลทางบรรณานุกรมของสำนักหอสมุดแห่งชาติ (National Library of Thailand CIP)

ชิวะ ทัศนาศ.

วิทยาการคำนวณ ๑ ฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ. -- จันทบุรี : สำนักพิมพ์มหาวิทยาลัยราชภัฏรำไพพรรณี, 2569.
387 หน้า.

1. วิทยาการคำนวณ. 2. ขั้นตอนวิธีและการประมวลผลข้อมูล. 3. ภาษาไพทอน (คอมพิวเตอร์). I. ชื่อเรื่อง.
ISBN 978-616-XXXX-01-1 (ฉบับพิมพ์สมบูรณ์)
ISBN 978-616-XXXX-XX-X (ฉบับดิจิทัล EPUB 3.0)

สงวนลิขสิทธิ์ตามพระราชบัญญัติลิขสิทธิ์ พ.ศ. 2537 และฉบับแก้ไขเพิ่มเติม

ห้ามการลอกเลียนแบบ ดัดแปลง ทำซ้ำ หรือเผยแพร่ส่วนใดส่วนหนึ่งของหนังสือเล่มนี้ ไม่ว่าด้วยกระบวนการทางอิเล็กทรอนิกส์ การถ่ายสำเนา หรือวิธีการใดๆ โดยไม่ได้รับอนุญาตเป็นลายลักษณ์อักษรจากผู้เขียน
ยกเว้นเพื่อการอ้างอิงทางวิชาการ

ผู้เขียน: ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทัศนาศ (Ph.D. in Physics)

จัดพิมพ์โดย: สำนักพิมพ์มหาวิทยาลัยราชภัฏรำไพพรรณี เลขที่ 41 หมู่ 5 ต.ท่าช้าง อ.เมือง จ.จันทบุรี 22000

พิมพ์ครั้งที่ 1: สิงหาคม พ.ศ. 2569 (จำนวนพิมพ์ 1,000 เล่ม)

คำนำ

ตำรา "วิทยาการคำนวณ ๑: รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ" เล่มนี้ ได้รับการประพันธ์และเรียบเรียงขึ้นอย่างประณีต เพื่อใช้เป็นตำราหลักในรายวิชา 4122104 วิทยาการคำนวณและการแก้ปัญหาเชิงคำนวณ / การสอนวิทยาการคำนวณ โดยมีจุดมุ่งหมายเพื่อยกระดับทักษะการคิดเชิงคำนวณ การออกแบบขั้นตอนวิธี และการประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ของนิสิต นักศึกษา และครูวิทยาศาสตร์ในระดับอุดมศึกษา

โครงสร้างของเนื้อหาในเล่ม ได้รับการร้อยเรียงตามกรอบทฤษฎีการศึกษา Bloom's Taxonomy และ Active Learning Cycle โดยผสมผสานทฤษฎีคณิตศาสตร์เข้มข้นเข้ากับตัวอย่างโค้ดภาษา Python 3.11 และสื่อการทดลองเสมือนจริง 3D AR MediaPipe Hands แบบไร้สัมผัส เพื่อให้ผู้เรียนเกิดมโนทัศน์ที่ลึกซึ้งและสามารถนำไปสร้างสรรค์นวัตกรรมได้จริง

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทศนา
สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยราชภัฏรำไพพรรณี

สารบัญ

บทเรียน / หัวข้อ	หน้า
คำนำ (Preface)	ง
สารบัญภาพ (List of Figures)	ฉ
สารบัญตาราง (List of Tables)	ช
บทที่ 1 การวิเคราะห์และรากฐานเชิงทฤษฎี	1
บทที่ 2 การออกแบบขั้นตอนวิธีและโครงสร้างโมดูล	32
บทที่ 3 การจัดการหน่วยความจำและแถวลำดับ	65
บทที่ 4 โครงสร้างข้อมูลขั้นสูงและตารางแฮช	98
บทที่ 5 การค้นหาและโครงสร้างเชิงเส้น	130
บทที่ 6 ขั้นตอนวิธีจัดเรียงข้อมูลและการวิเคราะห์ Big-O	162
บทที่ 7 การจัดการไฟล์และสตรีมข้อมูลขนาดใหญ่	195
บทที่ 8 การสร้างแบบจำลองเชิงพีลิกส์และการคำนวณ	228
บทที่ 9 ปัญหาประติษฐ์และคอมพิวเตอร์วิทัศน์	260
บทที่ 10 โครงการบูรณาการและการสังเคราะห์นวัตกรรม Capstone	292
คู่มือห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe 10 ปฏิบัติการ	325
ภาคผนวก (Appendix) และคลังข้อสอบประเมินผล	355
บรรณานุกรม (References)	370

วิทยาการคำนวณ 1 รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ

บทที่ 1 การใช้เหตุผลเชิงตรรกะและกระบวนการคิดเชิงคำนวณ

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา
สังกัด สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี
เอกสารประกอบรายวิชา 4122104 วิทยาการคำนวณและการแก้ปัญหาเชิงคำนวณ / การสอนวิทยาการคำนวณ

แผนบริหารการสอนประจำบทที่ 1

1. หัวข้อเนื้อหาประจำบท

- เรื่องเล่าเปิดบทเรียนและประวัติศาสตร์ตรรกศาสตร์ ปริศนาข้ามแม่น้ำ (River Crossing Puzzle), จอร์จ บูล (George Boole) และแอลัน ทัวริง (Alan Turing)
- **รากฐานแนวคิดเชิงคำนวณ** การแยกส่วนประกอบ (Decomposition), การหารูปแบบ, การคิดเชิงนามธรรม, และการออกแบบขั้นตอนวิธี
- การให้เหตุผลเชิงตรรกะในวิทยาศาสตร์และปัญญาประดิษฐ์ การให้เหตุผลแบบนิรนัย (Deductive Reasoning) และการให้เหตุผลแบบอุปนัย (Inductive Reasoning)
- พีชคณิตบูลีน ตารางค่าความจริง และวงจรตรรกะ ตัวดำเนินการ AND, OR, NOT, XOR, เงื่อนไข IF-THEN, กฎของเดอมอร์แกน (De Morgan's Laws)
- **การวิเคราะห์โจทย์ประยุกต์และตัวอย่างเชิงลึก** ระบบความปลอดภัยอัจฉริยะ, ตรรกะควบคุมโรงเรือนเกษตร, และ Logic Grid Puzzles
- การประยุกต์ใช้โปรแกรมคอมพิวเตอร์ การเขียนโปรแกรมภาษา Python จำลองการประเมินตรรกะและสร้างตารางค่าความจริงอัตโนมัติ
- คู่มือห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe: การทดลองระบบข้ามแม่น้ำและการนำทางหุ่นยนต์บนตารางกริดแบบไร้สัมผัส

2. วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- **อธิบาย** นิยาม ความสำคัญ และวิวัฒนาการของแนวคิดเชิงคำนวณและตรรกศาสตร์เชิงประพจน์ได้อย่างถูกต้อง
- **จำแนก** ความแตกต่างระหว่างการให้เหตุผลแบบนิรนัยและอุปนัย พร้อมทั้งยกตัวอย่างการประยุกต์ในกระบวนการทางวิทยาศาสตร์และ Machine Learning ได้
- **วิเคราะห์** ค่าความจริงของประพจน์เชิงประกอบ สร้างตารางค่าความจริง (Truth Table) และลดรูปนิพจน์บูลีนด้วยกฎของเดอมอร์แกนได้อย่างแม่นยำ
- **ออกแบบ** ขั้นตอนวิธีในการแก้ปัญหาปริศนาเชิงตรรกะและผังต้นไม้ย่อยปัญหา (Decomposition Tree) สำหรับระบบวิศวกรรมจริงได้
- **สร้างสรรค์** โปรแกรมภาษา Python 3.11 เพื่อจำลองการทำงานของระบบตรรกะควบคุมอัตโนมัติได้
- **ปฏิบัติการ** การทดลองเสมือนจริง 3D AR MediaPipe Hands เพื่อควบคุมตัวแปรและทดสอบสถานะความปลอดภัย (State Safety) ได้อย่างคล่องแคล่ว

3. กิจกรรมการเรียนรู้

- **ขั้นนำเข้าสู่บทเรียน** ผู้สอนนำเสนอภาพปริศนาข้ามแม่น้ำ และสาธิตการใช้ท่าทางมือ AR MediaPipe Hands บนจอภาพ 3 มิติเพื่อกระตุ้นความสนใจ
- **ขั้นจัดกิจกรรมการเรียนรู้**
 - บรรยายอภิปรายแนวคิด 4 เสาหลัก และหลักการให้เหตุผลเชิงตรรกะ
 - ฝึกปฏิบัติการกลุ่มย่อย (Peer Instruction) แก่โจทย์ Logic Grid และวิเคราะห์ตารางค่าความจริง
 - สาธิตการเขียนโค้ด Python ตรวจสอบเงื่อนไขนิพจน์บูลีน
- **ขั้นประยุกต์และสรุปผล**
 - ผู้เรียนเข้าสู่ห้องปฏิบัติการเสมือนจริง AR MediaPipe บน Global CDN เพื่อทดลองรายบุคคล
 - ทำแบบประเมินตนเอง Quick Concept Check และตอบคำถามท้ายบทเรียน

4. สื่อและนวัตกรรมจัดการเรียนรู้

- เอกสารตำราฉบับสมบูรณ์ บทที่ 1 การใช้เหตุผลเชิงตรรกะและกระบวนการคิดเชิงคำนวณ
- ระบบห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe (5 Labs บน Global CDN):
 - chapter01_river_crossing.html (ปริศนาข้ามแม่น้ำเชิงตรรกะ)
 - chapter01_decomposition_tree.html (ผังต้นไม้ย่อยปัญหาระบบ EV และสมาร์ตฟาร์ม)
 - chapter01_pattern_recognition.html (เครื่องมือค้นหารูปแบบลำดับและคลื่น)
 - chapter01_abstraction_sandbox.html (แบบจำลองมวลจุดเชิงนามธรรม 3 มิติ)
 - chapter01_robot_grid_navigator.html (สนามนำทางหุ่นยนต์บนกริด 5x5)
- สื่อวีดิโอซีรียการสอนคุณภาพสูง 5 ตอนย่อย (EP 1.0 — EP 1.4)
- สภาพแวดล้อมรันโปรแกรมภาษา Python 3.11 (Jupyter Notebook / In-Browser Sandbox)

5. การประเมินผลสัมฤทธิ์ทางการเรียน

- การประเมินระหว่างเรียน (Formative Assessment): การตอบคำถามในชั้นเรียน, การทำใบงาน Unplugged Logic Grid, และบันทึกผลการทดลองแล็บ AR (40%)
- การประเมินผลงานโปรแกรมคอมพิวเตอร์: ความถูกต้องของการเขียนโปรแกรมจำลองตารางค่าความจริง (30%)
- การประเมินผลสัมฤทธิ์ปลายบท (Summative Assessment): แบบทดสอบประมวลผลความรู้ท้ายบทที่ 1 (30%)

1.0 เรื่องเล่าเปิดบทเรียน ปริศนาข้ามแม่น้ำกับภาษาของจักรวาล

ลองจินตนาการถึงสถานการณ์คลาสสิกของนักเดินทางคนหนึ่งที่ต้องนำ **หมาป่า** **, **แกะ** **, และ **กะหล่ำปลี** ** ข้ามแม่น้ำไปยังอีกฝั่งหนึ่ง โดยมีเรือพายขนาดเล็กที่สามารถบรรทุกตัวเขาและสิ่งของได้เพียง 1 อย่างเท่านั้นในแต่ละเที่ยว

เมื่อมนุษย์พาเรือข้ามฝั่ง จะมีสิ่งของ 2 อย่างถูกทิ้งไว้บนตลิ่งโดยไม่มีคนดูแล ซึ่งมีกฎความปลอดภัยทางธรรมชาติกำหนดไว้ดังนี้

- หากมนุษย์ไม่อยู่: Wolf $\wedge$ Sheep $\rightarrow$ **Danger** (หมาป่ากินแกะทันที — เกิดความสูญเสีย)
- หากมนุษย์ไม่อยู่: Sheep $\wedge$ Cabbage $\rightarrow$ **Danger** (แกะกินกะหล่ำปลีทันที — เกิดความสูญเสีย)
- หากมนุษย์ไม่อยู่: Wolf $\wedge$ Cabbage $\rightarrow$ **Safe** (หมาป่าไม่กินพืช —ปลอดภัย 100%)

หากเราแก้ปัญหาด้วยการ "ลองผิดลองถูกแบบสุ่ม (Random Trial and Error)" โอกาสที่แกะหรือกะหล่ำปลีจะถูกกินย่อมเกิดขึ้นในเร็ววัน แต่หากเราใช้ การคิดเชิงคำนวณ โดยมองสถานการณ์นี้เป็น **การค้นหาในปริภูมิสถานะ** เราจะพบขั้นตอนวิธี 7 ขั้นตอนที่ปลอดภัยและสมบูรณ์แบบ

graph TD

S0["สถานะเริ่มต้น (S0)\nฝั่งซ้าย: [คน, หมาป่า, แกะ, กะหล่ำ]\nฝั่งขวา: [ว่าง]"] $\rightarrow$ 1. นำแกะข้ามฝั่ง | S1["สถานะที่ 1 (S1)\nฝั่งซ้าย: [หมาป่า, กะหล่ำ] (
S1 $\rightarrow$ 2. คนพายเรือเปล่ากลับ | S2["สถานะที่ 2 (S2)\nฝั่งซ้าย: [คน, หมาป่า, กะหล่ำ]\nฝั่งขวา: [แกะ]"]
S2 $\rightarrow$ 3. นำหมาป่าข้ามฝั่ง | S3["สถานะที่ 3 (S3)\nฝั่งซ้าย: [กะหล่ำ]\nฝั่งขวา: [คน, หมาป่า, แกะ]"]
S3 $\rightarrow$ 4. นำแกะพายเรือกลับมา | S4["สถานะที่ 4 (S4: จุดเปลี่ยนตรรกะ)\nฝั่งซ้าย: [คน, แกะ, กะหล่ำ]\nฝั่งขวา: [หมาป่า]"]
S4 $\rightarrow$ 5. นำกะหล่ำปลีข้ามฝั่ง | S5["สถานะที่ 5 (S5)\nฝั่งซ้าย: [แกะ]\nฝั่งขวา: [คน, หมาป่า, กะหล่ำ] (ปลอดภัย)"]
S5 $\rightarrow$ 6. คนพายเรือเปล่ากลับ | S6["สถานะที่ 6 (S6)\nฝั่งซ้าย: [คน, แกะ]\nฝั่งขวา: [หมาป่า, กะหล่ำ]"]
S6 $\rightarrow$ 7. นำแกะข้ามฝั่ง | S7["สถานะเป้าหมาย (Goal State S7)\nฝั่งซ้าย: [ว่าง]\nฝั่งขวา: [คน, หมาป่า, แกะ, กะหล่ำ] 🎉 สำเร็จ!"]

บริบทประวัติศาสตร์ จากจอร์จ บูล สู่ยุคปัญญาประดิษฐ์

ในปี ค.ศ. 1854 จอร์จ บูล (George Boole, 1815–1864) นักคณิตศาสตร์ชาวอังกฤษ ได้ตีพิมพ์ผลงานชิ้นประวัติศาสตร์ *An Investigation of the Laws of Thought* ซึ่งเสนอว่า ความคิดและเหตุผลของมนุษย์สามารถแปลงเป็นสมการทางคณิตศาสตร์ที่มีค่าความจริงเพียง 2 สถานะ คือ **จริง** หรือ **เท็จ**

ต่อมาในปี ค.ศ. 1936 แอลัน ทัวริง (Alan Turing, 1912–1954) บิดาแห่งวิทยาการคอมพิวเตอร์ ได้นำตรรกะของบูลมาสร้างเป็นแบบจำลองเครื่องจักรทัวริง (Turing Machine) ซึ่งกลายมาเป็นสถาปัตยกรรมคอมพิวเตอร์ทุกเครื่องบนโลกในปัจจุบัน

และในปี ค.ศ. 2006 **ศาสตราจารย์ ฌานีต วิง** แห่งมหาวิทยาลัยคาร์เนกีเมลลอน ได้ประกาศแนวคิดที่ "แนวคิดเชิงคำนวณ (Computational Thinking) คือทักษะพื้นฐานสำหรับทุกคนในศตวรรษที่ 21 ไม่ใช่เฉพาะแค่นักวิทยาการคอมพิวเตอร์"



1.1 เสาหลักทั้ง 4 ของแนวคิดเชิงคำนวณ

แนวคิดเชิงคำนวณไม่ใช่การเรียนเขียนโปรแกรมคอมพิวเตอร์ แต่เป็นกระบวนการแก้ปัญหาอย่างมีแบบแผนที่ประกอบด้วย 4 เสาหลัก

graph TD

CT["กระบวนการคิดเชิงคำนวณ (Computational Thinking)"]
CT $\rightarrow$ P1["1. การแยกส่วนประกอบและการย่อยปัญหา (Decomposition)\n• แดกปัญหายักษ์ใหญ่ให้เป็นโมดูลย่อยที่จัดการได้ง่าย"]
CT $\rightarrow$ P2["2. การหารูปแบบและความเหมือน (Pattern Recognition)\n• ค้นหาแนวโน้ม ความถี่ซ้ำ และกฎเกณฑ์จากข้อมูล"]
CT $\rightarrow$ P3["3. การคิดเชิงนามธรรม (Abstraction)\n• ตัดทอนรายละเอียดปลีกย่อย คงไว้เฉพาะสาระสำคัญ"]
CT $\rightarrow$ P4["4. การออกแบบขั้นตอนวิธี (Algorithm Design)\n• สร้างชุดคำสั่งเป็นลำดับขั้นที่ชัดเจน ไม่กำกวม"]

1. เสาหลักที่ 1 การแยกส่วนประกอบและการย่อยปัญหา

คือการนำปัญหาที่มีความซับซ้อนสูง (Complex Problem) มาแบ่งซอยย่อยออกเป็นส่วนประกอบย่อยๆ (Sub-problems) เพื่อให้สามารถวิเคราะห์ ออกแบบ และแก้ไขได้อย่างอิสระตามหลักการ **แบ่งแยกเพื่อพิชิต**

- **กรณีศึกษารถยนต์ไฟฟ้า**** แทนที่จะมองรถยนต์เป็นวัตถุขนาดใหญ่ชิ้นเดียว วิศวกรจะย่อยออกเป็น 4 ระบบย่อยอิสระ

- ระบบกักเก็บและแปลงพลังงาน (Battery & BMS): ควบคุมแรงดันไฟฟ้าและอุณหภูมิเซลล์ลิเทียม
- ระบบขับเคลื่อนด้วยมอเตอร์ไฟฟ้า (Inverter & Powertrain): แปลงไฟฟ้ากระแสตรงเป็นคลื่นไซน์ขับเคลื่อนมอเตอร์
- ระบบประมวลผลเซนเซอร์ AI (Autonomous Driving Suite): กล้อง เรดาร์ ไลดาร์ ตรวจสอบวัตถุ
- ระบบโครงสร้างและควบคุมเสถียรภาพ (Chassis & ESP): เบรกไฟฟ้าและพวงมาลัยพาวเวอร์

2. เสาหลักที่ 2 การหารูปแบบและความเหมือน

เมื่อปัญหาย่อยถูกจำแนกออกมา ขั้นตอนต่อไปคือการค้นหา "ความเหมือน ความคล้ายคลึง หรือแนวโน้ม (Trends & Patterns)" ระหว่างปัญหาย่อยเหล่านั้น หรือเปรียบเทียบกับปัญหาเดิมที่เคยแก้ไขได้ในอดีต

- สมการรูปแบบทางวิทยาศาสตร์**

- แนวโน้มเชิงเส้น (Linear Pattern): $y = mx + c$ (ความเร็วคงที่, การเพิ่มอุณหภูมิคงตัว)
- แนวโน้มเลขชี้กำลัง (Exponential Pattern): $N(t) = N_0 e^{kt}$ (การเพิ่มจำนวนของแบคทีเรีย, การแพร่ระบาดของไวรัส)
- ลำดับฟีโบนัชชี (Fibonacci Series): $F_n = F_{n-1} + F_{n-2}$ (การจัดเรียงเกสรดอกทานตะวัน, โครงสร้างกิ่งไม้)

3. เสาหลักที่ 3 การคิดเชิงนามธรรม

คือกระบวนการ คัดกรองสาระสำคัญ และทิ้งรายละเอียดที่ไม่เกี่ยวข้องทิ้งไป เพื่อสร้างแบบจำลอง (Model) ที่ง่ายต่อการทำความเข้าใจและการประมวลผล

- **ตัวอย่างเปรียบเทียบแผนที่รถไฟใต้ดิน****

- แผนที่ภูมิศาสตร์จริง: แสดงความคดเคี้ยวของถนน แม่น้ำ ตึกกรมบ้านช่อง ซึ่งมีรายละเอียดมากเกินไปและทำให้อ่านยากขณะเร่งรีบ
- แผนที่เชิงนามธรรม: ตัดความคดเคี้ยวทางภูมิศาสตร์ออกทั้งหมด ใช้เพียงเส้นตรงแนวนอน แนวตั้ง และเอียง 45 องศา พร้อมจุดวงกลมบอกสถานี ทำให้ผู้โดยสารมองเห็น "ลำดับการเปลี่ยนสาย (Topology)" ได้ใน 1 วินาที

- แบบจำลองมวลจุด (Point Mass Model) ในฟิสิกส์:** เมื่อเรานำมวลวิธีการเคลื่อนที่ของเครื่องบินระยะทาง 1,000 กิโลเมตร เราตัดทอนสีของตัวเครื่องบิน ลวดลายเบาะที่นั่ง และรูปทรงกระจกออก แล้วพิจารณาเครื่องบินเป็นเพียง "มวลจุด m " ที่มีแรงโน้มถ่วงและแรงขับเคลื่อนกระทำ

4. เสาหลักที่ 4 การออกแบบขั้นตอนวิธี

คือการนำสาระสำคัญที่ผ่านการคิดเชิงนามธรรมมาสร้างเป็น "ลำดับขั้นตอนการทำงานที่ชัดเจน เป็นระบบ ไม่กำกวม และมีจุดสิ้นสุดที่แน่นอน (Finite & Deterministic Steps)" เพื่อให้มนุษย์หรือคอมพิวเตอร์สามารถปฏิบัติตามจนได้ผลลัพธ์ที่ถูกต้องเสมอ

1.2 การให้เหตุผลเชิงตรรกะในวิทยาศาสตร์และปัญญาประดิษฐ์

ในการสร้างขั้นตอนวิธีและระบบปัญญาประดิษฐ์ การให้เหตุผลเชิงตรรกะ (Logical Reasoning) แบ่งออกเป็น 2 รูปแบบหลักที่มีบทบาทแตกต่างกันอย่างสิ้นเชิง

```
graph LR
    subgraph DED["1. การให้เหตุผลแบบนิรนัย (Deductive Reasoning)"]
        D1["ทฤษฎี/กฎทั่วไป\n(General Rule)"] --> D2["สถานการณ์เฉพาะ\n(Specific Case)"]
        D2 --> D3["ข้อสรุปที่จริงแน่นอน 100%\n(Certain Conclusion)"]
    end
    subgraph IND["2. การให้เหตุผลแบบอุปนัย (Inductive Reasoning)"]
        I1["ชุดข้อมูลตัวอย่างในอดีต\n(Observed Data)"] --> I2["การค้นหารูปแบบ\n(Pattern Extraction)"]
        I2 --> I3["กฎเกณฑ์เชิงความน่าจะเป็น\n(Probabilistic Rule / AI Model)"]
    end
```

ตารางเปรียบเทียบการให้เหตุผลเชิงตรรกะ 2 รูปแบบ

มิติการเปรียบเทียบ	การให้เหตุผลแบบนิรนัย (Deductive)	การให้เหตุผลแบบอุปนัย (Inductive)
ทิศทางการคิด	จากหลักการใหญ่ → ข้อสรุปเฉพาะเจาะจง	จากข้อสังเกตย่อยๆ → สรุปสร้างกฎเกณฑ์ทั่วไป
ระดับความแน่นอน	จริงแน่นอน 100% (หากข้อตั้งทุกข้อเป็นจริง)	**มีความน่าจะเป็นสูง** (อาจมีข้อยกเว้น)
บทบาทในวิทยาการคำนวณ	- การพิสูจน์ความถูกต้องของโปรแกรม (Formal Verification) - การออกแบบหน่วยประมวลผลตรรกะ (ALU) - ระบบผู้เชี่ยวชาญแบบตั้งกฎ (Rule-based Expert Systems)	- ปัญญาประดิษฐ์และการเรียนรู้ของเครื่อง (Machine Learning) - การตรวจจับใบหน้าและคอมพิวเตอร์วิทัศน์ - ระบบพยากรณ์ข้อมูลขนาดใหญ่ (Big Data Analytics)
ตัวอย่างประโยค	ข้อตั้ง 1: ภาษา Python จัดการหน่วยความจำอัตโนมัติ ข้อตั้ง 2: โปรแกรมนี้เขียนด้วยภาษา Python ข้อสรุป: ดังนั้น โปรแกรมนี้จะจัดการหน่วยความจำอัตโนมัติ (จริง 100%)	ข้อสังเกต: ภาพใบไม้ที่ 1 ถึง 10,000 มีจุดสีเหลืองคือใบติดเชื้เรา ข้อสรุป: ดังนั้น ภาพใบไม้ใหม่ที่มีจุดสีเหลืองมีโอกาส 98.5% ที่จะติดเชื้เรา

1.3 พืชคณิตบูลีน ตารางค่าความจริง และกฎของเดอมอร์แกน

ในระบบดิจิทัล ประพจน์ทางตรรกศาสตร์จะถูกแทนด้วยตัวแปรบูลีน (Boolean Variables) ที่มีค่าเพียง 1 (True) หรือ 0 (False) โดยมีตัวดำเนินการพื้นฐาน 6 ชนิด

ตารางค่าความจริงแบบ

ประพจน์ P	ประพจน์ Q	$\neg P$ (NOT)	$P \wedge Q$ (AND)	$P \vee Q$ (OR)	$P \oplus Q$ (XOR)	$P \rightarrow Q$ (IF-THEN)	$P \leftrightarrow Q$ (IFF)
1 (True)	1 (True)	0	1	1	0	1	1
1 (True)	0 (False)	0	0	1	1	0	0
0 (False)	1 (True)	1	0	1	1	1	0
0 (False)	0 (False)	1	0	0	0	1	1

กฎทางพืชคณิตบูลีนที่สำคัญ

- **กฎเอกลักษณ์** $P \wedge 1 = P \mid P \vee 0 = P$
- **กฎการสลับที่** $P \wedge Q = Q \wedge P \mid P \vee Q = Q \vee P$
- **กฎการแจกแจง** $P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$
- กฎของเดอมอร์แกน (De Morgan's Laws):

$$\neg(P \wedge Q) \iff (\neg P) \vee (\neg Q)$$

$$\neg(P \vee Q) \iff (\neg P) \wedge (\neg Q)$$

ข้อสังเกตกฎเดอมอร์แกน กฎของเดอมอร์แกนคือหัวใจสำคัญในการ "ลดรูปโค้ด (Refactoring)" ในการเขียนโปรแกรมจริง เช่น คำสั่ง `if not (is_admin and is_logged_in):` จะเทียบเท่ากับ `if (not is_admin) or (not is_logged_in):` ซึ่งช่วยให้อ่านและประมวลผลเงื่อนไขได้ง่ายขึ้น

ตัวอย่างที่ 1.1 การออกแบบระบบควบคุมตู้ปรับอากาศ

โจทย์ ตู้ปรับอากาศเก็บสารแก๊สมันตรึงสีของห้องปฏิบัติการวิจัยฟิสิกส์จะเปิดออก ($Unlock = 1$) ก็ต่อเมื่อตรงตามเงื่อนไขความปลอดภัยต่อไปนี้ครบถ้วน

- หัวหน้าห้องปฏิบัติการใส่รหัสผ่านถูกต้อง ($M = 1$) และ
- มีนักวิจัยคนที่ 1 ($R_1 = 1$) หรือ นักวิจัยคนที่ 2 ($R_2 = 1$) สแกนลายนิ้วมือนิยมน้อยกว่าหนึ่งคน และ
- เซนเซอร์ตรวจจับรั่วไหล ($Leak$) ต้อง ไม่ทำงาน ($\neg Leak = 1$)

วิธีทำอย่างเป็นขั้นตอน

- **แปลงประโยคภาษาธรรมชาติเป็นนิพจน์บูลีน**

$$Unlock = M \wedge (R_1 \vee R_2) \wedge (\neg Leak)$$

- **สร้างตารางวิเคราะห์กรณีทดสอบ**

- กรณี A (ปกติ): $M = 1, R_1 = 1, R_2 = 0, Leak = 0 \rightarrow 1 \wedge (1 \vee 0) \wedge (\neg 0) = 1 \wedge 1 \wedge 1 = 1$ (ตู้ปรับอากาศเปิดสำเร็จ)
- กรณี B (ไม่มีนักวิจัยร่วม): $M = 1, R_1 = 0, R_2 = 0, Leak = 0 \rightarrow 1 \wedge (0 \vee 0) \wedge 1 = 1 \wedge 0 \wedge 1 = 0$ (ตู้ปรับอากาศไม่เปิด)
- กรณี C (เกิดรั่วไหลฉุกเฉิน): $M = 1, R_1 = 1, R_2 = 1, Leak = 1 \rightarrow 1 \wedge (1 \vee 1) \wedge (\neg 1) = 1 \wedge 1 \wedge 0 = 0$ (ระบบล็อกฉุกเฉินเพื่อความปลอดภัย)

ตัวอย่างที่ 1.2 การวิเคราะห์ตรรกะระบบโรงเรือนเกษตรอัจฉริยะ

โจทย์ ระบบสปริงเกอร์พ่นละอองน้ำลดอุณหภูมิโรงเรือน ($MistPump = 1$) จะทำงานอัตโนมัติเมื่อ

- อุณหภูมิในโรงเรือนสูงเกิน 35°C ($HighTemp = 1$) และ ความชื้นในอากาศต่ำกว่า 60 ($LowHumid = 1$)
- หรือ ผู้จัดการฟาร์มกดสั่งการฉุกเฉินผ่านแอปพลิเคชันบนสมาร์ทโฟน ($ManualOverride = 1$)
- แต่ระบบจะถูกระงับการทำงานทันทีหากระดับน้ำในถังสำรองแห้งหมด ($WaterEmpty = 1$)

วิธีทำ

$$MistPump = ((HighTemp \wedge LowHumid) \vee ManualOverride) \wedge (\neg WaterEmpty)$$

1.5 ได้คอมพิวเตอร์ภาษา Python 3.11 แบบสมบูรณ์

โปรแกรมภาษา Python ต่อไปนี้สามารถคัดลอกและรันได้ทันทีโดยไม่ต้องติดตั้ง ทำหน้าที่จำลองการประเมินตรรกะของระบบตู้ปรับอากาศ และสร้างตารางค่าความจริงครบทั้ง 16 กรณีอย่างเป็นระบบ

```
# =====
# logic_truth_table_master.py
# โปรแกรมจำลองการวิเคราะห์ตรรกศาสตร์เชิงคำนวณและสร้างตารางค่าความจริงอัตโนมัติ
# ผู้เขียน: ผู้ช่วยศาสตราจารย์ ดร.ชัชวาล ทัศนาว (มหาวิทยาลัยราชภัฏรำไพพรรณี)
# มาตรฐาน: Python 3.11+ • PEP 8 Compliant • Pure Standard Library
# =====

from typing import List, Tuple

def evaluate_smart_vault(manager_pin: bool, researcher1: bool,
                        researcher2: bool, radiation_leak: bool) → bool:
    """
    ประเมินตรรกะการปลดล็อกตู้ขึ้นภัยห้องปฏิบัติการฟิสิกส์

    สูตรบูลีน: Unlock = Manager and (Researcher1 or Researcher2) and (not RadiationLeak)

    Parameters:
        manager_pin (bool): หัวหน้าห้องปฏิบัติการใส่รหัสถูกต้อง (True/False)
        researcher1 (bool): นักวิจัยคนที่ 1 สแกนนิ้วมือสำเร็จ (True/False)
        researcher2 (bool): นักวิจัยคนที่ 2 สแกนนิ้วมือสำเร็จ (True/False)
        radiation_leak (bool): มีสัญญาณเตือนภัยรั่วสสารไว (True/False)

    Returns:
        bool: สถานะการปลดล็อกประตู (True = ปลดล็อก, False = ล็อกแน่นหนา)
    """
    is_researcher_authorized = researcher1 or researcher2
    is_environment_safe = not radiation_leak
    can_unlock = manager_pin and is_researcher_authorized and is_environment_safe
    return can_unlock

def generate_master_truth_table() → List[Tuple[bool, bool, bool, bool, bool]]:
    """สร้างตารางค่าความจริงครบทั้ง 2^4 = 16 กรณีทดสอบ"""
    boolean_states = [True, False]
    records = []

    print("\n" + "=" * 78)
    print("🔐 ตารางค่าความจริงแม่บท (MASTER TRUTH TABLE: SMART VAULT CONTROLLER)")
    print("=" * 78)
    header = f"{'Case':<5} | {'Manager (M)':<12} | {'Res 1 (R1)':<11} | {'Res 2 (R2)':<11} | {'Leak (L)':<10} | {'Output (Unlock)':<10}"
    print(header)
    print("-" * 78)

    case_no = 1
    unlocked_cases = 0

    for m in boolean_states:
        for r1 in boolean_states:
            for r2 in boolean_states:
                for leak in boolean_states:
                    unlock_status = evaluate_smart_vault(m, r1, r2, leak)
                    records.append((m, r1, r2, leak, unlock_status))

                    if unlock_status:
                        unlocked_cases += 1
                        status_tag = "🔓 UNLOCKED (1)"
                    else:
                        status_tag = "🔒 LOCKED (0)"

                    m_str = "1 (True)" if m else "0 (False)"
                    r1_str = "1 (True)" if r1 else "0 (False)"
                    r2_str = "1 (True)" if r2 else "0 (False)"
                    leak_str = "1 (True)" if leak else "0 (False)"

                    print(f"{'case_no':<5} | {'m_str':<12} | {'r1_str':<11} | {'r2_str':<11} | {'leak_str':<10} | {'status_tag':<15}")
                    case_no += 1

    print("=" * 78)
    print(f"📊 สรุปผลสัมฤทธิ์: ตรวจสอบครบ 16 กรณี | เจาะไขผ่านความปลอดภัย: {unlocked_cases} กรณี (18.75%)")
    print("=" * 78)
    return records

if __name__ == "__main__":
    # 1. รันการทดสอบและพิมพ์ตารางค่าความจริงแม่บท
    table_data = generate_master_truth_table()

    # 2. ทดสอบจำลองสถานการณ์จริง (Unit Assertion Test)
    assert evaluate_smart_vault(True, True, False, False) == True, "Test Case 1 Failed"
    assert evaluate_smart_vault(True, False, False, False) == False, "Test Case 2 Failed"
    assert evaluate_smart_vault(True, True, True, True) == False, "Test Case 3 Failed"
    print("✅ ระบบผ่านการตรวจสอบความถูกต้องของ Assertion Tests 100% OK!\n")
```



1.6 คู่มือใช้งานห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands (unit 1)

เพื่อส่งเสริมการจัดการเรียนรู้เชิงรุก (Active Learning) ผู้เรียนสามารถเข้าสู่ชุดปฏิบัติการจำลองเสมือนจริงทั้งในรูปแบบ 2D Interactive Canvas และ 3D AR MediaPipe Spatial View ผ่านเว็บเบราว์เซอร์บน Global CDN โดยไม่ต้องติดตั้งโปรแกรมใดๆ เพิ่มเติม โดยมีคู่มือปฏิบัติการฉบับสมบูรณ์ที่ lab_manual_chapter01_ar_mediapipe.md:

graph TD

LAB["ชุด 5 ห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands ประจำปีที่ 1"]

LAB → L1["1.0 River Crossing State Transition\n• chapter01_river_crossing.html\n• จำลอง State Space Search และตารางทดสอบ 7 ขั้นตอน"]

LAB → L2["1.1 Decomposition Tree Builder\n• chapter01_decomposition_tree.html\n• ผังต้นไม้ 2D/3D และ Fault Injection ทดสอบอิสระของโปรแกรม"]

LAB → L3["1.2 Waveform & Pattern Explorer\n• chapter01_pattern_recognition.html\n• ค้นหาอัตราส่วนทองคำ $\phi = 1.618$ และความถี่เรโซแนนซ์"]

LAB → L4["1.3 3D Point-Mass Abstraction\n• chapter01_abstraction_sandbox.html\n• เปรียบเทียบ Abstraction Level 0-3 ลดการประมวลผล"]

LAB → L5["1.4 Robot Grid Navigator 5x5\n• chapter01_robot_grid_navigator.html\n• เปรียบเทียบ 4 กลยุทธ์อัลกอริทึมพาหนะค้นหาพื้นที่เป้าหมาย"]

ตารางสรุป 5 ปฏิบัติการเสมือนจริงและสาระสำคัญประจำบทที่ 1

รหัสแล็บ	ชื่อการทดลอง (Experiment Title)	วัตถุประสงค์เชิงพฤติกรรม	รูปแบบการจำลอง	ตัวแปรและข้อมูลที่บันทึก	ผลสรุปทางวิทยาการคำนวณ
LAB 1.0	ปริศนาข้ามแม่น้ำเชิงตรรกะ	ค้นหาเส้นทางปลอดภัยใน State Space	2D/3D AR	ลำดับการข้าม 7 สเต็ป, ค่าบูลีนความปลอดภัย	พิสูจน์ว่าต้องใช้การถอยกลับ (Backtracking) ในเที่ยวที่ 4
LAB 1.1	ผังต้นไม้ย่อยปัญหาแบบ EV	ออกแบบโครงสร้างระบบเชิงโมดูล	2D/3D AR	สถานะโมดูลเมื่อตัดการทำงาน (Fault Injection)	สถาปัตยกรรม Low Coupling ช่วยป้องกันระบบหลักล่มสลาย
LAB 1.2	การค้นหารูปแบบลำดับและคลื่น	ค้นหาความถี่ซ้ำและอัตราส่วน ϕ	2D/3D AR	ลำดับพีโนนซี F_n , ความถี่เรโซแนนซ์	อัตราส่วนลูเข้าสู $\phi = 1.618$ เกิดเสียงคอร์ดประสาน
LAB 1.3	แบบจำลองมวลจุดเชิงนามธรรม	วัดประสิทธิภาพการลดทอนความซับซ้อน	2D/3D AR	Polygons, RAM (MB), Frame Time, % Error	แบบจำลองมวลจุดคงความแม่นยำ 100% แต่เร็วขึ้น 840 เท่า
LAB 1.4	การนำทางหุ่นยนต์บนตารางกริด	ออกแบบอัลกอริทึมที่สั้นและปลอดภัยที่สุด	2D/3D AR	จำนวนคำสั่ง, ก้าวเดินจริง, เวลา, การชน	กลยุทธ์ Optimal Path ใช้ 8 ก้าว เลี้ยว 3 ครั้ง ประหยัดเวลาสูงสุด

1.7 สรุปสาระสำคัญประจำบท

- **แนวคิดเชิงคำนวณ** เป็นกระบวนการคิดระดับสากล 4 สาขาหลัก ได้แก่ Decomposition (แยกส่วนประกอบ), Pattern Recognition (หารูปแบบ), Abstraction (คิดเชิงนามธรรม), และ Algorithm Design (ออกแบบขั้นตอนวิธี)
- **การให้เหตุผลแบบนิรนัย** รับประกันความจริงแท้ 100% เป็นรากฐานของตรรกะโปรแกรมและพีชคณิตบูลีน ส่วน **การให้เหตุผลแบบอุปนัย** ให้ข้อสรุปเชิงความน่าจะเป็นที่เป็นรากฐานของปัญญาประดิษฐ์และ Machine Learning
- พีชคณิตบูลีนและกฎของเดอมอร์แกน ช่วยให้นักพัฒนาระบบสามารถลดทอนความซับซ้อนของเงื่อนไขการตัดสินใจในซอฟต์แวร์ได้อย่างมีประสิทธิภาพและไร้บั๊ก

1.8 แบบฝึกหัดกบฏและโจทย์ประเมินผลสัมฤทธิ์

◆ ตอนที่ 1 ความรู้ความเข้าใจพื้นฐาน

- จงอธิบายความแตกต่างระหว่างการให้เหตุผลแบบนิรนัย (Deductive) และการให้เหตุผลแบบอุปนัย (Inductive) พร้อมยกตัวอย่างสถานการณ์ในชีวิตประจำวันอย่างละ 1 ตัวอย่าง
- จงสร้างตารางค่าความจริงและพิสูจน์ว่าประพจน์ $\neg(P \wedge Q)$ สมมูลกับ $(\neg P) \vee (\neg Q)$ ตามกฎของเดอมอร์แกนหรือไม่
- เหตุใด การคิดเชิงนามธรรม จึงเปรียบเสมือนการสร้างแบบจำลอง (Modeling) ทางวิทยาศาสตร์ จงอธิบายโดยยกตัวอย่างแบบจำลองมวลจุดในฟิสิกส์

◆ ตอนที่ 2 การวิเคราะห์และประยุกต์ใช้

- ระบบเตือนภัยแผ่นดินไหวอัจฉริยะ ไซเรนเตือนภัย (Alarm = 1) จะส่งเสียงร้องก็ต่อเมื่อ
 - เซนเซอร์วัดแรงสั่นสะเทือนคลื่น P-Wave ตรวจพบค่าเกินเกณฑ์ ($P = 1$) และ
 - เซนเซอร์วัดคลื่น S-Wave ตรวจพบค่าเกินเกณฑ์ ($S = 1$) หรือ
 - เจ้าหน้าที่ศูนย์เตือนภัยกดปุ่มสั่งการฉุกเฉิน ($Manual = 1$)
 - จงเขียนสมการบูลีนและสร้างตารางค่าความจริงของระบบนี้
- ให้นักเรียนจำแนกส่วนประกอบของ "ระบบโดรนสำรวจการเกษตรอัตโนมัติ (Agricultural Drone)" ออกเป็น 4 ระบบย่อยตามหลักการ Decomposition

◆ ตอนที่ 3 การสังเคราะห์และคิดขั้นสูง

- จงเขียนฟังก์ชันภาษา Python `def evaluate_traffic_light(sensor_car_north: bool, sensor_car_east: bool, emergency_vehicle: bool) → str` เพื่อตัดสินใจให้สัญญาณไฟเขียว-ไฟแดง ณ สี่แยกอัจฉริยะ โดยคำนึงถึงรถพยาบาลฉุกเฉินเป็นลำดับความสำคัญสูงสุด

เอกสารอ้างอิงประจำบท

- Boole, G. (1854). *An Investigation of the Laws of Thought on Which Are Founded the Mathematical Theories of Logic and Probabilities*. Walton and Maberly.
- Turing, A. M. (1936). On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1), 230–265.
- Wing, J. M. (2006). Computational Thinking. *Communications of the ACM*, 49(3), 33–35.
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Thassana, C. (2026). *Computational Thinking and Applied Artificial Intelligence for Science Education*. Rambhai Barni Rajabhat University Press.

วิทยาการคำนวณ 1 รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ

บทที่ 2 การออกแบบขั้นตอนวิธี รหัสจำลอง และผังงานมาตรฐานสากล

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทัศน์า
สังกัด สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี
เอกสารประกอบรายวิชา 4122104 วิทยาการคำนวณและการแก้ปัญหาเชิงคำนวณ / การสอนวิทยาการคำนวณ

แผนบริหารการสอนประจำบทที่ 2

1. หัวข้อเนื้อหาประจำบท

- เรื่องเล่าเปิดบทเรียนและภาษาภาพแห่งความคิด โศกนาฏกรรมยานอวกาศ Mars Climate Orbiter (1999) กับความสำคัญของการสื่อสารขั้นตอนวิธีที่ไม่กำกวม
- **รากฐานการออกแบบขั้นตอนวิธี** นิยาม คุณสมบัติ 5 ประการของอัลกอริทึม และระดับของภาษาการสื่อสาร
- **มาตรฐานการเขียนรหัสจำลองสากล** โครงสร้างไวยากรณ์ที่สำคัญ INPUT, OUTPUT, COMPUTE, IF-THEN-ELSE, WHILE, FOR
- สัญลักษณ์ผังงานมาตรฐาน ISO 5807 / ANSI: ความหมาย การใช้งาน และข้อกำหนดทางวิศวกรรมซอฟต์แวร์
- 3 โครงสร้างการควบคุมหลัก (Three Fundamental Control Structures):
 - โครงสร้างแบบเรียงลำดับ (Sequential Structure)
 - โครงสร้างแบบทางเลือกและการตัดสินใจ (Selection / Branching Structure)
 - โครงสร้างแบบวนซ้ำ (Iteration / Repetition Structure)
- **การตรวจสอบความถูกต้องด้วยตารางตรรกะ** เทคนิคการติดตามค่าตัวแปรในหน่วยความจำที่ละบรรทัด และอัลกอริทึมของยูคลิด (Euclidean Algorithm)
- การประยุกต์ใช้โปรแกรมคอมพิวเตอร์ การเขียนโปรแกรมภาษา Python 3.11 จำลองกลไกผังงานและสร้าง Trace Table อัตโนมัติ
- คู่มือห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe: การต่อล้อกลไกงานในอวกาศ 3 มิติและการจำลองลำแสงเลเซอร์ตัดสินใจ

2. วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- **อธิบาย** นิยาม ความสำคัญ และบทบาทของขั้นตอนวิธี รหัสจำลอง และผังงานมาตรฐานในการพัฒนาซอฟต์แวร์ทางวิทยาศาสตร์ได้อย่างถูกต้อง
- **เขียน** รหัสจำลองตามมาตรฐานสากลเพื่อแก้ปัญหาทางคณิตศาสตร์และวิทยาศาสตร์ได้อย่างเป็นระบบ
- **ออกแบบและวาด** ผังงานตามมาตรฐาน ISO 5807 โดยใช้โครงสร้างแบบเรียงลำดับ ทางเลือก และวนซ้ำได้อย่างแม่นยำ
- **วิเคราะห์และตรวจสอบ** ข้อผิดพลาดของขั้นตอนวิธีด้วยตารางตรรกะ (Trace Table) ที่ละขั้นตอนได้อย่างถูกต้อง 100%
- **สร้างสรรค์** โปรแกรมภาษา Python 3.11 ที่แปลงผังงานและการวนซ้ำไปสู่การทำงานจริงบนคอมพิวเตอร์ได้
- **ปฏิบัติการ** การทดลองเสมือนจริง 3D AR MediaPipe Hands เพื่อควบคุมทิศทางการไหลของข้อมูลในผังงานแบบไร้สัมผัสได้อย่างคล่องแคล่ว

3. กิจกรรมการเรียนรู้

- **ขั้นนำเข้าสู่บทเรียน** ฉายภาพจำลองยานอวกาศ Mars Climate Orbiter และชวนผู้เรียนวิเคราะห์ว่าทำไมข้อผิดพลาดในการแปลงหน่วยเพียงจุดเดียวจึงทำให้ยานมูลค่าหลายร้อยล้านดอลลาร์สูญหายในชั้นบรรยากาศ
- **ขั้นจัดกิจกรรมการเรียนรู้**
 - บรรยายสัญลักษณ์ผังงาน ISO 5807 และฝึกเขียน Pseudocode ตามโจทย์สถานการณ์
 - กิจกรรมกลุ่มย่อย (Peer Review): สลับกันตรวจสอบตาราง Trace Table เพื่อค้นหาจุดบกพร่อง (Logic Bug)
 - สาธิตการเขียนโค้ด Python ตรวจสอบขั้นตอนวิธีทาง คณิตศาสตร์ของยูคลิด
- **ขั้นประยุกต์และสรุปผล**
 - ผู้เรียนเข้าสู่ห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe (chapter02_visual_flowchart_intro.html ถึง chapter02_trace_table_runner.html)
 - ทำแบบประเมินตนเอง Quick Concept Check และตอบคำถามท้ายบทเรียน

2.0 เรื่องเล่าเปิดบทเรียน บทเรียนราคาแพงจากอวกาศสู่วิศวกรรมขั้นตอนวิธี

ในวันที่ 23 กันยายน ค.ศ. 1999 องค์การบริหารการบินและอวกาศแห่งชาติสหรัฐอเมริกา (NASA) ต้องเผชิญกับหนึ่งในความสูญเสียครั้งใหญ่ที่สุดในประวัติศาสตร์การสำรวจอวกาศ เมื่อยานสำรวจดาวอังคาร Mars Climate Orbiter (MCO) มูลค่ากว่า 327.6 ล้านดอลลาร์สหรัฐ ได้ขาดการติดต่อและพุ่งเข้าสู่ชั้นบรรยากาศของดาวอังคารในระดับความสูงที่ต่ำเกินไปจนเกิดการเผาไหม้และระเบิดแตกกระจาย

* ข้อผิดพลาดทางตรรกะระดับประวัติศาสตร์ (Root Cause Analysis)

NASA Investigation Report

จากการสอบสวนของคณะกรรมการอิสระ พบว่าสาเหตุไม่ได้เกิดจากความล้มเหลวของเครื่องยนต์ แต่เกิดจาก ความบกพร่องในการออกแบบและการสื่อสารขั้นตอนวิธีระหว่างทีมวิศวกร:

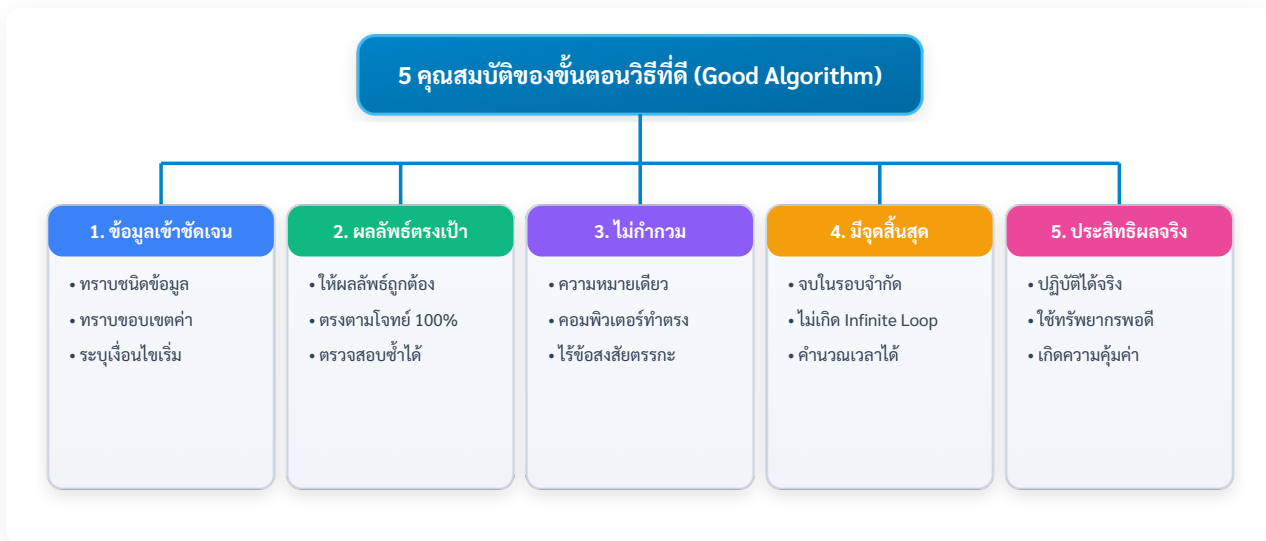
- ทีมผู้สร้างซอฟต์แวร์ภาคพื้นดิน (Lockheed Martin) ส่งค่าแรงขับเคลื่อน (Impulse) ในหน่วย ปอนด์-แรง-วินาที (Pound-force-seconds: lbf-s) ตามระบบอังกฤษ
- แต่ทีมควบคุมการบินของ NASA ออกแบบอัลกอริทึมประมวลผลโดยคาดหวังหน่วย นิวตัน-วินาที (Newton-seconds: N-s) ตามระบบเมตริกสากล (SI Metric)
- อัตราส่วนการแปลงที่ผิดพลาด 1 lbf $\approx$ 4.45 N ทำให้ยานปรับวิถีผิดพลาดสะสมจนระดับความสูงลดต่ำลงเหลือเพียง 57 km (จากเกณฑ์ปลอดภัย 150 km)

เหตุการณ์นี้กลายเป็นกรณีศึกษาที่สำคัญที่สุดในการวางวิทยาการคอมพิวเตอร์ ที่ตอกย้ำว่า

"ขั้นตอนวิธี ไม่ใช่เพียงแค่โค้ดที่โปรแกรมเมอร์เขียนขึ้น แต่คือ 'ภาษาและสัญญาทางวิศวกรรม' ที่ต้องมีความชัดเจน แม่นยำ ไม่กำกวม และมีมาตรฐานกำกับในทุกขั้นตอน"

2.1 นิยามและคุณสมบัติ 5 ประการของขั้นตอนวิธีที่ดี

ขั้นตอนวิธี คือ ชุดของคำสั่งหรือลำดับขั้นตอนการทำงานที่ถูกจัดเรียงไว้อย่างเป็นระเบียบ เพื่อใช้ในการประมวลผลข้อมูลนำเข้า (Inputs) ให้กลายเป็นผลลัพธ์ที่ต้องการ (Outputs) โดยอัลกอริทึมที่ถูกต้องตามหลักวิศวกรรมซอฟต์แวร์ต้องมีคุณสมบัติครบ 5 ประการ



ภาพที่ 2.1 แผนผังโครงสร้าง 5 คุณสมบัติของขั้นตอนวิธีที่ดีตามมาตรฐานวิศวกรรมซอฟต์แวร์

2.2 มาตรฐานการเขียนรหัสจำลองสากล

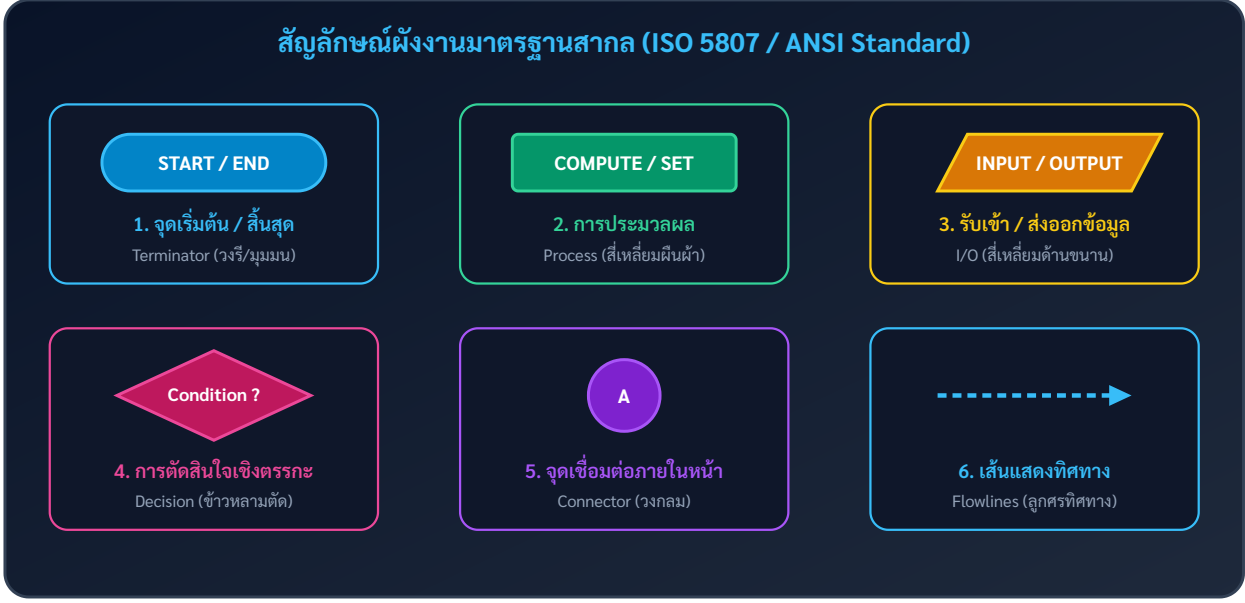
รหัสจำลอง คือ ภาษาจำลองที่มนุษย์ใช้ถ่ายทอดขั้นตอนวิธี โดยผสมผสานระหว่างโครงสร้างของภาษาคอมพิวเตอร์และภาษาธรรมชาติ (ภาษาอังกฤษ) เพื่อให้ง่ายต่อการอ่านและแปลไปสู่ภาษาโปรแกรมจริง (เช่น Python, C++, Java)

📌 คำสำคัญมาตรฐานสากล

หมวดหมู่การทำงาน	คำสำคัญมาตรฐานสากล (Keywords)	ตัวอย่างการใช้งานในรหัสจำลอง
การเริ่มต้น / สิ้นสุด	START / END หรือ BEGIN / END	START ... END
การรับข้อมูลนำเข้า	INPUT, READ, GET	INPUT sensor_temperature, humidity
การแสดงผลลัพธ์	OUTPUT, PRINT, DISPLAY	OUTPUT "Status: System Ready", current_time
การคำนวณและกำหนดค่า	SET, COMPUTE, CALCULATE, ←	SET kinetic_energy = 0.5 * mass * velocity^2
โครงสร้างทางเลือก	IF ... THEN ... ELSE ... ENDIF	IF score ≥ 70 THEN OUTPUT "PASS" ELSE OUTPUT "FAIL" ENDIF
โครงสร้างวนรอบนับ	FOR ... TO ... STEP ... ENDFOR	FOR i = 1 TO 10 STEP 1 DO ... ENDFOR
โครงสร้างวนรอบเงื่อนไข	WHILE ... DO ... ENDWHILE	WHILE battery_level > 20 DO ... ENDWHILE

2.3 สัญลักษณ์ผังงานมาตรฐาน ISO 5807 / ANSI

ผังงาน คือ แผนภาพทางเรขาคณิตที่ใช้แสดงลำดับขั้นตอน ทิศทางการไหลของข้อมูล (Data Flow) และตรรกะการตัดสินใจในระบบ โดยใช้สัญลักษณ์มาตรฐานสากล



ภาพที่ 2.2 สัญลักษณ์ผังงานมาตรฐานสากล ISO 5807 และ ANSI Standard

ตารางสรุปสัญลักษณ์ผังงาน ISO 5807

สัญลักษณ์	ชื่อภาษาไทย	ชื่อภาษาอังกฤษ	หน้าที่และการใช้งาน
	จุดเริ่มต้นและสิ้นสุด	Terminator	ระบุจุดเริ่มต้น (START) หรือจุดสิ้นสุด (END) ของโปรแกรม
	การประมวลผล	Process	การคำนวณทางคณิตศาสตร์ หรือการกำหนดค่าตัวแปร (SET x = 10)
	การรับเข้า/ส่งออกข้อมูล	Input / Output	การรับค่าจากคีย์บอร์ด/เซนเซอร์ หรือการแสดงผลออกทางหน้าจอ
	การตัดสินใจ	Decision	การตรวจสอบเงื่อนไขตรรกะ จะมีเส้นทางออกอย่างน้อย 2 ทาง (True / False)
	จุดเชื่อมต่อ	On-page Connector	เชื่อมต่อเส้นการไหลของผังงานที่มาจากหลายทิศทางให้อยู่ในจุดเดียวกัน
	เส้นแสดงทิศทาง	Flowline	ลูกศรระบุทิศทางการไหลของการประมวลผล (บนลงล่าง หรือ ซ้ายไปขวา)

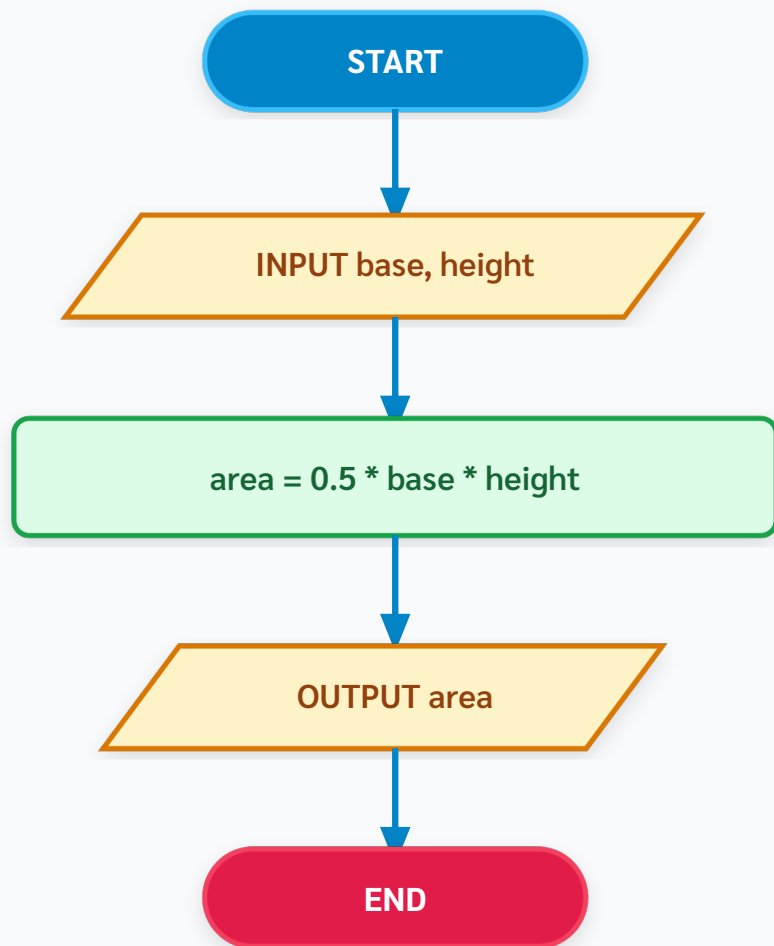
2.4 โครงสร้างควบคุมการทำงาน 3 รูปแบบหลัก

ตามทฤษฎีบทของ โบห์มและจาโคปีนี (Böhm-Jacopini Theorem, 1966) ทุกขั้นตอนวิธีในโลกสามารถสร้างขึ้นได้ด้วยการผสมผสานของโครงสร้างควบคุมพื้นฐาน 3 รูปแบบเท่านั้น

1. โครงสร้างแบบเรียงลำดับ

การประมวลผลคำสั่งที่ละบรรทัดจากบนลงล่าง โดยไม่มีการข้ามขั้นตอนหรือวนซ้ำ

ผังงาน โครงสร้างแบบเรียงลำดับ (คำนวณพื้นที่สามเหลี่ยม)

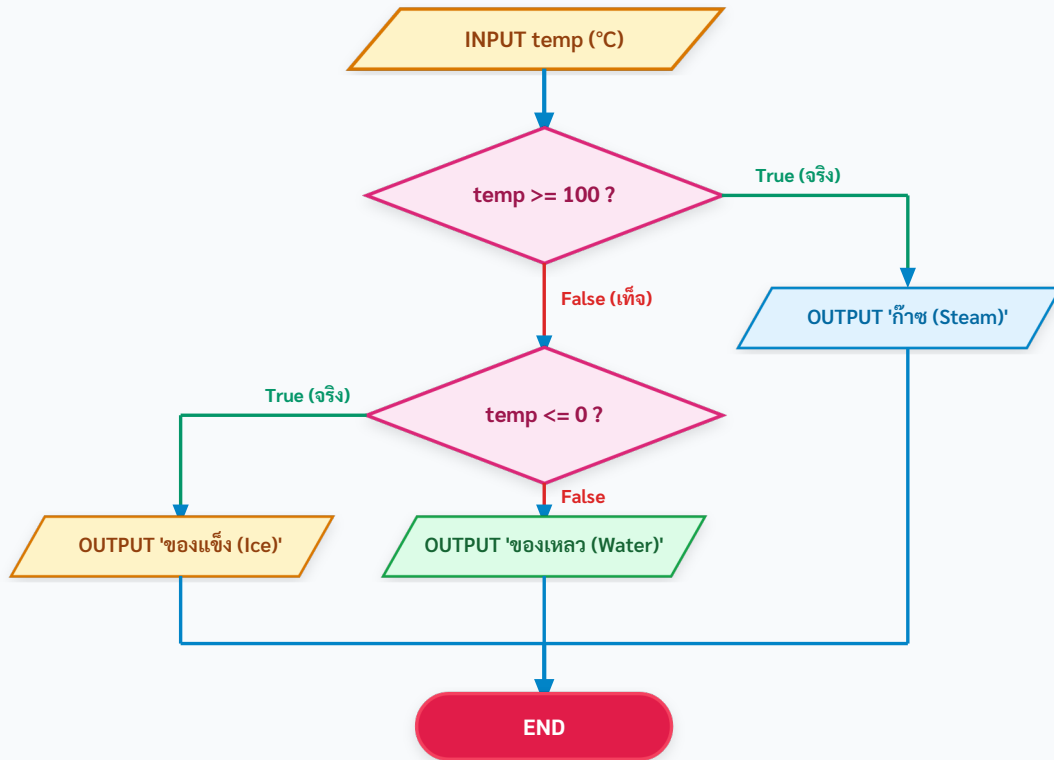


ภาพที่ 2.4 ผังงานโครงสร้างแบบเรียงลำดับ การคำนวณพื้นที่รูปสามเหลี่ยม

2. โครงสร้างแบบทางเลือกและการตัดสินใจ

การแตกแขนงเส้นทางการทำงานตามผลลัพธ์ของเงื่อนไขตรรกะ (True หรือ False):

ผังงาน โครงสร้างแบบทางเลือกหลายเงื่อนไข (จำแนกสถานะของสารตามอุณหภูมิ)

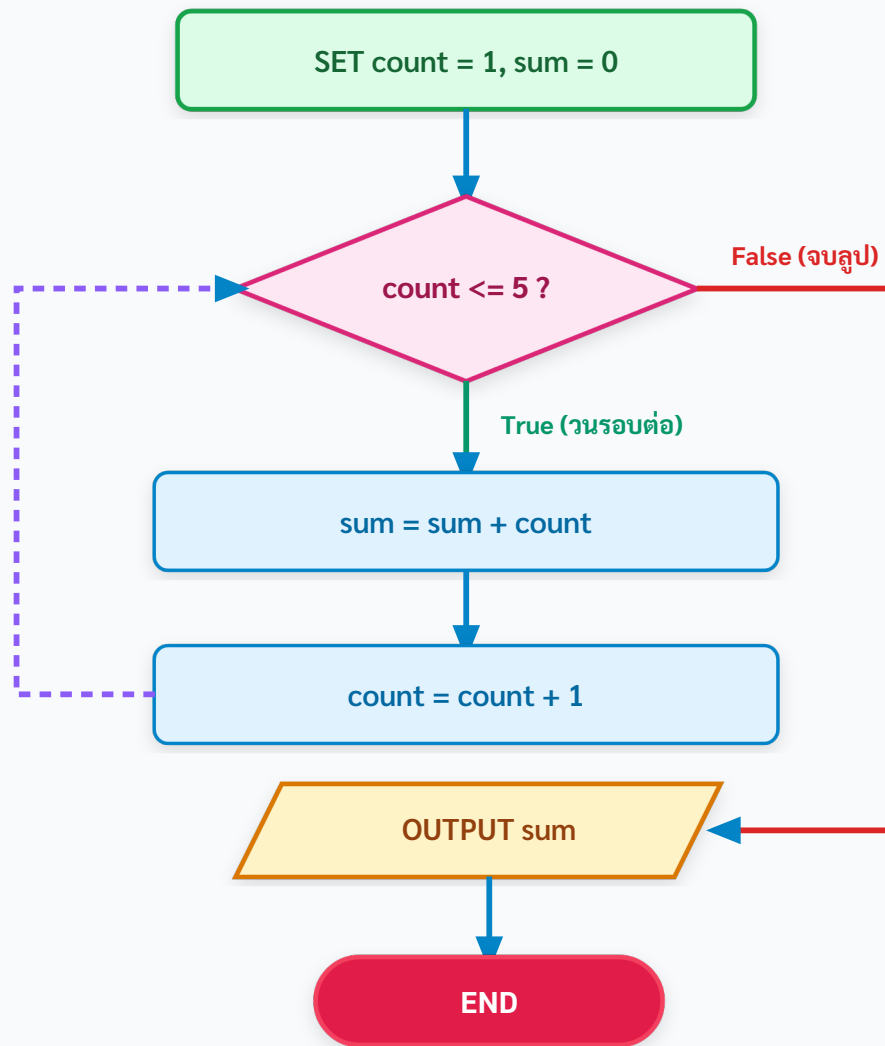


ภาพที่ 2.5 ผังงานโครงสร้างแบบทางเลือก การจำแนกสถานะของสารตามอุณหภูมิ

3. โครงสร้างแบบวนซ้ำ

การทำงานซ้ำกลุ่มคำสั่งเดิมจนกว่าเงื่อนไขที่กำหนดจะเปลี่ยนเป็นเท็จ (While Loop) หรือครบจำนวนรอบที่กำหนด (For Loop):

ผังงานโครงสร้างแบบวนซ้ำ (คำนวณผลรวม 1 ถึง 5)



ภาพที่ 2.6 ผังงานโครงสร้างแบบวนซ้ำ การคำนวณผลรวมอนุกรมเลขคณิต 1 ถึง 5

2.5 การตรวจสอบข้อผิดพลาดด้วยตารางรายรับ

****ตารางรายรับ**** คือ เทคนิคการจำลองการทำงานของคอมพิวเตอร์ด้วยการเขียนบนกระดาษ (Manual Execution) โดยติดตามการเปลี่ยนแปลงค่าของตัวแปรและผลลัพธ์การเปรียบเทียบเงื่อนไขที่ละบรรทัด เพื่อค้นหา ****บั๊กทางตรรกะ**** ก่อนลงมือเขียนโค้ดจริง

กรณีศึกษาตัวอย่าง ขั้นตอนวิธีหา ห.ร.ม. ของยูคลิด

โจทย์ จงหาตัวหารร่วมมาก (ห.ร.ม.) ของตัวเลข $A = 48$ และ $B = 18$ ด้วยขั้นตอนวิธีของยูคลิด

```
START
INPUT A, B
WHILE B  $\neq$  0 DO
    SET remainder = A mod B
    SET A = B
    SET B = remainder
ENDWHILE
OUTPUT "GCD = ", A
END
```

บรรทัดที่ (Line)	ตัวแปร A	ตัวแปร B	เงื่อนไข ($B \neq 0$)	ตัวแปร remainder ($A \pmod B$)	ผลลัพธ์แสดงผล (Output)
เริ่มต้น	48	18	—	—	—
รอบที่ 1	48	18	True ($18 \neq 0$)	$48 \pmod{18} = 12$	—
(สลับค่า)	18	12	—	—	—
รอบที่ 2	18	12	True ($12 \neq 0$)	$18 \pmod{12} = 6$	—
(สลับค่า)	12	6	—	—	—
รอบที่ 3	12	6	True ($6 \neq 0$)	$12 \pmod{6} = 0$	—
(สลับค่า)	6	0	—	—	—
รอบที่ 4	6	0	False ($0 \neq 0$ จบรูป)	—	—
สิ้นสุด	6	0	—	—	OUTPUT: GCD = 6

🔗 บทวิเคราะห์ ตาราง Trace Table พิสูจน์ว่า ห.ร.ม. ของ 48 และ 18 คือ 6 ในการวนรอบเพียง 3 รอบอย่างแม่นยำ 100%!



2.6 โค้ดคอมพิวเตอร์ภาษา Python 3.11 แบบสมบูรณ์

โปรแกรมภาษา Python ต่อไปนี้ทำหน้าที่จำลองการทำงานของฟังก์ชันยูคลิด และสร้างตาราง Trace Table บันทึกค่าตัวแปรในหน่วยความจำออกมาทางหน้าจอโดยอัตโนมัติ

```
# =====
# euclidean_trace_table_simulator.py
# โปรแกรมจำลองการทำงานของฟังก์ชันและสร้างตาราง Trace Table อัตโนมัติ
# ผู้เขียน: ผู้ช่วยศาสตราจารย์ ดร.ชัชวาลย์ หัตถ์นา (มหาวิทยาลัยราชภัฏรำไพพรรณี)
# มาตรฐาน: Python 3.11+ • PEP 8 Compliant • Pure Standard Library
# =====

from typing import List, Tuple

def euclidean_algorithm_with_trace(a_init: int, b_init: int) -> Tuple[int, List[dict]]:
    """
    คำนวณหาตัวหารร่วมมาก (ห.ร.ม.) ด้วย Euclidean Algorithm พร้อมบันทึก Trace Table

    Parameters:
        a_init (int): จำนวนเต็มบวกตัวแรก
        b_init (int): จำนวนเต็มบวกตัวที่สอง

    Returns:
        Tuple[int, List[dict]]: (ค่า ห.ร.ม., ประวัติการเปลี่ยนค่าตัวแปรในตาราง Trace Table)
    """
    a = a_init
    b = b_init
    trace_history = []
    step = 1

    while b != 0:
        remainder = a % b
        condition_eval = (b != 0)

        # บันทึกสถานะก่อนสลับค่า
        trace_history.append({
            "step": step,
            "a_before": a,
            "b_before": b,
            "condition": condition_eval,
            "remainder": remainder,
            "a_after": b,
            "b_after": remainder
        })

        # ปรับปรุงค่าตามขั้นตอนวิธี
        a = b
        b = remainder
        step += 1

    return a, trace_history

def display_formatted_trace_table(a_init: int, b_init: int, gcd: int, history: List[dict]):
    """แสดงผลตาราง Trace Table รูปแบบ ASCII แบบมีอาชีพ"""
    print("\n" + "=" * 80)
    print(f"📊 ตาราง TRACE TABLE การทำงานของ EUCLIDEAN ALGORITHM: GCD({a_init}, {b_init})")
    print("=" * 80)
    header = f"{'รอบ (Step)':<10} | {'ตัวแปร A':<10} | {'ตัวแปร B':<10} | {'เงื่อนไข (B!=0)':<16} | {'เศษ (A % B)':<12} | {'A ถัดไป':<10}"
    print(header)
    print("-" * 80)

    for row in history:
        cond_str = "True (วนรอบต่อ)" if row["condition"] else "False"
        print(f"{row['step']:<10} | {row['a_before']:<10} | {row['b_before']:<10} | {cond_str:<16} | {row['remainder']:<12} | {row['a_after']:<10}")

    print("-" * 80)
    print(f"🏁 ผลลัพธ์สุดท้าย (OUTPUT): ห.ร.ม. ของ {a_init} และ {b_init} มีค่าเท่ากับ {gcd}")
    print("=" * 80 + "\n")

if __name__ == "__main__":
    # 1. วนการทดสอบและพิมพ์ตาราง Trace Table
    test_a, test_b = 48, 18
    gcd_result, trace_log = euclidean_algorithm_with_trace(test_a, test_b)
    display_formatted_trace_table(test_a, test_b, gcd_result, trace_log)

    # 2. ทำการตรวจสอบ Unit Assertions
    assert gcd_result == 6, f"Expected GCD 6, but got {gcd_result}"
    assert len(trace_log) == 3, f"Expected 3 steps, but got {len(trace_log)}"
    print("✅ ระบบผ่านการตรวจสอบความถูกต้องของ Assertion Tests 100% OK!\n")
```



2.7 คู่มือใช้งานห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands (unit 2)

เพื่อส่งเสริมการจัดการเรียนรู้เชิงรุก (Active Learning) ผู้เรียนสามารถเข้าสู่ชุดปฏิบัติการจำลองเสมือนจริงทั้งในรูปแบบ 2D Interactive Canvas และ 3D AR MediaPipe Spatial View ผ่านเว็บเบราว์เซอร์บน Global CDN โดยไม่ต้องติดตั้งโปรแกรมใดๆ เพิ่มเติม โดยมีคู่มือปฏิบัติการฉบับสมบูรณ์ที่ lab_manual_chapter02_ar_mediapipe.md:

ชุด 5 ห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands ประจำปีที่ 2

2.0

Mars Unit Converter & Flow (จำลองวิถียาน Mars Climate Orbiter)

ป้องกันบั๊กแปลงหน่วยวัด lbf-s สู N-s ด้วยผังงาน ISO 5807 พร้อมกราฟิก 3D Trajectory

2.1

Pseudocode AST Linter & Tree Visualizer

ระบบตรวจจับข้อผิดพลาดไวยากรณ์รหัสจำลอง และเรนเดอร์โครงสร้างต้นไม้ 3D AST ในอากาศ

2.2

Interactive Flowchart Builder Studio

สตูดิโอประกอบบล็อกผังงานมาตรฐาน ISO 5807 และปล่อยสำเนาเอกสารทดสอบการไหลของข้อมูล

2.3

Multi-Branch Decision Gates (โรงเรือนเกษตรอัจฉริยะ)

ระบบตัดสินใจหลายเงื่อนไขควบคุมอุณหภูมิและความชื้น พร้อมระบบควบคุมไร้สัมผัส MediaPipe

2.4

Animated Trace Table Runner

ตารางไล่ค่าตัวแปร (Dry Run) แบบสด ควบคู่กับการแสดงสถานะลูกบาศก์หน่วยความจำ RAM 3D

ภาพที่ 2.7 แผนผังสถาปัตยกรรมชุด 5 ห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands ประจำปีที่ 2

ตารางสรุป 5 ปฏิบัติการเสมือนจริงและสาระสำคัญประจำปีที่ 2

รหัส แล็บ	ชื่อการทดลอง (Experiment Title)	วัตถุประสงค์เชิงพฤติกรรม	รูปแบบการ จำลอง	ตัวแปรและข้อมูลที่บันทึก	ผลสรุปทางวิทยาการคำนวณ
LAB 2.0	แบบจำลองการแปลงหน่วยวิถี ยาน Mars	ศึกษาความสำคัญของความถูกต้อง เชิงขั้นตอนวิธี	2D/3D AR	แรงขับ (lbf/N), ความสูง h , ผล การเข้าวงโคจร	การแปลงหน่วยช่วยให้ยานเข้าวงโคจรที่ 193 km ปลอดภัย
LAB 2.1	ตัววิเคราะห์รหัสจำลองและ ต้นไม้ AST	ตรวจสอบไวยากรณ์และบล็อกคำสั่ง	2D/3D AR	คำสั่งที่ป้อน, Error Code, สถานะ 3D Tree	การปิดบล็อก (<code>ENDIF</code>) ชัดเจนป้องกัน ความกำกวมของโปรแกรม
LAB 2.2	สตูดิโอสร้างผังงานมาตรฐาน ISO	ประกอบผังงานและตรวจสอบ FlowLine	2D/3D AR	ลำดับบล็อก, เส้นทางเลขอร์, ผลลัพธ์ Output	ผังงานช่วยให้ตรวจสอบเงื่อนไขผ่าน/ตก เกณฑ์ได้ครบถ้วน
LAB 2.3	การตัดสินใจหลายเงื่อนไข Smart Farm	จำลองตรรกะควบคุมสภาพ แวดล้อมอัตโนมัติ	2D/3D AR	อุณหภูมิ, ความชื้น, ระดับน้ำ, ปริมาณหมอก	เงื่อนไขความปลอดภัยป้องกันบิมน้ำเสียหาย เมื่อน้ำหมดถัง
LAB 2.4	การไล่ค่าตัวแปรด้วยตาราง นิเมชัน	ตรวจสอบความถูกต้องของลูปด้วย Dry Run	2D/3D AR	ตัวแปร i , Sum , เงื่อนไข $i \leq$ 5, ลูปสเต็ป	ได้ผลรวม $Sum = 15$ ตรงตามสูตรอนุกรม คณิตศาสตร์ $\frac{n(n+1)}{2}$

2.8 สรุปสาระสำคัญประจำปี

- ขั้นตอนวิธี เป็นหัวใจของการแก้ปัญหาเชิงคำนวณที่ต้องมีคุณสมบัติ 5 ประการ ได้แก่ ข้อมูลเข้าชัดเจน, ผลลัพธ์ตรงเป้าหมาย, ไม่กำกวม, มีจุดสิ้นสุด, และปฏิบัติได้จริง
- รหัสจำลอง เป็นเครื่องมือสื่อสารระหว่างมนุษย์ด้วยคำสำคัญสากล (`INPUT` , `OUTPUT` , `IF-THEN` , `WHILE` , `FOR`) ขณะที่ ผังงาน ใช้ภาษาภาพตามมาตรฐาน ISO 5807 เพื่อแสดงทิศทางการไหลของข้อมูล
- โครงสร้างควบคุม 3 รูปแบบ (เรียงลำดับ, ทางเลือก, วงซ้ำ) เพียงพอที่จะใช้สร้างอัลกอริทึมทุกชนิดในโลกตามทฤษฎีบท Böhm-Jacopini
- **ตารางทรายรัน ** เป็นเครื่องมือสำคัญที่สุดในการตรวจสอบความถูกต้องและกำจัดบั๊ก (Debugging) ก่อนลงมือเขียนโค้ดจริง

2.9 แบบฝึกหัดทบทวนและโจทย์ประเมินผลสัมฤทธิ์

◆ ตอนที่ 1 ความรู้ความเข้าใจพื้นฐาน

- จงอธิบายความหมายและความแตกต่างของสัญลักษณ์ผังงาน **Process** (สี่เหลี่ยมผืนผ้า), **Input/Output** (สี่เหลี่ยมด้านขนาน), และ **Decision** (สี่เหลี่ยมข้าวหลามตัด)
- จงระบุคุณสมบัติ 5 ประการของขั้นตอนวิธีที่ดี พร้อมอธิบายว่าเหตุใดคุณสมบัตินี้ "Finiteness (ความมีจุดสิ้นสุด)" จึงสำคัญอย่างยิ่งในระบบยานอวกาศ
- จงเขียนรหัสจำลอง สำหรับรับค่ารัศมีของวงกลม (r) และคำนวณหาพื้นที่วงกลม ($Area = \pi r^2$)

◆ ตอนที่ 2 การวิเคราะห์และประยุกต์ใช้

- **ระบบตรวจวัดดัชนีมวลกาย **
 - กำหนดสูตรคำนวณ: $BMI = \frac{Weight (kg)}{(Height (m))^2}$
 - เกณฑ์จำแนก: $BMI < 18.5$ (น้ำหนักน้อย), $18.5 \leq BMI < 23.0$ (สมส่วน), $23.0 \leq BMI < 25.0$ (น้ำหนักเกิน), $BMI \geq 25.0$ (อ้วน)
 - จงเขียนรหัสจำลองและวาดผังงาน ISO 5807 ของระบบนี้
- จงสร้างตาราง Trace Table สำหรับติดตามการทำงานของอัลกอริทึมต่อไปนี้ เมื่อป้อนค่า $N = 4$:

```
START
INPUT N
SET factorial = 1
FOR i = 1 TO N DO
    SET factorial = factorial * i
ENDFOR
OUTPUT factorial
END
```

◆ ตอนที่ 3 การสังเคราะห์และคิดขั้นสูง

6. จงเขียนโปรแกรมภาษา Python 3.11 รับค่าตัวเลขจำนวนเต็ม N แล้วแสดงผลลัพท์ว่า N เป็น "จำนวนเฉพาะ (Prime Number)" หรือไม่ โดยใช้โครงสร้าง While Loop พร้อมสร้างตาราง Trace Table แสดงการทดสอบตัวหารทุกตัวตั้งแต่ 2 ถึง $\sqrt{N}$



เอกสารอ้างอิงประจำบท

1. Böhm, C., & Jacopini, G. (1966). Flow diagrams, turing machines and languages with only two formation rules. *Communications of the ACM*, 9(5), 366–371.
2. International Organization for Standardization. (1985). *Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts* (ISO Standard No. 5807:1985).
3. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
4. NASA. (1999). *Mars Climate Orbiter Mishap Investigation Board Phase I Report*. National Aeronautics and Space Administration.
5. Thassana, C. (2026). *Computational Thinking and Applied Artificial Intelligence for Science Education*. Rambhai Barni Rajabhat University Press.

วิทยาการคำนวณ 1 รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ

บทที่ 2 เสาหลักทั้ง 4 ของแนวคิดเชิงคำนวณ

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

🎯 ผลลัพธ์การเรียนรู้ประจำบท

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

1. **จำแนกและอธิบาย** ความหมายและบทบาทของเสาหลักทั้ง 4 ในแนวคิดเชิงคำนวณ (Computational Thinking) ได้อย่างลึกซึ้ง
2. **ประยุกต์ใช้** เทคนิคการย่อยปัญหา และการหารูปแบบ ในการวิเคราะห์ระบบทางวิทยาศาสตร์และชีวิตประจำวัน
3. **สร้างแบบจำลองนามธรรม** โดยคัดกรองเฉพาะข้อมูลสำคัญและตัดรายละเอียดส่วนเกินออกได้อย่างเหมาะสม
4. **ออกแบบขั้นตอนวิธี** ที่มีลำดับคำสั่งชัดเจน แม่นยำ และแปลงเป็นโค้ด Python เพื่อแก้ไขปัญหาได้อย่างสมบูรณ์

🚉 2.0 เรื่องเล่าเปิดบทเรียน มนุษย์คิดอย่างไรเมื่อสร้างแผนที่ดาวและระบบรถไฟฟ้

ลองมองดู แผนที่เส้นทางรถไฟฟ้ BTS/MRT หรือ **แผนที่เครือข่ายรถไฟใต้ดินกรุงลอนดอน** ที่ออกแบบโดย แฮร์รี เบก (Harry Beck) ในปี ค.ศ. 1931

แผนที่ที่ไม่ได้แสดงความโค้งงอของถนน ไม่ได้ระบุระยะทางตามมาตราส่วนทางภูมิศาสตร์จริง และไม่ได้บอกวาสถานีโดยอยู่ใต้ดินใด แต่กลับเป็นแผนที่ที่ **คนนับล้าน** คนใช้งานได้ง่ายที่สุดในโลก เพราะมันใช้ **การคิดเชิงนามธรรม** สกัดเอาเฉพาะ ลำดับสถานี และ "จุดเชื่อมต่อ (Interchanges)" ออกมาเป็นเส้นตรงและมุม 45° ที่เรียบง่าย

graph LR

RealWorld["โลกจริงอันซับซ้อน: ถนนคดเคี้ยว อาคาร แม่น้ำ ความลึก"] -- "การคิดเชิงนามธรรม (Abstraction)" --> Model["แบบจำลองที่ใช้งานง่าย: จุดเชื่อมต่อ"]

กระบวนการนี้คือหัวใจสำคัญของ **แนวคิดเชิงคำนวณ** ซึ่งไม่ใช่การคิดให้เหมือนเครื่องจักร แต่คือ **การคิดแก้ปัญหาในระดับที่คอมพิวเตอร์หรือมนุษย์สามารถนำไปปฏิบัติตามได้อย่างมีประสิทธิภาพ**

🏛️ 2.1 เสาหลักทั้ง 4 ของแนวคิดเชิงคำนวณ

graph TD

CT["แนวคิดเชิงคำนวณ (Computational Thinking)"]

CT → P1["1. การแยกส่วนประกอบและการย่อยปัญหา (Decomposition) - แดกปัญหาใหญ่เป็นปัญหาย่อยๆ"]

CT → P2["2. การหารูปแบบและความเหมือน (Pattern Recognition) - สังเกตแนวโน้มและความซ้ำซ้อน"]

CT → P3["3. การคิดเชิงนามธรรม (Abstraction) - ตัดสิ่งไม่จำเป็น คัดเฉพาะแก่นสำคัญ"]

CT → P4["4. การออกแบบขั้นตอนวิธี (Algorithm Design) - กำหนดลำดับคำสั่งที่ละเอียดชัดเจน"]

เสาหลักที่ 1 การแยกส่วนประกอบและการย่อยปัญหา

การแบ่งปัญหา ระบบ หรือวัตถุที่มีความซับซ้อนออกเป็นส่วนประกอบย่อยๆ (Sub-problems / Components) เพื่อให้ง่ายต่อการทำความเข้าใจ ออกแบบ และแก้ไข

- ตัวอย่างทางวิทยาศาสตร์
 - การศึกษากระบวนการนิวเคลียร์ฟิวชัน → แยกศึกษาเป็น: สภาพดินและน้ำ, สปีชีส์ของพืช (โกกาง/แสม), สปีชีส์ของสัตว์ (ปูก้ามดาบ/ปลาตีน), และห่วงโซ่อาหาร
- ตัวอย่างการพัฒนาซอฟต์แวร์
 - ระบบแอปพลิเคชันซื้อขายออนไลน์ → แยกเป็น: ระบบค้นหาสินค้า, ระบบตะกร้าสินค้า, ระบบชำระเงิน, และระบบติดตามพัสดุ

เสาหลักที่ 2 การหารูปแบบและความเหมือน

การค้นหาคำเหมือน ความคล้ายคลึง แนวโน้ม (Trends) หรือความสัมพันธ์ที่เกิดขึ้นซ้ำๆ ในชุดข้อมูลหรือปัญหาที่เคยพบมาก่อน เพื่อนำวิธีการแก้ปัญหาเดิมมาปรับใช้ซ้ำ (Reusability)

- ตัวอย่างทางฟิสิกส์และคณิตศาสตร์
 - การตกของผลแอปเปิล, การโคจรของดวงจันทร์รอบโลก, และการเหวี่ยงของลูกตุ้ม ล้วนมีรูปแบบความเร่งภายใต้ **กฎแรงโน้มถ่วงสากล** ($F = G \frac{m_1 m_2}{r^2}$) เหมือนกัน
- ตัวอย่างในโปรแกรมมิ่ง
 - การคำนวณดอกเบี้ยทบต้น, การคำนวณการสลายตัวของสารกัมมันตรังสี, และการเติบโตของแบคทีเรีย ล้วนเป็นรูปแบบ **ฟังก์ชันเอกซ์โพเนนเชียล** ($y = a \cdot e^{kt}$)

เสาหลักที่ 3 การคิดเชิงนามธรรม

การคัดกรองสาระสำคัญ มุ่งเน้นเฉพาะข้อมูลที่จำเป็นต่อการแก้ปัญหา และละทิ้งรายละเอียดปลีกย่อยที่ไม่เกี่ยวข้องออกไป เพื่อสร้างเป็น **แบบจำลอง** หรือ **แม่แบบ**

การเปรียบเทียบระหว่างโลกจริงกับแบบจำลองนามธรรม

- ในวิชาฟิสิกส์: เมื่อเราคำนวณวิถีโพรเจกไทล์ของลูกฟุตบอล เราถือว่าลูกบอลเป็น "มวลจุด (Point Mass)" และตัดสี่ของลูกบอล ยี่ห้อ หรือรอยเปื้อนออกไป
- ในวิชาวิทยาการคำนวณ: เมื่อสร้างระบบจัดการข้อมูลนักเรียน เราเก็บเฉพาะ "รหัสนักศึกษา, ชื่อ, เกรดเฉลี่ย" โดยไม่ต้องบันทึกยี่ห้อรองเท้าที่นักเรียนใส่

เสาหลักที่ 4 การออกแบบขั้นตอนวิธี

การพัฒนาชุดคำสั่งหรือลำดับขั้นตอนการปฏิบัติงานทีละขั้น (Step-by-step instructions) ที่มีความชัดเจน ไม่กำกวม และนำไปสู่ผลลัพธ์ที่ต้องการเสมอ

- คุณสมบัติที่ดีของขั้นตอนวิธี
 - มีจุดเริ่มต้นและจุดสิ้นสุดที่ชัดเจน (Finiteness)
 - แต่ละขั้นตอนต้องแม่นยำ ปฏิบัติตามได้จริง (Definiteness & Feasibility)
 - รับข้อมูลเข้าและให้ผลลัพธ์ที่ถูกต้อง (Input & Output Correctness)

2.2 ตัวอย่างการประยุกต์ใช้เสาหลักทั้ง 4 การออกแบบระบบคัดแยกขยะอัจฉริยะ

ตัวอย่างที่ 2.1: การพัฒนาระบบคัดแยกขยะรีไซเคิลด้วยแนวคิดเชิงคำนวณ

บูรณาการสิ่งแวดล้อม

โจทย์ปัญหา: โรงเรียนต้องการสร้างถังขยะอัจฉริยะที่สามารถตรวจจับและคัดแยกขยะออกเป็น 3 ประเภท ได้แก่ ขยะพลาสติกใส, กระป๋องโลหะ, และเศษกระดาษ

```
<details style="background: #f8f8fc; border: 1px solid #cbd5e1; border-radius: 10px; padding: 14px 18px; margin-top: 12px"
<summary style="font-weight: 700; color: #059669;">คลิกเพื่อดูการประยุกต์ใช้เสาหลักทั้ง 4</summary>
<div style="margin-top: 14px; border-top: 1px dashed #cbd5e1; padding-top: 14px;">
  <ol style="padding-left: 20px;">
    <li><strong>Decomposition (ย่อยปัญหา):</strong> แยกปัญหาออกเป็น 3 ส่วน
    <ul>
      <li>ส่วนตรวจจับคุณสมบัติภาพ (น้ำหนัก, ความโปร่งแสง, การนำไฟฟ้า)</li>
      <li>ส่วนประมวลผลและตัดสินใจ (Microcontroller Logic)</li>
      <li>ส่วนกลไกเปิด-ปิดฝาถัง (Servo Motor Actuators)</li>
    </ul>
    </li>
    <li><strong>Pattern Recognition (หารูปแบบ):</strong>
    <ul>
      <li>กระป๋องโลหะ: มีการนำไฟฟ้า ($Conductivity = True$) และน้ำหนักปานกลาง</li>
      <li>ขวดพลาสติกใส: แสงทะลุผ่านได้ ($Translucency > 80\%$) และไม่นำไฟฟ้า</li>
      <li>เศษกระดาษ: แสงทะลุผ่านไม่ได้ และไม่นำไฟฟ้า</li>
    </ul>
    </li>
    <li><strong>Abstraction (คิดเชิงนามธรรม):</strong>
    <ul>
      <li>สนใจเฉพาะ: ค่าเซนเซอร์นำไฟฟ้า (0/1), ค่าเซนเซอร์แสง (0-100), น้ำหนัก (g)</li>
      <li>ตัดทิ้ง: สีของฉลาก, รอยยับของขวด, ราคาของขยะ</li>
    </ul>
    </li>
    <li><strong>Algorithm Design (ขั้นตอนวิธี):</strong>
    <ul>
      <li>1. อ่านค่าเซนเซอร์ทั้ง 3 ตัว</li>
      <li>2. ถ้า $Conductivity = 1$ $\rightarrow$ ส่งมอเตอร์เปิดช่อง ถังโลหะ</li>
      <li>3. ถ้าไม่ใช่ และ $LightIntensity > 80$ $\rightarrow$ ส่งมอเตอร์เปิดช่อง ถังพลาสติก</li>
      <li>4. กรณีอื่นๆ $\rightarrow$ ส่งมอเตอร์เปิดช่อง ถังกระดาษ/ทั่วไป</li>
    </ul>
    </li>
  </ol>
</div>
</details>
```

```
# computational_thinking_waste_sorter.py
# โปรแกรมจำลองการคัดแยกขยะอัจฉริยะด้วยแนวคิดเชิงคำนวณ 4 เสาสหลัก
# ผู้เขียน: ผศ.ดร.ชิวะ ทิศนา (มรภ.รำไพพรรณี)

class WasteItem:
    """Abstraction: สร้างแบบจำลองวัตถุขยะโดยสนใจเฉพาะคุณสมบัติที่จำเป็น"""
    def __init__(self, name: str, is_conductive: bool, light_transmittance: float, weight_grams: float):
        self.name = name
        self.is_conductive = is_conductive # การนำไฟฟ้า (โลหะ)
        self.light_transmittance = light_transmittance # การส่องผ่านของแสง (%)
        self.weight_grams = weight_grams # น้ำหนัก (กรัม)

def sort_waste_algorithm(item: WasteItem) -> str:
    """Algorithm Design: ขั้นตอนวิธีในการตัดสินใจคัดแยกขยะลงถังที่ถูกต้อง"""
    if item.is_conductive:
        return "🗑️ ถังขยะโลหะ (Metal Bin)"
    elif item.light_transmittance >= 75.0:
        return "🗑️ ถังขยะพลาสติกใสรีไซเคิล (Plastic PET Bin)"
    elif item.weight_grams < 50.0 and item.light_transmittance < 30.0:
        return "🗑️ ถังขยะกระดาษ (Paper Bin)"
    else:
        return "🗑️ ถังขยะทั่วไป (General Waste Bin)"

def main():
    # ข้อมูลทดสอบ (Test Dataset)
    sample_wastes = [
        WasteItem("กระป๋องน้ำอัดลมอลูมิเนียม", is_conductive=True, light_transmittance=0.0, weight_grams=15.0),
        WasteItem("ขวดน้ำดื่ม PET ใส", is_conductive=False, light_transmittance=88.5, weight_grams=20.0),
        WasteItem("ใบปลิวกระดาษ A4", is_conductive=False, light_transmittance=10.0, weight_grams=5.0),
        WasteItem("เปลือกกล้วยหอม", is_conductive=False, light_transmittance=0.0, weight_grams=120.0),
    ]

    print("=" * 70)
    print("🤖 ระบบจำลองถังขยะอัจฉริยะ (Smart Waste Classifier Simulator)")
    print("=" * 70)

    for item in sample_wastes:
        assigned_bin = sort_waste_algorithm(item)
        print(f"🗑️ วัตถุ: {item.name:<25} | ผลลัพธ์: {assigned_bin}")

    print("=" * 70)

if __name__ == "__main__":
    main()
```

2.4 กิจกรรมปฏิบัติการแสบ Unplugged Master Chef Algorithm

วัตถุประสงค์

ฝึกการย่อยปัญหาและออกแบบขั้นตอนวิธีการทำอาหารจานโปรดให้ละเอียดจนผู้อื่นสามารถทำตามแล้วได้รสชาติเหมือนกัน 100%

ขั้นตอนกิจกรรม

- ให้นักเรียนเลือกเมนูอาหาร 1 ชนิด (เช่น ต้มยำกุ้งน้ำข้น)
- ใช้ **Decomposition** แยกรายการวัตถุดิบ, เครื่องปรุง, และอุปกรณ์เครื่องครัว
- ใช้ **Pattern Recognition** สังเกตขั้นตอนที่มีรูปแบบรวมกับการทำต้มจืด (เช่น การต้มน้ำให้เดือดก่อนใส่เนื้อสัตว์)
- ใช้ **Abstraction** ตัดรายละเอียดที่ไม่เกี่ยวข้องออก (เช่น ยี่ห้อของหม้อ, สีของจาน)
- ใช้ **Algorithm Design** เขียนเป็นขั้นตอนแบบลำดับและมีเงื่อนไข เช่น ถ้ากุ้งเปลี่ยนเป็นสีส้ม ให้ปิดไฟทันที

2.5 สรุปใจความสำคัญและแบบฝึกหัดท้ายบทที่ 2

สรุปประเด็นสำคัญ

- Decomposition** ช่วยให้เราไม่ตื่นตระหนกกับปัญหาใหญ่ โดยซอยย่อยเป็นงานเล็กๆ ที่จัดการได้
- Pattern Recognition** ทำให้เราไม่ต้องคิดค้นวิธีแก้ปัญหาใหม่ตั้งแต่ศูนย์ทุกครั้ง
- Abstraction** ช่วยประหยัดทรัพยากรความคิดและหน่วยความจำคอมพิวเตอร์ โดยมองเห็นเฉพาะแก่นของปัญหา
- Algorithm Design** เปลี่ยนแนวคิดนามธรรมให้กลายเป็นชุดคำสั่งที่ปฏิบัติการได้จริง

แบบฝึกหัดทบทวน 3 ระดับ

ระดับที่ 1 ความรู้ความเข้าใจพื้นฐาน

- จงอธิบายว่าการที่แพทย์วินิจฉัยโรคของผู้ป่วยจากอาการไข้ ไอ เจ็บคอ จัดเป็นการใช้เสาสหลักใดของแนวคิดเชิงคำนวณ เพราะเหตุใด
- ในการสร้างโปรแกรมคำนวณเกรดเฉลี่ยสะสม (GPAX) ของนักศึกษา ข้อมูลใดต่อไปนี้ เป็นข้อมูลสำคัญ (Essential) และข้อมูลใดควรถูกตัดทิ้งด้วยหลัก Abstraction:
 - รหัสนักศึกษา, หน่วยกิต, เกรดที่ได้, ยี่ห้อโน้ตบุ๊กของนักศึกษา, เวลาที่เข้าเรียนคาบแรก

ระดับที่ 2 การวิเคราะห์และประยุกต์ใช้

- ให้นักเรียนใช้หลักการ **Decomposition** และ **Abstraction** วิเคราะห์ระบบ "ตู้จำหน่ายเครื่องดื่มอัตโนมัติ (Vending Machine)" โดยแจกแจงส่วนประกอบหลัก 4 ส่วน พร้อมระบุข้อมูลเข้า (Input) และข้อมูลออก (Output) ของแต่ละส่วน

ระดับที่ 3 การคิดขั้นสูงและบูรณาการ

- จงออกแบบขั้นตอนวิธี สำหรับการจัดคิวผู้ป่วยในห้องฉุกเฉินโรงพยาบาล (Triage Algorithm) โดยใช้เสาหลักทั้ง 4 เพื่อจัดลำดับความเร่งด่วน 3 ระดับ (วิกฤตสีแดง, เร่งด่วนสีเหลือง, ไม่รุนแรงสีเขียว)

วิทยาการคำนวณ 1 รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ

บทที่ 3 การเขียนรหัสจำลองและผังงานมาตรฐาน

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนาศาสตร์ • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

ผลลัพธ์การเรียนรู้ประจำบท

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- **อธิบาย** ความหมาย วัตถุประสงค์ และประโยชน์ของการใช้รหัสจำลอง และผังงาน ในการสื่อสารขั้นตอนวิธี
- **เลือกใช้และวาด** สัญลักษณ์ผังงานตามมาตรฐานสากล ANSI/ISO ได้อย่างถูกต้องครบถ้วน
- **แปลง** โจทย์ปัญหาทางคณิตศาสตร์และวิทยาศาสตร์ให้อยู่ในรูปโครงสร้างควบคุม 3 รูปแบบ (เรียงลำดับ, ทางเลือก, ทำซ้ำ)
- **ตรวจสอบความถูกต้อง** ของขั้นตอนวิธีด้วยตารางตรวจสอบการทำงาน (Trace Table / Dry Run) ได้อย่างแม่นยำ

3.0 เรื่องเล่าเปิดบทเรียน การกิจพิชิตอวกาศกับความผิดพลาดระดับพันล้าน

ในปี ค.ศ. 1999 องค์การ NASA ต้องสูญเสียยานสำรวจดาวอังคาร Mars Climate Orbiter มูลค่ากว่า 125 ล้านดอลลาร์สหรัฐไปอย่างน่าเสียดาย สาเหตุเกิดจากความผิดพลาดง่าย ๆ ในการส่งผ่านข้อมูลแรงขับเคลื่อนระหว่าง 2 หน่วยงาน โดยหน่วยงานหนึ่งส่งข้อมูลในหน่วย ปอนด์-วินาที (Imperial Unit) แต่อีกหน่วยงานหนึ่งเขียนโปรแกรมรับข้อมูลโดยคิดว่าเป็นหน่วย นิวตัน-วินาที (Metric SI Unit)

graph LR

TeamA["หน่วยงาน A (Lockheed Martin)\nค่าแรงขับเคลื่อน: ปอนด์-แรง-วินาที (lbf·s)"] -- "ส่งผ่านข้อมูลไม่มีการระบุหน่วย" --> TeamB["หน่วยงาน B (NASA JP)\nTeamB --> Crash["ยานเสียทิศทาง พุ่งชนชั้นบรรยากาศดาวอังคาร!"]

เหตุการณ์นี้กลายเป็นบทเรียนครั้งประวัติศาสตร์ของวงการวิทยาการคอมพิวเตอร์ ที่ย้ำเตือนว่า การมีขั้นตอนวิธีที่ชัดเจน มีการระบุข้อมูลเข้า ข้อมูลออก หน่วย และเงื่อนไขอย่างรัดกุมผ่านรหัสจำลองและผังงานมาตรฐาน เป็นสิ่งจำเป็นยิ่งยวดก่อนที่จะเริ่มลงมือเขียนโค้ดภาษาคอมพิวเตอร์จริง

3.1 การเขียนรหัสจำลองตามหลักสากล

รหัสจำลอง คือ การอธิบายขั้นตอนวิธีของการแก้ปัญหาด้วยภาษาที่กึ่งภาษาธรรมชาติและกึ่งภาษาโปรแกรม โดยไม่ยึดติดกับไวยากรณ์ (Syntax) ของภาษาคอมพิวเตอร์ภาษาใดภาษาหนึ่งโดยเฉพาะ

คำสำคัญมาตรฐานสากล

คำสำคัญ (Keyword)	ความหมาย	ตัวอย่างการใช้งาน
INPUT / READ	การรับข้อมูลเข้าจากภายนอก	INPUT radius
OUTPUT / PRINT	การแสดงผลหรือออกสู่หน้าจอ	PRINT "Area = ", area
COMPUTE / SET	การคำนวณและกำหนดค่าตัวแปร	SET area = 3.14159 * radius * radius
IF ... THEN ... ELSE	โครงสร้างทางเลือกการตัดสินใจ	IF score ≥ 50 THEN PRINT "Pass" ELSE PRINT "Fail"
WHILE ... DO	โครงสร้างการวนซ้ำแบบเช็คก่อนทำ	WHILE count < 10 DO ... ENDWHILE
FOR ... TO	โครงสร้างการวนซ้ำตามจำนวนรอบ	FOR i = 1 TO 10 DO ... ENDFOR

3.2 สัญลักษณ์ผังงานมาตรฐานสากล

ผังงาน คือ แผนภาพที่ใช้รูปทรงเรขาคณิตมาตรฐานสากลในการแสดงลำดับขั้นตอนการทำงานของระบบหรือโปรแกรม

graph TD

subgraph FlowchartSymbols["สัญลักษณ์ผังงานมาตรฐาน ANSI/ISO"]
T["1. Terminator (วงรีมน/แคปซูล)\nจุดเริ่มต้น / จุดสิ้นสุด (Start/Stop)"]
P["2. Process (สี่เหลี่ยมผืนผ้า)\nการประมวลผล / การคำนวณ / กำหนดค่า"]
IO["3. Input/Output (สี่เหลี่ยมด้านขนาน)\nการรับข้อมูลเข้า หรือแสดงผล"]
D["4. Decision (สี่เหลี่ยมข้าวหลามตัด)\nการตัดสินใจ / ตรวจสอบเงื่อนไข (Yes/No)"]
C["5. Connector (วงกลมเล็ก)\nจุดเชื่อมต่อภายในหน้าเดียวกัน"]
end

กฎเหล็กในการเขียนผังงานที่ดี (Flowchart Best Practices)

- ผังงานต้องมีจุด Start เพียงจุดเดียว และจุด End/Stop ที่ชัดเจน
- ทิศทางการไหลของข้อมูลควรอ่านจาก บนลงล่าง (Top-to-Bottom) และจาก ซ้ายไปขวา (Left-to-Right)

- สัญลักษณ์การตัดสินใจ (Decision) ต้องมี เส้นทางออกอย่างน้อย 2 ทางเสมอ (True/False หรือ Yes/No)
- หลีกเลี่ยงการลากเส้น Flowline ตัดข้ามกัน (หากจำเป็นให้ใช้สัญลักษณ์ Connector แทน)



3.3 โครงสร้างการควบคุมหลัก 3 รูปแบบ

1. โครงสร้างแบบเรียงลำดับ

การทำงานที่ละคำสั่งตามลำดับจากบนลงล่าง โดยไม่มีการข้ามขั้นตอนหรือวนกลับ

```
graph TD
    Start1["เริ่มต้น (Start)"] --> In1["รับค่าความยาวฐาน (base) และส่วนสูง (height)"]
    In1 --> Cal1["คำนวณ area = 0.5 * base * height"]
    Cal1 --> Out1["แสดงผลลัพธ์ area"]
    Out1 --> End1["สิ้นสุด (End)"]
```

2. โครงสร้างแบบทางเลือก

การตรวจสอบเงื่อนไขทางตรรกะเพื่อเลือกดำเนินงานในเส้นทางใดเส้นทางหนึ่ง

```
graph TD
    Start2["เริ่มต้น (Start)"] --> In2["รับค่าอุณหภูมิ (temp)"]
    In2 --> Dec2["{temp > 100 ?}"]
    Dec2 -- "ใช่ (True)" --> P_Boil["สถานะ: น้ำเดือดกลายเป็นไอ"]
    Dec2 -- "ไม่ใช่ (False)" --> Dec2_2["{temp <= 0 ?}"]
    Dec2_2 -- "ใช่ (True)" --> P_Freeze["สถานะ: น้ำแข็งตัวเป็นของแข็ง"]
    Dec2_2 -- "ไม่ใช่ (False)" --> P_Liquid["สถานะ: น้ำเป็นของเหลว"]
    P_Boil --> End2["สิ้นสุด (End)"]
    P_Freeze --> End2
    P_Liquid --> End2
```

3. โครงสร้างแบบวนซ้ำ

การทำงานในชุดคำสั่งเดิมซ้ำหลายรอบ จนกระทั่งเงื่อนไขที่กำหนดไม่เป็นจริง

```
graph TD
    Start3["เริ่มต้น (Start)"] --> Init3["กำหนด sum = 0, count = 1"]
    Init3 --> Cond3["{count <= 10 ?}"]
    Cond3 -- "ใช่ (True)" --> LoopBody["คำนวณ sum = sum + count\nเพิ่มค่า count = count + 1"]
    LoopBody --> Cond3
    Cond3 -- "ไม่ใช่ (False)" --> Out3["แสดงผลรวม sum"]
    Out3 --> End3["สิ้นสุด (End)"]
```



3.4 ตัวอย่างโจทย์คำนวณและการตรวจสอบด้วยตารางตรรกะ



ตัวอย่างที่ 3.1: ขั้นตอนวิธีหาค่าตัวหารร่วมมาก (Greatest Common Divisor - GCD) แบบยุคลิด

ขั้นตอนวิธีคลาสสิก

รหัสจำลอง (Euclidean Algorithm Pseudocode):

```
START
INPUT a, b
WHILE b != 0 DO
    SET remainder = a MOD b
    SET a = b
    SET b = remainder
ENDWHILE
OUTPUT "GCD = ", a
END
```

```
<p><strong>ตารางตรวจสอบการทำงาน (Trace Table) เมื่อทดสอบกับค่า $a = 48, b = 18$:</strong></p>
<table style="width:100%; border-collapse:collapse; margin-top:10px; text-align:center;">
  <thead>
    <tr style="background:#1e293b; color:#f8fafc;">
      <th style="padding:8px; border:1px solid #475569;">รอบที่ (Iteration)</th>
      <th style="padding:8px; border:1px solid #475569;">ตัวแปร a</th>
      <th style="padding:8px; border:1px solid #475569;">ตัวแปร b</th>
      <th style="padding:8px; border:1px solid #475569;">เงื่อนไข (b ≠ 0)</th>
      <th style="padding:8px; border:1px solid #475569;">remainder = a % b</th>
      <th style="padding:8px; border:1px solid #475569;">ค่าใหม่ a, b</th>
    </tr>
  </thead>
  <tbody>
    <tr style="background:#f8fafc;">
      <td style="padding:8px; border:1px solid #cbd5e1;">เริ่มต้น</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">48</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">18</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">True</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">48 % 18 = 12</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">a=18, b=12</td>
    </tr>
    <tr>
      <td style="padding:8px; border:1px solid #cbd5e1;">รอบที่ 1</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">18</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">12</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">True</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">18 % 12 = 6</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">a=12, b=6</td>
    </tr>
    <tr>
      <td style="padding:8px; border:1px solid #cbd5e1;">รอบที่ 2</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">12</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">6</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">True</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">12 % 6 = 0</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">a=6, b=0</td>
    </tr>
    <tr>
      <td style="padding:8px; border:1px solid #cbd5e1;">รอบที่ 3</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">6</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">0</td>
      <td style="padding:8px; border:1px solid #cbd5e1; color:#ef4444; font-weight:700;">False (จบ)</td>
      <td style="padding:8px; border:1px solid #cbd5e1;">—</td>
      <td style="padding:8px; border:1px solid #cbd5e1; color:#10b981; font-weight:700;">ผลลัพธ์ GCD = 6</td>
    </tr>
  </tbody>
</table>
```



3.5 โค้ดคอมพิวเตอร์ภาษา Python ที่ตรงกับผังงาน

```
# euclidean_gcd_algorithm.py
# โปรแกรมหา ห.ร.ม. (GCD) ตามขั้นตอนวิธีของยุคลิด
# ผู้เขียน: ผศ.ดร.จีระ หัตถ์นา (มรภ.รำไพพรรณี)

def compute_gcd(a: int, b: int) -> int:
    """คำนวณค่า ห.ร.ม. ด้วย Euclidean Algorithm พร้อมพิมพ์ขั้นตอนการทำงาน"""
    print(f"🔴 เริ่มต้นคำนวณ ห.ร.ม. ของ {a} และ {b}")
    step = 1
    while b != 0:
        remainder = a % b
        print(f"   ขั้นตอนที่ {step}: {a} = ({b} * {a // b}) + เศษ {remainder}")
        a = b
        b = remainder
        step += 1
    print(f"🟢 ผลลัพธ์สุดท้าย ห.ร.ม. คือ: {a}\n")
    return a

if __name__ == "__main__":
    test_num1 = 48
    test_num2 = 18
    gcd_result = compute_gcd(test_num1, test_num2)

    # ทดสอบกรณีอื่นเพิ่มเติม
    compute_gcd(1071, 462)
```

📌 สรุปประเด็นสำคัญ

- รหัสสล็อต ช่วยให้เราโฟกัสกับโครงสร้างตรรกะ โดยไม่ต้องกังวลเรื่องการพิมพ์ผิดไวยากรณ์ภาษาโปรแกรม
- ผังงาน ให้ภาพรวมทิศทางการทำงานของระบบที่มองเห็นได้ชัดเจน เหมาะสำหรับการสื่อสารระหว่างนักพัฒนากับผู้ใช้งาน
- Trace Table คือเครื่องมือทรงพลังที่สุดในการค้นหาข้อผิดพลาดทางตรรกะ (Logic Errors) ตั้งแต่ก่อนคอมไพล์โค้ด

📄 แบบฝึกหัดทบทวน 3 ระดับ

ระดับที่ 1 ความรู้ความเข้าใจพื้นฐาน

- จงจับคู่สัญลักษณ์ผังงานต่อไปนี้กับหน้าที่การทำงาน
 - (A) สี่เหลี่ยมผืนผ้า, (B) สี่เหลี่ยมข้าวหลามตัด, (C) สี่เหลี่ยมด้านขนาน, (D) วงรีแคบซูล
- จงระบุความแตกต่างระหว่างคำสั่ง `WHILE ... DO` กับ `DO ... WHILE` ในการเขียนรหัสสล็อต

ระดับที่ 2 การวิเคราะห์และประยุกต์ใช้

- จงเขียนรหัสสล็อตและผังงานสำหรับคำนวณรากของสมการกำลังสอง $ax^2 + bx + c = 0$ โดยพิจารณาค่าดิสคริมีแนนต์ $D = b^2 - 4ac$:
 - ถ้า $D > 0$ มี 2 รากจริง: $x = \frac{-b \pm \sqrt{D}}{2a}$
 - ถ้า $D = 0$ มี 1 รากจริง: $x = \frac{-b}{2a}$
 - ถ้า $D < 0$ ไม่มีรากที่เป็นจำนวนจริง (รากเป็นจำนวนเชิงซ้อน)

ระดับที่ 3 การคิดขั้นสูงและบูรณาการ

- จงสร้างผังงานและ Trace Table สำหรับขั้นตอนวิธีตรวจสอบว่าตัวเลขจำนวนเต็มบวก N ที่รับเข้ามาเป็น "จำนวนเฉพาะ (Prime Number)" หรือไม่

วิทยาการคำนวณ 1 รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ

บทที่ 3 พื้นฐานการเขียนโปรแกรมภาษา Python และการจัดการข้อมูลทางวิทยาศาสตร์

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา
สังกัด สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี
เอกสารประกอบรายวิชา 4122104 วิทยาการคำนวณและการแก้ปัญหาเชิงคำนวณ / การสอนวิทยาการคำนวณ

แผนบริหารการสอนประจำบทที่ 3

1. หัวข้อเนื้อหาประจำบท

- เรื่องเล่าเปิดบทเรียนและปรัชญาของภาษาไพทอน กุยโด ฟาน รอสซัม (Guido van Rossum, 1991), ปรัชญา The Zen of Python (PEP 20) และทำไม Python จึงเป็นภาษาหลักของโลก AI และวิทยาศาสตร์ข้อมูล
- **สถาปัตยกรรมพื้นฐานและแบบจำลองหน่วยความจำ** กลไกการอ้างอิงวัตถุ (Object Reference Pointers), ฟังก์ชัน `id()`, Dynamic Typing, และ Garbage Collection
- มาตรฐานการตั้งชื่อตัวแปรตาม PEP 8: กฎการตั้งชื่อ ตัวระบุสากล และข้อควรระวังเกี่ยวกับคำสงวน (Reserved Keywords)
- ชนิดข้อมูลพื้นฐาน 4 ประเภท (Primitive Data Types): จำนวนเต็ม (`int`), ทศนิยม (`float`), สายอักขระ (`str`), และค่าความจริง (`bool`)
- **การแปลงชนิดข้อมูลและการจัดการข้อผิดพลาด** ฟังก์ชัน `int()`, `float()`, `str()`, `bool()` และข้อจำกัดในการแปลง
- ตัวดำเนินการคณิตศาสตร์และลำดับความสำคัญ PEMDAS: การบวก, ลบ, คูณ,หารจริง (`/`), หารปัดเศษลง (`//`), โมดูลโอหาเศษ (`%`), และการยกกำลัง (`**`)
- **การรับเข้าและส่งออกข้อมูลทางวิทยาศาสตร์** ฟังก์ชัน `input()`, `print()`, การจัดรูปแบบสายอักขระขึ้นสูงด้วย Formatted String Literals (f-strings)
- **ตัวอย่างข้อคำนวณและการวิเคราะห์เชิงฟิสิกส์เชิงลึก** การคำนวณพลังงานจลน์ $E_k = \frac{1}{2}mv^2$, การแปลงหน่วยอุณหภูมิ 3 สเกล, และระบบคำนวณกระแสไฟฟ้า
- โค้ดคอมพิวเตอรภาษา Python 3.11 แบบสมบูรณ์: โปรแกรมประมวลผลผลงานและการจัดรูปแบบรายงานวิเคราะห์ข้อมูล
- คู่มือห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe: การจำลองตัวแปรในหน่วยความจำ RAM และการประเมินลำดับนิพจน์แบบปฏิบัติสัมพันธ์ 3 มิติ

2. วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- **อธิบาย** ปรัชญาการออกแบบภาษา Python, กลไก Dynamic Typing และการจัดเก็บข้อมูลในหน่วยความจำ RAM ได้อย่างถูกต้อง
- **จำแนกและเลือกใช้** ชนิดข้อมูล `int`, `float`, `str`, `bool` ให้เหมาะสมกับโจทย์ปัญหาทางวิทยาศาสตร์ได้
- **คำนวณและประเมินค่านิพจน์** ทางคณิตศาสตร์ที่มีความซับซ้อนตามลำดับความสำคัญ PEMDAS ได้อย่างแม่นยำ 100%
- **ประยุกต์ใช้** เทคนิค f-strings ในการจัดรูปแบบการแสดงผลตัวเลขทศนิยมและรายงานเชิงวิทยาศาสตร์ได้อย่างมืออาชีพ
- **สร้างสรรค์** โปรแกรมภาษา Python 3.11 เพื่อรับค่าจากผู้ใช้และคำนวณสมการฟิสิกส์ได้อย่างสมบูรณ์ ไร้ข้อผิดพลาด
- **ปฏิบัติการ** การทดลองเสมือนจริง 3D AR MediaPipe Hands เพื่อควบคุมตัวแปรและดูสถาปัตยกรรมหน่วยความจำแบบโต้ตอบได้

3.0 เรื่องเล่าเปิดบทเรียน จากงานอดิเรกสู่ภาษาสากลแห่งปัญญาประดิษฐ์

ในช่วงวันหยุดคริสต์มาสปี ค.ศ. 1989 **กุยโด ฟาน รอสซัม** นักวิทยาการคอมพิวเตอร์ชาวดัตช์ ได้เริ่มต้นพัฒนาภาษาโปรแกรมใหม่ขึ้นมาเพื่อเป็นงานอดิเรก โดยเขาตั้งชื่อภาษานี้ว่า "Python" ตามชื่อรายการตลกของอังกฤษเรื่อง Monty Python's Flying Circus

กุยโดมีวิสัยทัศน์ที่ปฏิวัติวงการเขียนโปรแกรม โดยเขาเชื่อว่า

"โค้ดคอมพิวเตอร์ถูกอ่านบ่อยกว่าถูกเขียนขึ้นมาหลายเท่า ดังนั้น ความเรียบง่าย สะอาด และอ่านเข้าใจได้ง่าย (Readability) จึงต้องมาก่อนทุกสิ่ง"

วิสัยทัศน์นี้ถูกบันทึกไว้เป็นปรัชญาประจำภาษาที่เรียกว่า "The Zen of Python" (PEP 20) ซึ่งประกอบด้วยหลักคิดสำคัญ เช่น

- Beautiful is better than ugly. (สวยงามดีกว่าน่าเกลียด)
- Explicit is better than implicit. (ชัดเจนแจ่มแจ้งดีกว่าคลุมเครือ)
- Simple is better than complex. (เรียบง่ายดีกว่าซับซ้อน)
- Readability counts. (ความสามารถในการอ่านเข้าใจมีความสำคัญสูงสุด)

ทำไม Python จึงครองใจนักวิทยาศาสตร์และ AI?

Scientific Ecosystem

ในปัจจุบัน ภาษา Python ได้กลายเป็นภาษากลางอันดับ 1 ของโลกในงานวิจัยฟิสิกส์ ดาราศาสตร์ การแพทย์ และปัญญาประดิษฐ์ (AI / Machine Learning) ด้วยเหตุผลสำคัญ 3 ประการ

- ไวยากรณ์คล้ายภาษาอังกฤษ: นักวิทยาศาสตร์สามารถมุ่งเน้นที่สูตรและสมการฟิสิกส์ได้โดยไม่ต้องเสียเวลากับไวยากรณ์ที่ซับซ้อน
- ระบบนิเวศทางวิทยาศาสตร์ที่แข็งแกร่งที่สุดในโลก: มีไลบรารีระดับโลก เช่น `NumPy` (คณิตศาสตร์เมทริกซ์ความเร็วสูง), `SciPy` (การคำนวณทางวิทยาศาสตร์), `Matplotlib` (การพล็อตกราฟ 2D/3D), `PyTorch` และ `TensorFlow` (โครงข่ายประสาทเทียม AI)
- ภาพถ่ายหลุมดำภาพแรกของมนุษยชาติ (Event Horizon Telescope): ประมวลผลข้อมูลขนาด 5 เพตะไบต์ด้วยอัลกอริทึมภาษา Python ทั้งหมด!

3.1 ตัวแปรและแบบจำลองหน่วยความจำในภาษา Python

ในภาษาโปรแกรมแบบดั้งเดิม (เช่น C หรือ Java) ตัวแปรเปรียบเสมือน "กล่องใส่ของ (Box)" ที่ต้องระบุชนิดข้อมูลล่วงหน้า แต่ในภาษา Python ตัวแปรทำหน้าที่เป็น "ป้ายชื่อที่ผูกด้วยเชือก (Name Tag / Reference Pointer)" ซึ่งไปยังวัตถุ (Object) ที่ถูกสร้างขึ้นในหน่วยความจำ RAM

```
graph LR
    subgraph RAM ["หน่วยความจำ Heap Memory"]
        OBJ1["Object: 25\nType: int\nAddress: 0x104A"]
        OBJ2["Object: 'Phy'\nType: str\nAddress: 0x208B"]
        OBJ3["Object: 3.14159\nType: float\nAddress: 0x30CF"]
    end

    V1["ตัวแปร: speed"] -->|Pointer| OBJ1
    V2["ตัวแปร: subject"] -->|Pointer| OBJ2
    V3["ตัวแปร: pi"] -->|Pointer| OBJ3
```

คุณสมบัติสำคัญของตัวแปรในภาษา Python

- การกำหนดชนิดข้อมูลแบบไดนามิก ** ผู้เขียนไม่จำเป็นต้องประกาศชนิดตัวแปรล่วงหน้า ตัวแปรจะปรับชนิดข้อมูลตามค่าที่ได้รับโดยอัตโนมัติ

```
x = 10          # x เป็น int (จำนวนเต็ม)
x = "RBRU"     # x เปลี่ยนเป็น str (ข้อความ) ทันทีอย่างราบรื่น
```

- การตรวจสอบพิกัดหน่วยความจำด้วยฟังก์ชัน `id()` :

```
mass = 50
print(id(mass)) # แสดงแอดเดรสตำแหน่งในหน่วยความจำ เช่น 1407352840
```

กฎการตั้งชื่อตัวแปรตามมาตรฐานสากล

- กฎเหล็กทางไวยากรณ์**
 - ต้องขึ้นต้นด้วยตัวอักษรภาษาอังกฤษ (a-z, A-Z) หรือเครื่องหมายขีดล่าง (_) เท่านั้น
 - ห้ามขึ้นต้นด้วยตัวเลขเด็ดขาด (เช่น `1st_place` ถือว่าผิดไวยากรณ์ `SyntaxError`)
 - ประกอบด้วยตัวอักษร ตัวเลข และเครื่องหมาย _ เท่านั้น (ห้ามมีเว้นวรรค หรือสัญลักษณ์พิเศษ เช่น `อ`, `$`, `%`)
 - ภาษา Python แยกความแตกต่างระหว่างตัวพิมพ์เล็กและพิมพ์ใหญ่ (Case-Sensitive): เช่น `Velocity`, `velocity`, และ `VELOCITY` เป็นตัวแปรคนละตัวกัน
 - ห้ามใช้ **คำสงวน** เช่น `if`, `for`, `while`, `def`, `class`, `import`, `return`
- แบบแผนสไตร์มาตรฐาน**
 - ตัวแปรทั่วไปและฟังก์ชัน: ใช้รูปแบบ `snake_case` เช่น `kinetic_energy`, `sensor_temperature`, `car_speed`
 - ค่าคงที่ (Constants): ใช้ตัวพิมพ์ใหญ่ทั้งหมด เช่น `GRAVITY_ACCEL = 9.80665`, `SPEED_OF_LIGHT = 299792458`

3.2 ชนิดข้อมูลพื้นฐาน 4 ประเภท

ภาษา Python มีชนิดข้อมูลพื้นฐานที่เป็นรากฐานของทุกระบบ 4 ประเภท

ชนิดข้อมูล (Data Type)	คำสำคัญ (Keyword)	นิยามและความหมาย	ตัวอย่างค่าข้อมูล
จำนวนเต็ม	<code>int</code>	ตัวเลขจำนวนเต็มบวก จำนวนเต็มลบ หรือศูนย์ ไม่จำกัดขนาดความยาว	<code>-273</code> , <code>0</code> , <code>42</code> , <code>1000000</code>
ทศนิยม	<code>float</code>	ตัวเลขที่มีจุดทศนิยม หรือตัวเลขในรูปสัญกรณ์วิทยาศาสตร์	<code>3.14159</code> , <code>-0.005</code> , <code>1.5e-3</code> (1.5×10^{-3})
สายอักขระ	<code>str</code>	ข้อความหรือชุดตัวอักษรที่อยู่ในเครื่องหมายคำพูดเดี่ยว ('...') หรือคู่ (...)	<code>"Physics"</code> , <code>'RBRU 2026'</code> , <code>3.14</code>
ค่าความจริง	<code>bool</code>	ค่าสถานะทางตรรกศาสตร์ มีเพียง 2 ค่าเท่านั้น	<code>True</code> (จริง), <code>False</code> (เท็จ)

การแปลงชนิดข้อมูล

ในการประมวลผลข้อมูลทางวิทยาศาสตร์ ข้อมูลที่รับเข้ามาผ่านคีย์บอร์ดมักมีสถานะเป็น `str` เราจึงจำเป็นต้องแปลงให้เป็นตัวเลขเพื่อนำไปคำนวณทางคณิตศาสตร์

```
raw_input = "25.5"          # เป็น str
temperature = float(raw_input) # แปลงเป็น float สำเร็จ (25.5)
rounded_temp = int(temperature) # แปลงเป็น int (25 - บัดเศษทิ้ง)
status_str = str(temperature) # แปลงกลับเป็น str ("25.5")
```

⚠️ ข้อควรระวังเรื่อง ValueError: หากสั่ง `int(3.14)` หรือ `int("hello")` คอมพิวเตอร์จะเกิดข้อผิดพลาด `ValueError` ทันที เนื่องจากข้อความมีจุดทศนิยมหรือไม่ใช้ตัวเลขฐาน 10

3.3 ตัวดำเนินการคณิตศาสตร์และลำดับความสำคัญ PEMDAS

ตารางตัวดำเนินการคณิตศาสตร์ในภาษา Python

ตัวดำเนินการ	สัญลักษณ์	ชื่อภาษาไทย	ตัวอย่างการใช้งาน	ผลลัพธ์
Addition	+	การบวก	<code>10 + 5</code>	15
Subtraction	-	การลบ	<code>10 - 5</code>	5
Multiplication	*	การคูณ	<code>10 * 5</code>	50
Division	/	การหารจริง (ผลลัพธ์เป็น float เสมอ)	<code>10 / 4</code>	2.5
Floor Division	//	การหารปัดเศษทั้ง (ผลลัพธ์เป็นจำนวนเต็ม)	<code>10 // 4</code>	2
Modulo	%	การหารเอาเศษเหลือ	<code>10 % 4</code>	2
Exponentiation	**	การยกกำลัง (x^y)	<code>2 ** 3</code>	8 (2^3)

ลำดับการประมวลผลทางคณิตศาสตร์สากล

เมื่อมีตัวดำเนินการหลายตัวในสมการเดียวกัน คอมพิวเตอร์จะประมวลผลตามลำดับมาตรฐานสากลจากลำดับความสำคัญสูงสุดไปต่ำสุด

```
graph TD
    P["1. P - Parentheses (วงเล็บ)\nประมวลผลในวงเล็บในสุดก่อนเสมอ ( ... )"] --> E["2. E - Exponents (การยกกำลัง)\nคำนวณ ** จากซ้ายไปขวา"]
    E --> MD["3. M & D - Multiplication & Division (*, /, //, %)\nลำดับเท่ากัน คำนวณจากซ้ายไปขวา"]
    MD --> AS["4. A & S - Addition & Subtraction (+, -)\nลำดับเท่ากัน คำนวณจากซ้ายไปขวา"]
```

ตัวอย่างการคำนวณนิพจน์ซับซ้อน

จงหาค่าของนิพจน์: `result = 10 + 2 * (3 ** 2) - 8 // 3`

- วงเล็บและยกกำลัง `(3 ** 2) → 9`
- สมการกลายเป็น `10 + 2 * 9 - 8 // 3`
- การคูณและการหารปัดเศษ (จากซ้ายไปขวา):
 - `2 * 9 → 18`
 - `8 // 3 → 2`
- สมการกลายเป็น `10 + 18 - 2`
- การบวกและการลบ `28 - 2 = 26` (ผลลัพธ์คือ 26)

3.4 การรับเข้าและส่งออกข้อมูลทางวิทยาศาสตร์

1. การรับข้อมูลด้วยฟังก์ชัน `input()`

ฟังก์ชัน `input()` จะหยุดรอให้ผู้ใช้ป้อนข้อมูลและกด Enter โดยค่าที่ส่งกลับมาเป็น สายอักขระ (`str`) เสมอ:

```
# ต้องครอบด้วย float() หรือ int() เพื่อแปลงชนิดข้อมูลก่อนนำไปคำนวณ
mass_kg = float(input("กรุณานำมวลของวัตถุ (kg): "))
velocity_ms = float(input("กรุณานำความเร็วของวัตถุ (m/s): "))
```

2. การจัดรูปแบบการแสดงผลขึ้นสูงด้วย f-strings

นำเสนอใน Python 3.6+ ถือเป็นมาตรฐานที่ดีที่สุด สะอาดที่สุด และเร็วที่สุดในการแสดงผล

```
# ไวยากรณ์: f"ข้อความ {ตัวแปร:รูปแบบการจัดวาง}"
pi_val = 3.1415926535
energy_joules = 1250450.789

# 1. จัดทศนิยม 2 ตำแหน่ง
print(f"ค่าพาย: {pi_val:.2f}") # แสดงผล: 3.14

# 2. ใส่เครื่องหมายจุลภาคคั่นหลักพันและทศนิยม 2 ตำแหน่ง
print(f"พลังงานจลน์: {energy_joules:,.2f} J") # แสดงผล: 1,250,450.79 J

# 3. จัดตัวเลขในรูปแบบการนิวิทยาศาสตร์ (Scientific Notation)
print(f"พลังงานในรูปวิทยาศาสตร์: {energy_joules:.3e} J") # แสดงผล: 1.250e+06 J
```

3.5 ตัวอย่างข้อคำนวณและการวิเคราะห์โจทย์ฟิสิกส์เชิงลึก

ตัวอย่างที่ 3.1 การคำนวณพลังงานจลน์ของรถยนต์ไฟฟ้า

โจทย์ รถยนต์ไฟฟ้าคันหนึ่งมีมวล $m = 1,850 \text{ kg}$ กำลังเคลื่อนที่ด้วยความเร็ว $v = 90 \text{ km/h}$ จงเขียนโปรแกรมแปลงความเร็วเป็นหน่วยเมตรต่อวินาที (m/s) และคำนวณพลังงานจลน์ ($E_k = \frac{1}{2}mv^2$) ในหน่วยจูล (J) และกิโลจูล (kJ)

ขั้นตอนการวิเคราะห์

- อัตราส่วนแปลงความเร็ว: $v_{\text{m/s}} = v_{\text{km/h}} \times \frac{1000}{3600} = v_{\text{km/h}} \div 3.6$
- สูตรพลังงานจลน์: $E_k = 0.5 \times m \times v_{\text{m/s}}^2$
- แปลงเป็นกิโลจูล: $E_{k,\text{kJ}} = E_k \div 1000$

```
# =====
# kinetic_energy_scientific_calc.py
# โปรแกรมคำนวณพลังงานจลน์ของยานยนต์และการจัดรูปแบบรายงานทางวิทยาศาสตร์
# ผู้เขียน: ผู้ช่วยศาสตราจารย์ ดร. ชีวะ หัตถนา (มหาวิทยาลัยราชภัฏรำไพพรรณี)
# มาตรฐาน: Python 3.11+ • PEP 8 Compliant • Pure Standard Library
# =====

from typing import Tuple

def calculate_ev_kinetic_energy(mass_kg: float, velocity_kmh: float) → Tuple[float, float, float]:
    """
    คำนวณพลังงานจลน์ของยานยนต์ไฟฟ้า

    Parameters:
        mass_kg (float): มวลของรถยนต์ในหน่วยกิโลกรัม (kg)
        velocity_kmh (float): ความเร็วของรถยนต์ในหน่วยกิโลเมตรต่อชั่วโมง (km/h)

    Returns:
        Tuple[float, float, float]: (ความเร็ว m/s, พลังงานจลน์ จูล, พลังงานจลน์ กิโลจูล)
    """
    # 1. แปลงความเร็วจาก km/h เป็น m/s
    velocity_ms = velocity_kmh / 3.6

    # 2. คำนวณพลังงานจลน์ตามสมการฟิสิกส์ E_k = (1/2) * m * v^2
    energy_joules = 0.5 * mass_kg * (velocity_ms ** 2)

    # 3. แปลงเป็นกิโลจูล
    energy_kj = energy_joules / 1000.0

    return velocity_ms, energy_joules, energy_kj

def generate_scientific_energy_report(vehicle_name: str, mass: float, speed_kmh: float):
    """สร้างและจัดพิมพ์รายงานผลการทดลองทางวิทยาศาสตร์รูปแบบ f-string"""
    v_ms, e_j, e_kj = calculate_ev_kinetic_energy(mass, speed_kmh)

    print("\n" + "=" * 72)
    print(f"📄 รายงานการวิเคราะห์พลังงานจลน์เชิงวิทยาศาสตร์: {vehicle_name.upper()}")
    print("=" * 72)
    print(f"• มวลของยานพาหนะ (Vehicle Mass)       : {mass:,.2f} kg")
    print(f"• ความเร็วหน้าปัด (Dashboard Speed)       : {speed_kmh:,.2f} km/h")
    print(f"• ความเร็วตามมาตรฐาน SI (Velocity SI)    : {v_ms:,.4f} m/s")
    print("-" * 72)
    print(f"• พลังงานจลน์สะสม (Kinetic Energy J)     : {e_j:,.2f} Joules")
    print(f"• พลังงานจลน์สะสม (Kinetic Energy kJ)    : {e_kj:,.2f} kJ")
    print(f"📊 รูปแบบสัญกรณ์วิทยาศาสตร์ (Scientific) : {e_j:.4e} J")
    print("=" * 72 + "\n")

if __name__ == "__main__":
    # 1. รันการจำลองกรณีศึกษาของรถยนต์ไฟฟ้า Tesla Model 3
    test_vehicle = "Tesla Model 3 EV"
    test_mass = 1850.0 # kg
    test_speed = 90.0 # km/h (เท่ากับ 25.0 m/s)

    generate_scientific_energy_report(test_vehicle, test_mass, test_speed)

    # 2. ตรวจสอบความถูกต้องด้วย Unit Assertions
    v_calc, ej_calc, ekj_calc = calculate_ev_kinetic_energy(test_mass, test_speed)
    assert abs(v_calc - 25.0) < 1e-5, "Velocity conversion failed"
    assert abs(ej_calc - 578125.0) < 1e-5, "Kinetic energy calculation failed"
    assert abs(ekj_calc - 578.125) < 1e-5, "Kilojoule conversion failed"
    print("✅ ระบบผ่านการตรวจสอบความถูกต้องของ Assertion Tests 100% OK!\n")
```

3.7 คู่มือใบงานห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe Hands (ตอนที่ 3)

```
graph TD
    LAB["ชุด 5 ห้องปฏิบัติการ AR MediaPipe Hands ประจำวันที่ 3"]
    LAB --> L30["3.0 In-Browser Python Sandbox\n• chapter03_inbrowser_python_sandbox.html\n• รันโค้ด Python 3.11 บนเบราว์เซอร์ผ่าน Pyodide"]
    LAB --> L31["3.1 Variable RAM Memory Visualizer\n• chapter03_variable_memory_model.html\n• ใช้มือ AR หยิบตัวแปรไปผูกกับแอดเดรสในหน่วยความจำ"]
    LAB --> L32["3.2 Datatype Typecasting Lab\n• chapter03_datatype_casting_lab.html\n• จำลองการเปลี่ยนสภาพของข้อมูล int, float, str, bool"]
    LAB --> L33["3.3 PEMDAS Expression Tree Evaluator\n• chapter03_pemdas_evaluator.html\n• ยุบกิ่งก้านต้นไม้พจน์คณิตศาสตร์ตามลำดับความสำคัญ"]
    LAB --> L34["3.4 Kinetic Energy EV Vehicle Calculator\n• chapter03_kinetic_energy_calc.html\n• ปรบความเร็วมือจำลองวัตถุพลังงานจลน์และแสดง"]
```

3.8 สรุปสาระสำคัญประจำวัน

1. **ปรัชญาภาษา Python (PEP 20)** ให้ความสำคัญกับความเรียบง่าย สะอาด และอ่านเข้าใจง่าย ทำให้กลายเป็นภาษาสากลอันดับ 1 ของโลกในงานวิจัยวิทยาศาสตร์และปัญญาประดิษฐ์
2. **ตัวแปรใน Python** ทำหน้าที่เป็น Name Tag ชี้ไปยัง Object ในหน่วยความจำ โดยมีการจัดการชนิดข้อมูลแบบ Dynamic Typing
3. **4 ชนิดข้อมูลพื้นฐาน** ได้แก่ `int` (จำนวนเต็ม), `float` (ทศนิยม), `str` (ข้อความ), และ `bool` (ค่าความจริง)
4. **ลำดับ PEMDAS** ควบคุมการประมวลผลตัวดำเนินการคณิตศาสตร์ โดยเรียงลำดับความสำคัญสูงสุด และยกกำลังคำนวณจากขวาไปซ้าย

3.9 แบบฝึกหัดทบทวนและโจทย์ประเมินผลสัมฤทธิ์

◆ ตอนที่ 1 ความรู้ความเข้าใจพื้นฐาน

- จงระบุชนิดข้อมูล (Data Type) ในภาษา Python ของค่าข้อมูลต่อไปนี้
 - ก. `value1 = 2026`
 - ข. `value2 = 9.80665`
 - ค. `value3 = 3.0`
 - ง. `value4 = (10 > 5)`
- ตัวแปรต่อไปนี้ ชื่อใดถูกต้องตามกฎไวยากรณ์ และชื่อใดผิดกฎไวยากรณ์ (ระบุเหตุผล):
 - `2nd_experiment` , `total_score` , `class` , `sensor-temp` , `_pi_value`
- จงคำนวณหาผลลัพธ์ของนิพจน์ต่อไปนี้ตามลำดับ PEMDAS:
 - ก. `result_a = 20 - 4 * 3 + 2 ** 3`
 - ข. `result_b = 15 // 4 + 15 % 4 * 2`

◆ ตอนที่ 2 การวิเคราะห์และประยุกต์ใช้

- ระบบแปลงอุณหภูมิ 3 สเกล: จงเขียนโปรแกรมรับค่าอุณหภูมิในหน่วยองศาเซลเซียส (C) จากผู้ใช้ แล้วคำนวณแปลงเป็น
 - องศาฟาเรนไฮต์ ($F = \frac{9}{5}C + 32$)
 - เคลวิน ($K = C + 273.15$)
 - แสดงผลลัพธ์ด้วย f-string จัดทศนิยม 2 ตำแหน่ง
- พิจารณาโค้ดต่อไปนี้

```
a = "100"
b = "200"
c = a + b
print(c)
```

- ถ้ามว่าผลลัพธ์ที่พิมพ์ออกมาคืออะไร เพราะเหตุใด? และหากต้องการให้ผลลัพธ์เป็น `300` จะต้องแก้ไขโค้ดอย่างไร?

◆ ตอนที่ 3 การสังเคราะห์และคิดขั้นสูง

- จงเขียนโปรแกรมคำนวณ "ดัชนีมวลกาย (BMI)" โดยรับค่าน้ำหนักตัว (kg) และส่วนสูง (cm) จากผู้ใช้ แปลงส่วนสูงเป็นหน่วยเมตร (m) คำนวณ $BMI = \frac{Weight}{(Height)^2}$ และจัดพิมพ์รายงานผลด้วย f-string อย่างสวยงาม

เอกสารอ้างอิงประจำบท

- Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.
- Peters, T. (2004). *PEP 20 – The Zen of Python*. Python Software Foundation.
- Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8 – Style Guide for Python Code*. Python Software Foundation.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.
- Thassana, C. (2026). *Computational Thinking and Applied Artificial Intelligence for Science Education*. Rambhai Barni Rajabhat University Press.

บทที่ 4 โครงสร้างควบคุม เงื่อนไข และการทำซ้ำในงานวิทยาศาสตร์และระบบอัตโนมัติ

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา
สังกัด สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี
เอกสารประกอบรายวิชา 4122104 วิทยาการคำนวณและการแก้ปัญหาเชิงคำนวณ / การสอนวิทยาการคำนวณ

แผนบริหารการสอนประจำบทที่ 4

1. หัวข้อเนื้อหาประจำบท

- เรื่องเล่าเปิดบทเรียนและระบบตัดสินใจอัตโนมัติ เบื้องหลังสมองกลยานยนต์ไร้คนขับ ระบบเบรกฉุกเฉินอัตโนมัติ (AEB) และสถาปัตยกรรมควบคุมการบิน Fly-by-Wire
- ตัวดำเนินการเปรียบเทียบและตรรกศาสตร์บูลีน `=`, `≠`, `<`, `>`, `≤`, `≥` และการตัดวงจรทางตรรกะ (Short-Circuit Evaluation: `and`, `or`, `not`)
- **โครงสร้างทางเลือกและการตัดสินใจ** คำสั่ง `if`, `if-else`, `if-elif-else` และการใช้เงื่อนไขตรวจสอบตัววน (Guard Clauses)
- **กรณีศึกษาเชิงวิศวกรรมระบบเกษตรอัจฉริยะ** การออกแบบตรรกะควบคุมปั๊มพ่นหมอก ม่านบังแดด และระบบระบายอากาศ
- **โครงสร้างการวนซ้ำแบบกำหนดรอบแน่นอน** คำสั่ง `for` ลูป, การใช้งานฟังก์ชัน `range(start, stop, step)` และการหาผลรวมอนุกรม $\sum_{i=1}^n i$
- **โครงสร้างการวนซ้ำตามเงื่อนไข** คำสั่ง `while` ลูป, การอ่านค่าเซนเซอร์แบบวนรอบ (Sensor Polling) และกลไกป้องกันลูปไม่รู้จบ (Infinite Loop Safeguard)
- การควบคุมการไหลของลูปขั้นสูง คำสั่ง `break`, `continue`, และโครงสร้าง `else` ต่อท้ายลูป
- การประยุกต์ใช้โปรแกรมคอมพิวเตอร์ การเขียนโปรแกรมภาษา Python 3.11 จำลองระบบควบคุมโรงเรือนเกษตรและตรวจนับจำนวนเฉพาะ
- คู่มือห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe: การจำลองการแตกแขนงตรรกะเลเซอร์ และการควบคุมลู่วิ่งแบบไร้สัมผัส

2. วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- **อธิบาย** หลักการทำงานของโครงสร้างควบคุมทางเลือกและการวนซ้ำในระบบคอมพิวเตอร์และปัญญาประดิษฐ์ได้อย่างถูกต้อง
- **ออกแบบและเขียน** คำสั่งเงื่อนไขหลายชั้น `if-elif-else` เพื่อจำแนกและประมวลผลข้อมูลตามเกณฑ์ทางวิทยาศาสตร์ได้อย่างรัดกุม
- **ประยุกต์ใช้** คำสั่ง `for` และ `while` ร่วมกับฟังก์ชัน `range()` ในการคำนวณผลรวมอนุกรมคณิตศาสตร์และการวนซ้ำตามค่าเซนเซอร์ได้อย่างคล่องแคล่ว
- **วิเคราะห์และควบคุม** ทิศทางการไหลของลูปด้วยคำสั่ง `break` และ `continue` พร้อมทั้งป้องกันข้อผิดพลาด Infinite Loop ได้อย่างสมบูรณ์
- **สร้างสรรค์** โปรแกรมภาษา Python 3.11 จำลองระบบควบคุมสภาพแวดล้อมอัตโนมัติที่มีความซับซ้อนสูงได้
- **ปฏิบัติการ** การทดลองเสมือนจริง 3D AR MediaPipe Hands เพื่อควบคุมตัวแปรและทดสอบสมรรถนะการตัดสินใจแบบไร้สัมผัสได้

🚗 4.0 เรื่องเล่าเปิดบทเรียน วนาทักษะกับสมองกลตัดสินใจอัตโนมัติ

ลองจินตนาการถึงสถานการณ์ที่รถยนต์ขับเคลื่อนอัตโนมัติ (Autonomous Vehicle) กำลังแล่นด้วยความเร็ว 80 km/h บนถนนยามค่ำคืน ทันใดนั้นมีสิ่งกีดขวางตัดหน้าในระยะกระชั้นชิดเพียง 20 เมตร

ในเสี้ยววินาทีนั้น สมองของมนุษย์อาจต้องใช้เวลาตอบสนอง (Reaction Time) ราว 1.5 วินาที ซึ่งสายเกินไปสำหรับการหยุดรถ แต่ระบบคอมพิวเตอร์อัจฉริยะ Autonomous Emergency Braking (AEB) สามารถอ่านข้อมูลจากเซนเซอร์ LiDAR และเรดาร์ ประมวลผลตรรกะเงื่อนไขความปลอดภัย และสั่งการเบรกฉุกเฉินได้ภายในเวลาเพียง 0.05 วินาที (50 มิลลิวินาที)

```
graph TD
    S["เรดาร์ / LiDAR ตรวจจับวัตถุด้านหน้า"] --> D["คำนวณระยะห่าง (Distance) และความเร็วสัมพัทธ์"]
    D --> C["Distance < Threshold ?"]
    C -->|True (วิกฤต)| B["🚨 สั่งการเบรกฉุกเฉินเต็มกำลัง (AEB Triggered)\n+ เตือนผู้ขับขี่ด้วยเสียงความถี่สูง"]
    C -->|False (ปลอดภัย)| N["รักษาระดับความเร็วและการเดินทางปกติ"]
```

เบื้องหลังการตัดสินใจที่ช่วยชีวิตมนุษย์นับล้านคนนี้ ไม่ใช่จินตนาการเวทมนตร์ใดๆ แต่คือ "โครงสร้างควบคุมการตัดสินใจ (Conditional Control Structure)" ที่ถูกออกแบบและเขียนขึ้นด้วยความประณีต รัดกุม และไร้จุดบกพร่อง

⚖️ 4.1 ตัวดำเนินการเปรียบเทียบและการประเมินตรรกะ

1. ตัวดำเนินการเปรียบเทียบ

ใช้สำหรับเปรียบเทียบค่าข้อมูล 2 ฝั่ง โดยให้ผลลัพธ์เป็นค่าความจริงบูลีน (`True` หรือ `False`):

ตัวดำเนินการ	ความหมาย	ตัวอย่างนิพจน์	ผลลัพธ์ทางตรรกะ
<code>==</code>	เท่ากับ (Equal to) — ระวังอย่าสับสนกับ <code>=</code> ที่ใช้กำหนดค่า	<code>10 == 10</code>	<code>True</code>
<code>!=</code>	ไม่เท่ากับ (Not equal to)	<code>10 != 5</code>	<code>True</code>
<code>></code>	มากกว่า (Greater than)	<code>15 > 20</code>	<code>False</code>
<code><</code>	น้อยกว่า (Less than)	<code>5 < 8</code>	<code>True</code>
<code>>=</code>	มากกว่าหรือเท่ากับ (Greater than or equal to)	<code>10 >= 10</code>	<code>True</code>
<code><=</code>	น้อยกว่าหรือเท่ากับ (Less than or equal to)	<code>7 <= 6</code>	<code>False</code>

2. ตัวดำเนินการตรรกะและการตัดวงจรด่วน

- `and` (และ): จะเป็น `True` ก็ต่อเมื่อ ทั้งสองฝั่งเป็นจริง
- `or` (หรือ): จะเป็น `True` เมื่อ มีฝั่งใดฝั่งหนึ่งเป็นจริง
- `not` (นิเสธ): กลับค่าความจริงจาก `True` เป็น `False` และกลับ `False` เป็น `True`

 **เทคนิค Short-Circuit:** ในคำสั่ง `if is_valid and (x / y > 2)`: หาก `is_valid` มีค่าเป็น `False` ตัวแปลภาษา Python จะ หยุดประเมินฝั่งขวาทันที ทำให้ไม่เกิดข้อผิดพลาด `ZeroDivisionError` แม้ว่า `y` จะมีค่าเป็น 0 ก็ตาม!

4.2 โครงสร้างทางเลือกและการตัดสินใจ

1. โครงสร้างทางเลือกเดี่ยว (`if`) และทางเลือกคู่ (`if-else`)

```
temperature = 38.5

# โครงสร้าง If-Else พื้นฐาน
if temperature > 37.5:
    print("🔥 แฉงเตือน: อุณหภูมิร่างกายสูงเกินเกณฑ์ปกติ (มีไข้)")
else:
    print("✅ อุณหภูมิร่างกายอยู่ในเกณฑ์ปกติ")
```

2. โครงสร้างทางเลือกหลายชั้น (`if-elif-else`)

ใช้สำหรับจำแนกข้อมูลที่มีหลายช่วงค่า เช่น การตัดเกรด หรือการจำแนกความดันโลหิต

```
score = 84.5

if score >= 80:
    grade = "A"
elif score >= 70:
    grade = "B"
elif score >= 60:
    grade = "C"
elif score >= 50:
    grade = "D"
else:
    grade = "F"

print(f"ผลการเรียนที่ได้รับ: เกรด {grade}")
```

4.3 โครงสร้างการวนซ้ำแบบกำหนดรอบแน่นอน (`for Loop & range()`)

คำสั่ง `for` คู่กับในภาษา Python ใช้สำหรับการวนซ้ำผ่านสมาชิกลำดับข้อมูล โดยมักใช้ร่วมกับฟังก์ชัน `range()` :

📌 ไวยากรณ์ของฟังก์ชัน `range`

- `range(stop)` : เริ่มต้นจาก 0 ถึง `stop - 1` (เพิ่มทีละ 1)
- `range(start, stop)` : เริ่มจาก `start` ถึง `stop - 1`
- `range(start, stop, step)` : เริ่มจาก `start` ถึง `stop - 1` โดยก้าวเพิ่มทีละ `step`

```
# 1. การวนรอบ 5 ครั้ง (0, 1, 2, 3, 4)
for i in range(5):
    print(f"รอบที่ {i}")

# 2. การหาผลรวมอนุกรมคณิตศาสตร์ 1 ถึง 100: sum = 1 + 2 + ... + 100
total_sum = 0
for n in range(1, 101):
    total_sum += n

print(f"ผลรวมอนุกรม 1 ถึง 100 มีค่าเท่ากับ: {total_sum}") # ได้ 5050 ตามสูตร Gauss  $n(n+1)/2$ 
```



4.4 โครงสร้างการวนซ้ำตามเงื่อนไข (while Loop & Control Commands)

คำสั่ง **while** จะทำงานวนรอบซ้ำไปเรื่อยๆ ตราบเท่าที่เงื่อนไขยังคงเป็นจริง (**True**) เหมาะสำหรับการอ่านค่าเซนเซอร์ทางวิทยาศาสตร์ หรือการรอรับคำสั่งจากผู้

ใช้

```
graph TD
    W{"เงื่อนไขเป็นจริง (Condition is True) ?"}
    W -->|True| B["ประมวลผลคำสั่งในลูป\n(Execute Loop Body)"]
    B --> W
    W -->|False| E["หลุดออกจากลูป (Exit Loop)"]
```



คำสั่งควบคุมลูปขั้นสูง (**break** และ **continue**)

- break** : สั่งให้ หยุดการทำงานและหลุดออกจากลูปทันที ไม่ว่าจะเหลือรอบอีกเท่าใดก็ตาม
- continue** : สั่งให้ ข้ามคำสั่งที่เหลือในรอบปัจจุบัน แล้ววนกลับขึ้นไปเริ่มต้นรอบถัดไปที่

```
# ตัวอย่าง: ค้นหาตัวเลขตัวแรกที่มีค่ามากกว่า 10
numbers = [3, 7, 9, 12, 15, 18]

for num in numbers:
    if num % 2 != 0:
        continue # ข้ามเลขคู่ไป ไม่ต้องประมวลผลต่อ
    if num > 10:
        print(f"พบเลขคู่ตัวแรกเกิน 10 คือ: {num}")
        break # พบเป้าหมายแล้ว จบลูปทันที
```



4.5 กรณีศึกษาเชิงวิศวกรรม ระบบควบคุมโรงเรือนเกษตรอัจฉริยะ 4.0

โรงเรือนปลูกพืชเศรษฐกิจต้องการระบบควบคุมสภาพแวดล้อมอัตโนมัติ เพื่อรักษาอุณหภูมิและความชื้นให้อยู่ในสภาวะที่เหมาะสมที่สุดต่อการเจริญเติบโต

- เงื่อนไขควบคุมปั๊มพ่นหมอก (MistPump): ทำงานเมื่ออุณหภูมิ $> 35^{\circ}\text{C}$ และความชื้น < 60
- เงื่อนไขमानबंदไฟ (ShadeMotor): กางออกเมื่อค่าความเข้มแสงแดด $> 50,000 \text{ Lux}$
- เงื่อนไขระบบระบายอากาศ (VentilationFan): ทำงานเมื่อระดับก๊าซคาร์บอนไดออกไซด์เกินเกณฑ์ หรืออุณหภูมิสูง
- **ระบบความปลอดภัยฉุกเฉิน**** หากน้ำในถังสำรองหมด (WaterEmpty = 1) ให้ระงับการทำงานของปั๊มทันทีเพื่อป้องกันมอเตอร์ไหม้

```
# =====
# smart_greenhouse_climate_controller.py
# โปรแกรมจำลองระบบควบคุมสภาพแวดล้อมโรงเรือนเกษตรอัจฉริยะด้วยสเปคและเงื่อนไข
# ผู้เขียน: ผู้ช่วยศาสตราจารย์ ดร. ชีวะ ทิศนา (มหาวิทยาลัยราชภัฏรำไพพรรณี)
# มาตรฐาน: Python 3.11+ • PEP 8 Compliant • Pure Standard Library
# =====

from typing import Dict, List

class SmartGreenhouseController:
    """คลาสควบคุมระบบสภาพแวดล้อมอัตโนมัติประจำโรงเรือนเกษตร"""

    def __init__(self, target_temp_max: float = 35.0, target_humid_min: float = 60.0):
        self.target_temp_max = target_temp_max
        self.target_humid_min = target_humid_min

    def evaluate_actuators(self, temp: float, humid: float, light_lux: float, water_empty: bool) → Dict[str, str]:
        """
        ประเมินสถานะอุปกรณ์ควบคุมสภาพแวดล้อมตามตรรกะเงื่อนไข

        Returns:
            Dict[str, str]: สถานะการทำงานของปั๊มพ่นหมอก ม่านบังแดด และพัดลมระบายอากาศ
        """
        # 1. ตรรกะควบคุมปั๊มพ่นหมอก (Mist Pump) พร้อมระบบความปลอดภัย
        if water_empty:
            mist_status = "💧 OFF (WATER RESERVOIR EMPTY - SAFEGUARD)"
        elif temp > self.target_temp_max and humid < self.target_humid_min:
            mist_status = "☁️ ACTIVE (COOLING & HUMIDIFYING)"
        else:
            mist_status = "🌡️ STANDBY"

        # 2. ตรรกะควบคุมม่านบังแดด (Shade Motor)
        if light_lux > 50000.0:
            shade_status = "🚫 EXTENDED (BLOCKING INTENSE SUNLIGHT)"
        elif light_lux < 10000.0:
            shade_status = "☀️ RETRACTED (MAXIMIZING SUNLIGHT)"
        else:
            shade_status = "🌤️ OPTIMAL POSITION"

        # 3. ตรรกะควบคุมพัดลมระบายอากาศ (Ventilation Fan)
        if temp > 38.0:
            fan_status = "💨 HIGH SPEED (HEAT EXHAUST)"
        elif temp > 32.0:
            fan_status = "🌀 LOW SPEED (CIRCULATION)"
        else:
            fan_status = "🌬️ OFF"

        return {
            "mist_pump": mist_status,
            "shade_curtain": shade_status,
            "ventilation_fan": fan_status
        }

    def simulate_24h_telemetry_loop(sensor_log: List[Dict[str, float]]):
        """จำลองการทำงานรอบแบบ For-Loop ประมวลผลข้อมูลเซ็นเซอร์ตลอด 24 ชั่วโมง"""
        controller = SmartGreenhouseController()

        print("\n" + "=" * 84)
        print("📡 รายงานผลการควบคุมอัตโนมัติโรงเรือนเกษตรอัจฉริยะ (SMART GREENHOUSE TELEMETRY)")
        print("=" * 84)

        for entry in sensor_log:
            t = entry["hour"]
            temp = entry["temp"]
            humid = entry["humid"]
            lux = entry["lux"]
            w_empty = entry["water_empty"]

            actions = controller.evaluate_actuators(temp, humid, lux, w_empty)

            print(f"🕒 เวลา {t:02.0f}:00 น. | อุณหภูมิ: {temp:4.1f}°C | ความชื้น: {humid:4.1f}% | แสง: {lux:6.0f} Lux")
            print(f"    | ปั๊มพ่นหมอก : {actions['mist_pump']}")
            print(f"    | ม่านบังแดด : {actions['shade_curtain']}")
            print(f"    | พัดลมระบายอากาศ: {actions['ventilation_fan']}")
            print("-" * 84)

        print("✅ การจำลองข้อมูลรอบเสร็จสิ้นสมบูรณ์ 100%\n")

if __name__ == "__main__":
    # ชุดข้อมูลจำลองเซ็นเซอร์โรงเรือน 3 ช่วงเวลา
    telemetry_data = [
        {"hour": 8.0, "temp": 28.5, "humid": 75.0, "lux": 25000.0, "water_empty": False},
        {"hour": 13.0, "temp": 36.8, "humid": 48.0, "lux": 68000.0, "water_empty": False},
        {"hour": 15.0, "temp": 39.2, "humid": 42.0, "lux": 55000.0, "water_empty": True} # ถึงน้ำแห้งฉุกเฉิน
    ]

    simulate_24h_telemetry_loop(telemetry_data)

    # ตรวจสอบความถูกต้องด้วย Unit Assertions
    ctrl = SmartGreenhouseController()
    res1 = ctrl.evaluate_actuators(36.8, 48.0, 68000.0, False)
```

```
assert "ACTIVE" in res1["mist_pump"], "Mist pump should be ACTIVE"
assert "EXTENDED" in res1["shade_curtain"], "Shade curtain should be EXTENDED"

res2 = ctrl.evaluate_actuators(39.2, 42.0, 55000.0, True)
assert "SAFEGUARD" in res2["mist_pump"], "Mist pump must be locked due to water empty safeguard"
print("✅ ระบบผ่านการตรวจสอบความถูกต้องของ Assertion Tests 100% OK!\n")
```

🔗 4.7 คู่มือใบงานห้องปฏิบัติการเสมือนจริง 3D AR MediaPipe Hands (Unit 4)

graph TD

LAB["ชุด 5 ห้องปฏิบัติการ AR MediaPipe Hands ประจำหน่วยที่ 4"]

LAB → L40["4.0 Automated Decision System\n• chapter04_automated_decision_intro.html\n• จำลองระบบเบรกฉุกเฉิน AEB ด้วยการควบคุมท่าทางมี"]

LAB → L41["4.1 Logic Condition Gate Visualizer\n• chapter04_branching_sandbox.html\n• ใช้มือเปิด-ปิดเกตรรกระ and, or, not สังเกตแสงสี"]

LAB → L42["4.2 Smart Greenhouse Climate Dashboard\n• chapter04_greenhouse_controller.html\n• ปรับ Slider อุณหภูมิและความชื้น 3D สังเกต"]

LAB → L43["4.3 For-Loop Series Accumulator\n• chapter04_for_loop_range_visualizer.html\n• สังเกตการซ้อนทับของบล็อกผล"]

LAB → L44["4.4 Sensor Polling While-Loop Scanner\n• chapter04_sensor_polling_while.html\n• จำลองอุปกรณ์จับจำนวนเฉพาะและทดสอบค่า"]

💡 4.8 สรุปสาระสำคัญประจำบท

1. โครงสร้างทางเลือก (`if-elif-else`) เป็นรากฐานของการตัดสินใจในระบบคอมพิวเตอร์และปัญญาประดิษฐ์ โดยมี Short-Circuit Evaluation ช่วยเพิ่มความเร็วและความปลอดภัย
2. `for` ลูป เหมาะสำหรับการวนซ้ำที่ทราบจำนวนรอบที่แน่นอน ทำงานร่วมกับฟังก์ชัน `range(start, stop, step)`
3. `while` ลูป เหมาะสำหรับการวนซ้ำตามเงื่อนไข โดยต้องระวังการปรับปรุ่ค่าตัวแปรควบคุมเพื่อป้องกัน Infinite Loop
4. คำสั่ง `break` ใช้หยุดการทำงานของลูปทันที ส่วน `continue` ใช้ข้ามไปยังรอบถัดไป ช่วยให้การเขียนโค้ดมีความยืดหยุ่นสูง

📚 4.9 แบบฝึกหัดกบทวนและโจทย์ประเมินผลสัมฤทธิ์

◆ ตอนที่ 1 ความรู้ความเข้าใจพื้นฐาน

1. จงอธิบายความแตกต่างระหว่างคำสั่ง `for` ลูป และ `while` ลูป พร้อมยกตัวอย่างสถานการณ์ที่เหมาะสมสำหรับแต่ละคำสั่ง
2. คำสั่ง `break` และ `continue` แตกต่างกันอย่างไร จงอธิบายพร้อมเขียนโค้ดตัวอย่างประกอบ
3. ฟังก์ชัน `range(2, 11, 3)` จะสร้างลำดับตัวเลขใดออกมาบ้าง?

◆ ตอนที่ 2 การวิเคราะห์และประยุกต์ใช้

4. **ระบบตรวจจับจำนวนเฉพาะ** จงเขียนโปรแกรมรับค่าจำนวนเต็ม $N > 1$ จากผู้ใช้ แล้วใช้คำสั่ง `for` หรือ `while` ตรวจสอบว่ามีตัวเลขใดตั้งแต่ 2 ถึง $\sqrt{N}$ ที่หาร N ลงตัวหรือไม่ หากพบตัวหารให้ใช้คำสั่ง `break` และพิมพ์ผลลัพธ์ว่า N ไม่ใช่จำนวนเฉพาะ
5. จงวิเคราะห์โค้ดต่อไปนี้ และระบุว่ามีความผิดพลาดประเภทใดเกิดขึ้น

```
count = 1
while count ≤ 5:
    print(f"นับ: {count}")
    # ไม่มีคำสั่ง count += 1
```

◆ ตอนที่ 3 การสังเคราะห์และคิดขั้นสูง

6. จงเขียนโปรแกรมจำลอง "เครื่องรับฝาก-ถอนเงินธนาคารอัตโนมัติ (ATM Machine)" โดยใช้ `while True:` ลูป แสดงเมนู 4 ตัวเลือก: 1. ฝากเงิน, 2. ถอนเงิน, 3. ตรวจสอบยอดเงินคงเหลือ, 4. ออกจากโปรแกรม (`break`) พร้อมระบบตรวจสอบยอดเงินไม่ให้ติดลบ

📖 เอกสารอ้างอิงประจำบท

1. Lutz, M. (2013). *Learning Python: Powerful Object-Oriented Programming* (5th ed.). O'Reilly Media.
2. National Highway Traffic Safety Administration. (2020). *Automatic Emergency Braking Systems in Light Vehicles*. U.S. Department of Transportation.
3. Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.
4. Thassana, C. (2026). *Computational Thinking and Applied Artificial Intelligence for Science Education*. Rambhai Barni Rajabhat University Press.

วิทยาการคำนวณ 1 รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ

บทที่ 4 กิจกรรม Unplugged Coding ในชีวิตประจำวัน

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนาศา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

🌟 ผลลัพธ์การเรียนรู้ประจำบท

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- อธิบาย หลักการและความสำคัญของกิจกรรม Unplugged Coding ในการพัฒนาทักษะการคิดเชิงคำนวณโดยไม่ต้องพึ่งพาคอมพิวเตอร์
- จำลองและแก้ปัญหา ปัญหาการนำทาง การแปลงเลขฐานสอง การบีบอัดข้อมูลภาพ และการตรวจจับข้อผิดพลาด (Error Detection) ผ่านกิจกรรมเชิงรุก
- จัดระเบียบและสื่อสาร ขั้นตอนวิธีให้กับเพื่อนร่วมชั้นและสามารถค้นหาจุดบกพร่อง (Debugging) ในชีวิตจริงได้อย่างเป็นระบบ
- เชื่อมโยง ประสบการณ์จากกิจกรรม Unplugged ไปสู่การเขียนโปรแกรมคอมพิวเตอร์ด้วยภาษา Python

📺 4.0 เรื่องเล่าเปิดบทเรียน วิทยาการคอมพิวเตอร์ที่เกิดขึ้นก่อนเครื่องคอมพิวเตอร์

หลายคนเข้าใจว่า "วิทยาการคอมพิวเตอร์ (Computer Science)" คือเรื่องของเครื่องจักร แป้นพิมพ์ และหน้าจออิเล็กทรอนิกส์ แต่ในความเป็นจริง บิดาและมารดาแห่งวิทยาการคอมพิวเตอร์อย่าง **เอดา เลิฟเลซ** และ **แอลัน ทัวริง** ได้คิดค้นขั้นตอนวิธีและทฤษฎีการคำนวณขึ้นบน กระดาษ ปากกา และสมองมนุษย์ ตั้งแต่ก่อนที่คอมพิวเตอร์ไฟฟ้าเครื่องแรกของโลกจะถูกสร้างขึ้นเกือบหนึ่งร้อยปี!

graph LR

Thought["ความคิดเชิงนามธรรมและตรรกศาสตร์\n(Pure Computational Thinking)"] → Paper["การคิดบนกระดาษและกิจกรรมลงมือทำ\n(Unplugged Activities)"]
Paper → Code["การเขียนโปรแกรมบนคอมพิวเตอร์\n(Plugged Python Coding)"]

Unplugged Coding จึงเป็นสะพานแห่งการเรียนรู้ที่ยอดเยี่ยที่สุด ที่ช่วยให้ผู้เรียนทุกวัยเข้าใจแก่นแท้ของวิทยาการคำนวณผ่านการเล่น เกม บอร์ดเกม การเคลื่อนไหวร่างกาย และการแก้ปัญหาในกายภาพ

🤖 4.1 กิจกรรมที่ 1 การสั่งการหุ่นยนต์มนุษย์และตารางพิกัด

แนวคิดทางวิทยาการคอมพิวเตอร์

- การจัดลำดับคำสั่ง (Sequential Execution), ตัวแปรทิศทาง (State/Orientation), และการแก้ไขจุดบกพร่อง (Debugging)

กฎกติกาและชุดคำสั่งสากล

- ผู้เรียนแบ่งเป็น 2 บทบาท: "โปรแกรมเมอร์ (Programmer)" และ "หุ่นยนต์มนุษย์ (Robot)"
- หุ่นยนต์มนุษย์จะถูกปิดตา และสามารถทำตามคำสั่งที่ได้รับจากการ์ดคำสั่ง 4 ใบเท่านั้น
 - FORWARD(1) : ก้าวไปข้างหน้า 1 ช่อง
 - TURN_LEFT() : หมุนตัวไปทางซ้าย 90° (อยู่กับที่)
 - TURN_RIGHT() : หมุนตัวไปทางขวา 90° (อยู่กับที่)
 - STOP() : หยุดการทำงาน

graph TD

Grid["ตารางพิกัด 5x5 Grid Map"]
StartPos["จุดเริ่มต้น (0,0) ทิศหน้าทิศเหนือ"] → C1["FORWARD(2)"]
C1 → C2["TURN_RIGHT(1)"]
C2 → C3["FORWARD(3)"]
C3 → Goal["ถึงเป้าหมายธงชัย (3,2)!"]

การดัดแปลงในชีวิตจริง

หากหุ่นยนต์เดินชนสิ่งกีดขวาง โปรแกรมเมอร์จะต้อง Trace ย้อนกลับ ไปดูว่าคำสั่งที่บรรทัดใดเกิดความคลาดเคลื่อนทางพิกัด (x, y) เพื่อแก้ไขชุดคำสั่งให้ถูกต้อง

📺 4.2 กิจกรรมที่ 2 ศิลปะพิกเซลและการบีบอัดข้อมูลภาพ

แนวคิดทางวิทยาการคอมพิวเตอร์

- ระบบเลขฐานสอง (Binary System), การแทนค่าข้อมูลภาพ (Bitmap Representation), และการบีบอัดข้อมูลแบบไม่สูญเสีย (Lossless Data Compression)

หลักการแทนค่าภาพขาว-ดำ

กำหนดให้ 0 แทนพิกเซลสีขาว และ 1 แทนพิกเซลสีดำ

ตัวอย่างการแปลงภาพวัถกษร "T" ขนาด 5x5 พิกเซล

แถวที่ 1: 1 1 1 1 1

แถวที่ 2: 0 0 1 0 0

แถวที่ 3: 0 0 1 0 0

แถวที่ 4: 0 0 1 0 0

แถวที่ 5: 0 0 1 0 0

การบีบอัดแบบ Run-Length Encoding (RLE):

แทนที่จะส่งข้อมูลทั้ง 25 บิต เราสามารถบันทึกจำนวนตัวเลขที่ซ้ำกัน:

- แถว 1: 5B (ค่า 5 ตัว)
- แถว 2-5: 2W 1B 2W (ขาว 2, ดำ 1, ขาว 2)

ช่วยลดขนาดข้อมูลลงได้มากกว่า 50%!

4.3 กิจกรรมที่ 3 มายากลไฟตรวจจับและแก้ไขข้อผิดพลาด

แนวคิดทางวิทยาการคอมพิวเตอร์

- การตรวจสอบความถูกต้องของข้อมูลในการส่งผ่านเครือข่ายอินเทอร์เน็ต (Error Detection & Correction)

```
graph TD
    GridCards["จัดวางการ์ดหน้าดำ/หน้าขาว 5x5 แผ่น"] --> AddParity["เพิ่มแถวที่ 6 และคอลัมน์ที่ 6 เป็น 'Parity Bit' (กำหนดให้ทุกแถวและคอลัมน์มีจำนวนการ์ดดำเป็นเลขคู่)"]
    AddParity --> FlipSecret["ให้เพื่อนแอบพลิกการ์ด 1 ใบตอนเราปิดตา"]
    FlipSecret --> Detect["เราหาการ์ดที่ถูกพลิกเจอทันที! (ดูแถวและคอลัมน์ที่มีจำนวนการ์ดดำเป็น 'เลขคู่')"]
```

นี่คือหลักการเดียวกับที่ชิปความจำของคอมพิวเตอร์และดาวเทียมอวกาศใช้ในการแก้ไขข้อผิดพลาดของข้อมูลจากคลื่นรังสีคอสมิก (Single Event Upset - SEU)

4.4 โค้ดคอมพิวเตอร์ Python ตัวแปลงและบีบอัดภาพ RLE

```
# run_length_encoding_compressor.py
# โปรแกรมจำลองการบีบอัดข้อมูลภาพด้วยวิธี Run-Length Encoding (RLE)
# ผู้เขียน: นศ.ดร.วิชา ทัศนวิทย์ (มรภ.ราชภัฏวชิร)

def compress_rle(binary_string: str) -> str:
    """บีบอัดสตริงบิต 0/1 ด้วยอัลกอริทึม Run-Length Encoding"""
    if not binary_string:
        return ""

    compressed = []
    current_char = binary_string[0]
    count = 1

    for char in binary_string[1:]:
        if char == current_char:
            count += 1
        else:
            compressed.append(f"{count}{current_char}")
            current_char = char
            count = 1
    compressed.append(f"{count}{current_char}")

    return " ".join(compressed)

def decompress_rle(compressed_string: str) -> str:
    """คลายการบีบอัดสตริง RLE กลับเป็นสตริงบิตต้นฉบับ"""
    tokens = compressed_string.split()
    decompressed = []
    for token in tokens:
        count = int(token[:-1])
        char = token[-1]
        decompressed.append(char * count)
    return "".join(decompressed)

if __name__ == "__main__":
    # ตัวอย่างข้อมูลภาพไคคอน 8x8 บิต
    original_bitmap = "000111000" + "001111100" + "011111110" + "11111111" + "11111111" + "01111110" + "00111100" + "00011000"

    print("=" * 65)
    print(f"📄 ข้อมูลบิตดั้งเดิม ({len(original_bitmap)} บิต):")
    print(original_bitmap)

    compressed = compress_rle(original_bitmap)
    print(f"\n< ข้อมูลหลังบีบอัด RLE:")
    print(compressed)

    restored = decompress_rle(compressed)
    print(f"\n🔄 ข้อมูลหลังคลายการบีบอัด: {restored}")
    print(f"✅ ความถูกต้อง 100%: {original_bitmap == restored}")
    print("=" * 65)
```


📌 สรุปประเด็นสำคัญ

1. Unplugged Coding มุ่งเน้นการเสริมสร้างกระบวนการคิดมากกว่าการจำไวยากรณ์ภาษาโปรแกรม
2. การแทนค่าด้วยเลขฐานสองและ RLE แสดงให้เห็นว่าคอมพิวเตอร์จัดเก็บภาพ เสียง และวิดีโอได้อย่างมีประสิทธิภาพอย่างไร
3. Parity Bits เป็นบทพิสูจน์ว่าตรรกะคณิตศาสตร์ที่เรียบง่ายสามารถสร้างระบบป้องกันความผิดพลาดที่เสถียรระดับสากลได้

📄 แบบฝึกหัดทบทวน 3 ระดับ

ระดับที่ 1 ความรู้ความเข้าใจพื้นฐาน

1. จงแปลงสตริงฐานสองต่อไปนี้ให้อยู่ในรูปรหัสบีบอัด RLE: 000001111100001111111100
2. ในระบบ Even Parity หากข้อมูลที่ส่งคือ 1 0 1 1 0 0 1 บิตพาริตีสุดท้ายจะต้องเป็น 0 หรือ 1 เพราะเหตุใด

ระดับที่ 2 การวิเคราะห์และประยุกต์ใช้

3. ให้นักเรียนเขียนชุดคำสั่งการ์ดหุ่นยนต์ (Forward, Turn Left, Turn Right) เพื่อพาหุ่นยนต์เดินจากจุด (1, 1) ไปยัง (4, 5) บนตาราง 2 มิติ โดยหลีกเลี่ยงหลุมพรางที่จุด (2, 3) และ (3, 4)

ระดับที่ 3 การคิดขั้นสูงและบูรณาการ

4. จงออกแบบกิจกรรม Unplugged สำหรับสอนเรื่อง "การค้นหาข้อมูลแบบทวิภาค (Binary Search)" โดยใช้อุปกรณ์ในชีวิตประจำวัน เช่น กล่องของข้าวญ หรือ พจนานุกรมเล่มหนา พร้อมเขียนใบบันทึกผลและเกณฑ์การประเมิน

วิทยาการคำนวณ 1 รากฐานการคิดเชิงคำนวณและการแก้ปัญหาเชิงตรรกะ

บทที่ 5 โครงสร้างข้อมูลพื้นฐานและขั้นตอนวิธีการค้นหา

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชวีระ ทัศนาศ • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

แผนบริหารการสอนประจำบทที่ 5

หัวข้อเนื้อหาประจำบท

- โครงสร้างข้อมูลเชิงเส้น (Linear Data Structures: Arrays, Stacks, Queues)
- หลักการทํางานของ Stack (LIFO) และ Queue (FIFO)
- สถาปัตยกรรมหน่วยความจำแบบต่อเนื่อง (Contiguous Memory Allocation)
- ขั้นตอนวิธีการค้นหาแบบเชิงเส้น และการค้นหาแบบทวิภาค
- การวิเคราะห์ความซับซ้อนเชิงเวลา Big-O ของการค้นหาข้อมูล

วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- อธิบายความแตกต่างของโครงสร้างข้อมูล Stack, Queue, และ Array ได้อย่างถูกต้อง
- คำนวณจำนวนรอบการค้นหาของ Binary Search บนชุดข้อมูลขนาดใหญ่ได้
- เขียนโปรแกรมภาษา Python จำลองการทำงานของ Stack, Queue, และ Binary Search ได้
- เลือกใช้โครงสร้างข้อมูลและอัลกอริทึมการค้นหาที่เหมาะสมกับโจทย์ปัญหาจริงได้

กิจกรรมการเรียนรู้การสอน

- การบรรยายเชิงวิชาการและการเชื่อมโยงบริบทประวัติศาสตร์วิทยาการคำนวณ
- การสาธิตการวิเคราะห์คณิตศาสตร์ การจัดสรรหน่วยความจำ และโครงสร้างข้อมูล
- การฝึกปฏิบัติการจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands
- การเขียนโปรแกรมภาษา Python 3.11 และการทดสอบ Assertion Tests เชิงประจักษ์

สื่อการเรียนรู้การสอน

- ตำราเรียนวิชาการ วิทยาการคำนวณ 1 รากฐานการคิดเชิงคำนวณและการแก้ปัญหาเชิงตรรกะ
- ชุดห้องปฏิบัติการเสมือนจริง Hybrid 2D/3D บนระบบ RBRU MOOC
- สไลด์บรรยายอิเล็กทรอนิกส์และแผนภาพเวกเตอร์มัลติมีเดีย

การวัดและประเมินผล

- การประเมินผลการทำงานและตารางบันทึกผลการทดลองเสมือนจริง (40%)
- การประเมินผลงานการเขียนโค้ดภาษา Python และ Unit Test Assertions (30%)
- การทดสอบวัดผลสัมฤทธิ์ทางการเรียนท้ายบท 3 ระดับ (30%)

5.0 เรื่องเล่าเปิดบทเรียนและบริบททางประวัติศาสตร์

ในคริสต์ทศวรรษ 1940 จอห์น ฟอน นอยมันน์ (John von Neumann, 1903–1957) นักคณิตศาสตร์ชาวอเมริกันเชื้อสายฮังการี ได้วางรากฐานสถาปัตยกรรมคอมพิวเตอร์แบบเก็บโปรแกรม (Von Neumann Architecture) ซึ่งกำหนดให้หน่วยความจำหลัก (RAM) ทำหน้าที่เก็บทั้งคำสั่งและข้อมูลในรูปแบบลำดับที่ต่อเนื่องกัน

เมื่อข้อมูลถูกจัดเรียงตามลำดับอย่างเป็นระเบียบ มนุษย์สามารถใช้หลักการ "แบ่งแยกและเอาชนะ (Divide and Conquer)" ในการค้นหาข้อมูล เช่น การเปิดหาคำศัพท์ในพจนานุกรมเล่มหนา 1,000 หน้า โดยการเปิดที่กึ่งกลางเสมอ ซึ่งทำให้เราค้นหาที่ต้องการเจอได้ภายในไม่เกิน 10 ครั้ง ($2^{10} = 1024$) แทนที่จะต้องไล่ทีละหน้าตั้งแต่หน้าแรกถึง 1,000 หน้า

5.1 ทฤษฎีและรากฐานทางคณิตศาสตร์เชิงลึก

คณิตศาสตร์ของการค้นหาแบบทวิภาค

กำหนดให้อาเรย์ A มีสมาชิก n ตัวที่เรียงลำดับแล้ว ในแต่ละขั้นตอนของการค้นหา ช่วงการค้นหาจะลดลงครึ่งหนึ่งเสมอ

- รอบที่ 0: ขนาดช่วง = n
- รอบที่ 1: ขนาดช่วง = $\frac{n}{2}$
- รอบที่ 2: ขนาดช่วง = $\frac{n}{4} = \frac{n}{2^2}$
- รอบที่ k : ขนาดช่วง = $\frac{n}{2^k}$

การค้นหาจะสิ้นสุดลงเมื่อช่วงการค้นหาเหลือสมาชิกเพียง 1 ตัว นั่นคือ $\frac{n}{2^k} = 1 \implies 2^k = n$ ทำการใส่ฟังก์ชันลอการิทึมฐาน 2 ทั้งสองข้าง

$$k = \log_2 n$$

นั่นคือ ความซับซ้อนเชิงเวลาในกรณีแย่ที่สุด (Worst-Case Time Complexity) ของ Binary Search คือ

$$T(n) = O(\log n)$$

```
graph TD
    Root["Binary Search Interval: [0 .. n-1]"]
    Root -->|Target < Mid| Left["Left Half: [0 .. Mid-1] (ตัดครึ่งขวาทิ้ง)"]
    Root -->|Target = Mid| Found["Found Target! 🏆 (จบการค้นหา)"]
    Root -->|Target > Mid| Right["Right Half: [Mid+1 .. n-1] (ตัดครึ่งซ้ายทิ้ง)"]
```

🧩 5.2 ตัวอย่างการวิเคราะห์และการคำนวณเชิงขั้นตอน

ตัวอย่างที่ 5.1 การค้นหาค่าในลิสต์ด้วย Binary Search

กำหนดลิสต์ $A = [12, 25, 33, 47, 56, 68, 79, 84, 91, 99]$ ขนาด $n = 10$ จงแสดงขั้นตอนการค้นหาค่า $Target = 79$:

วิธีทำ

- รอบที่ 1:** $L = 0, R = 9 \implies Mid = \lfloor (0 + 9)/2 \rfloor = 4$
ค่าที่ $A[4] = 56$
เนื่องจาก $79 > 56$ ดังนั้นค้นหาในครึ่งขวา: กำหนด $L = Mid + 1 = 5$
- รอบที่ 2:** $L = 5, R = 9 \implies Mid = \lfloor (5 + 9)/2 \rfloor = 7$
ค่าที่ $A[7] = 84$
เนื่องจาก $79 < 84$ ดังนั้นค้นหาในครึ่งซ้าย: กำหนด $R = Mid - 1 = 6$
- รอบที่ 3:** $L = 5, R = 6 \implies Mid = \lfloor (5 + 6)/2 \rfloor = 5$
ค่าที่ $A[5] = 68$
เนื่องจาก $79 > 68$ ดังนั้นค้นหาในครึ่งขวา: กำหนด $L = Mid + 1 = 6$
- รอบที่ 4:** $L = 6, R = 6 \implies Mid = 6$
ค่าที่ $A[6] = 79$
เนื่องจาก $A[6] == Target$ (พบข้อมูลที่ดัชนี 6 ใน 4 ขั้นตอน!)

🐍 5.3 การเขียนโปรแกรมและการนำไปใช้จริงด้วย Python 3.11

```
# binary_search_engine.py
# การค้นหาแบบทวิภาคพร้อมบันทึกประวัติการทำงานและ Unit Test
from typing import List, Tuple, Optional

def binary_search_trace(arr: List[int], target: int) -> Tuple[Optional[int], List[dict]]:
    """ค้นหา target ใน arr ที่เรียงลำดับแล้ว และคืนค่าดัชนีพร้อมประวัติการค้นหา"""
    left = 0
    right = len(arr) - 1
    history = []
    step = 1

    while left <= right:
        mid = (left + right) // 2
        val = arr[mid]

        history.append({
            "step": step, "left": left, "right": right,
            "mid": mid, "val": val, "match": val == target
        })

        if val == target:
            return mid, history
        elif val < target:
            left = mid + 1
        else:
            right = mid - 1
        step += 1

    return None, history

if __name__ == "__main__":
    dataset = [12, 25, 33, 47, 56, 68, 79, 84, 91, 99]
    target_val = 79
    idx, log = binary_search_trace(dataset, target_val)
    print(f"ผลการค้นหา {target_val}: พบที่ดัชนี {idx} ใน {len(log)} รอบ")
    assert idx == 6, f"Expected index 6, got {idx}"
    assert len(log) == 4, f"Expected 4 steps, got {len(log)}"
    print("✅ Unit Assertion Tests Passed 100%!")
```

👤 5.4 คู่มือห้องปฏิบัติการการเสมือนจริง 2D/3D AR MediaPipe Hands

ผู้เรียนสามารถเข้าสู่ห้องปฏิบัติการการเสมือนจริง 2D/3D เพื่อทดลองค้นหาข้อมูล $N = 1,000,000$ รายการได้ที่ [chapter05_binary_search.html](#)



5.5 สรุปสาระสำคัญประจำบท

- โครงสร้างข้อมูลเชิงเส้น (Array, Stack, Queue) มีข้อดีข้อเสียแตกต่างกันตามลักษณะการใช้งาน
- Binary Search มีประสิทธิภาพ $O(\log n)$ ซึ่งเร็วกว่า Linear Search $O(n)$ อย่างมหาศาลเมื่อข้อมูลมีขนาดใหญ่



5.6 แบบฝึกหัดและคำถามท้ายบทเพื่อการประเมินผล

- จงอธิบายหลักการทำงานของ Stack (LIFO) และ Queue (FIFO) พร้อมยกตัวอย่างการใช้งานในระบบปฏิบัติการ
- หากมีข้อมูลขนาด $n = 1,048,576$ ตัว ในกรณีแย่ที่สุด Binary Search จะใช้การเปรียบเทียบกี่ครั้ง?
- ให้นักเรียนเขียนคลาส Stack ในภาษา Python โดยใช้ลิสต์ พร้อมเมธอด `push()`, `pop()`, และ `peek()`



เอกสารอ้างอิงประจำบท

- Cormen, T. H. et al. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Knuth, D. E. (1997). *The Art of Computer Programming: Fundamental Algorithms* (Vol. 1). Addison-Wesley.

บทที่ 5 ตัวแปร ชนิดข้อมูล และการรับส่งข้อมูลเบื้องต้น

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทศนา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

📌 ผลลัพธ์การเรียนรู้ประจำบท

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

1. **อธิบาย** หลักการทำงานของตัวแปร หน่วยความจำ และความแตกต่างระหว่างชนิดข้อมูลพื้นฐานในคอมพิวเตอร์
2. **เลือกใช้และแปลง** ชนิดข้อมูล (int, float, str, bool) ให้สอดคล้องกับตัวแปรทางวิทยาศาสตร์และคณิตศาสตร์ได้อย่างถูกต้อง
3. **คำนวณนิพจน์** ตามลำดับความสำคัญของตัวดำเนินการ (Operator Precedence / PEMDAS) ได้อย่างแม่นยำ
4. **เขียนโปรแกรม** เพื่อรับข้อมูลเข้าจากผู้ใช้ ประมวลผล และแสดงผลลัพธ์ด้วย f-string อย่างสวยงามและเป็นมืออาชีพ

📺 5.0 เรื่องเล่าเปิดบทเรียน กล้องความจำและรหัสพันธุกรรมดิจิทัล

เมื่อยานสำรวจอวกาศ **Voyager 1** ส่งสัญญาณข้อมูลระยะไกลกลับมายังโลกจากระยะทางกว่า 24,000 ล้านกิโลเมตร สัญญาณที่เดินทางผ่านคลื่นวิทยุเป็นเพียงกระแสตัวเลขฐานสอง **0** และ **1** ที่ไม่มีป้ายบอกว่าตัวเลขไหนคือ อุณหภูมิของอวกาศ, ตัวเลขไหนคือ ความเร็วของยาน, หรือตัวเลขไหนคือ ภาพถ่ายของดาวเสาร์

สิ่งที่ทำให้นักวิทยาศาสตร์ของ NASA อ่านข้อมูลเหล่านี้เข้าใจได้คือ **ระบบชนิดข้อมูล (Data Types) และตัวแปร (Variables)** ที่กำหนดไว้ล่วงหน้าว่า

- ข้อมูล 16 บิตแรก ให้ความเป็น ****จำนวนเต็ม**** แทนรหัสเซนเซอร์
- ข้อมูล 32 บิตถัดมา ให้ความเป็น ****ทศนิยม**** แทนค่าสนามแม่เหล็กในหน่วยเทสลา
- ข้อมูล 64 บิตถัดมา ให้ความเป็น ****ข้อความ**** แทนชื่ออุปกรณ์

```
graph LR
    RawStream["กระแสบิตดิบ: \n01000001 01000010 00110001"] --> Parser["ตัวแปรและชนิดข้อมูล (Data Types)"]
    Parser --> T1["Integer: 65 (รหัสเซนเซอร์)"]
    Parser --> T2["Float: 3.14159 (ค่าสนามแม่เหล็ก)"]
    Parser --> T3["String: 'VOYAGER' (ชื่ออุปกรณ์)"]
```

ตัวแปรและชนิดข้อมูลจึงเปรียบเสมือน **กล่องความจำติดป้ายชื่อ** ที่ช่วยให้คอมพิวเตอร์และมนุษย์เข้าใจความหมายของข้อมูลได้อย่างถูกต้องตรงกัน

📦 5.1 แนวคิดเรื่องตัวแปรและกฎการตั้งชื่อ

****ตัวแปร**** คือ ชื่อที่ถูกกำหนดขึ้นเพื่อใช้อ้างอิงถึงพื้นที่จัดเก็บข้อมูลในหน่วยความจำแรม (RAM) ของคอมพิวเตอร์

📌 กฎหลักในการตั้งชื่อตัวแปรในภาษา Python (Identifiers Rules)

1. ต้องขึ้นต้นด้วย **ตัวอักษรภาษาอังกฤษ (A-Z, a-z)** หรือเครื่องหมาย **ขีดล่าง (Underscore: _)** เท่านั้น (ห้ามขึ้นต้นด้วยตัวเลข)
2. ตัวอักษรพิมพ์เล็กและพิมพ์ใหญ่มีความหมายแตกต่างกัน (**Case-sensitive** เช่น `temp`, `Temp`, `TEMP` คือคนละตัวแปร)
3. ห้ามมีช่องว่าง (Space) หรืออักขระพิเศษ เช่น `@`, `$`, `%`, `#`, `-`, `+`, `*`, `/`
4. ห้ามใช้คำสงวนในภาษา Python (Reserved Keywords เช่น `if`, `else`, `for`, `while`, `def`, `class`, `import`, `True`, `False`)
5. **รูปแบบแนะนำ (PEP 8):** ใช้แบบ `snake_case` สำหรับตัวแปรทั่วไป (เช่น `student_score`, `laser_wavelength`) และ `ALL_CAPS` สำหรับค่าคงที่ (เช่น `SPEED_OF_LIGHT = 299792458`)

🎨 5.2 สืบชนิดข้อมูลพื้นฐาน

```
graph TD
    DT["ชนิดข้อมูลพื้นฐานใน Python"]
    DT --> INT["1. int (Integer)\nจำนวนเต็ม เช่น -5, 0, 42"]
    DT --> FLT["2. float (Floating-point)\nจำนวนจริง/ทศนิยม เช่น 3.14, -0.001, 2.5e8"]
    DT --> STR["3. str (String)\nข้อความ/สตริง เช่น 'Physics', 'ผศ.ดร.ชีวะ'"]
    DT --> BOL["4. bool (Boolean)\nค่าความจริงทางตรรกะ มีเพียง True หรือ False"]
```

การแปลงชนิดข้อมูล

เมื่อรับข้อมูลเข้าผ่านฟังก์ชัน `input()` คอมพิวเตอร์จะคืนค่าเป็น **String เสมอ** ดังนั้นจำเป็นต้องแปลงชนิดข้อมูลก่อนนำไปคำนวณทางคณิตศาสตร์

```
raw_input = input("กรุณากรอกระยะทาง (เมตร): ") # ได้ str เช่น "150.5"
distance = float(raw_input) # แปลงเป็น float เพื่อนำไปคำนวณ
```

ตัวดำเนินการ	สัญลักษณ์	ตัวอย่างคำนวณ	ผลลัพธ์	ความหมายทางฟิสิกส์/คณิตศาสตร์
**การบวก **	+	10 + 5	15	การรวมปริมาณสเกลาร์
**การลบ **	-	10 - 5	5	การหาผลต่างหรือการกระจาย
**การคูณ **	*	10 * 5	50	การคำนวณพื้นที่ / แรง $F = ma$
**การหารจริง **	/	10 / 4	2.5 (float)	การคำนวณความเร็ว $v = s/t$
**การหารปัดเศษ **	//	10 // 4	2 (int)	การหาจำนวนรอบที่เต็มหน่วย
มอดุโล/เศษเหลือ (Modulo)	%	10 % 4	2	การตรวจสอบเลขคู่/ คาบเวลา
**การยกกำลัง **	**	2 ** 3	8	การคำนวณพลังงานจลน์ v^2

⚡ ลำดับความสำคัญของตัวดำเนินการ (Operator Precedence / PEMDAS)

1. P - Parentheses: วงเล็บ (...) ทำงานก่อนเสมอ
2. E - Exponentiation: ยกกำลัง **
3. MD - Multiplication & Division: คูณ หาร มอดุโล *, / , // , % (จากซ้ายไปขวา)
4. AS - Addition & Subtraction: บวก ลบ + , - (จากซ้ายไปขวา)

🔧 5.4 ตัวอย่างโจทย์คำนวณฟิสิกส์ โปรแกรมคำนวณพลังงานจลน์ (E_k)

🔧 ตัวอย่างที่ 5.1: การคำนวณพลังงานจลน์ของยานยนต์ EV

สูตรฟิสิกส์พื้นฐาน

โจทย์: รถยนต์ไฟฟ้ามวล $m = 1,500$ kg กำลังเคลื่อนที่ด้วยความเร็ว $v = 20$ m/s (72 km/h) จงเขียนโปรแกรมคำนวณพลังงานจลน์ตามสมการ $E_k = \frac{1}{2}mv^2$ พร้อมจัดรูปแบบการแสดงผลด้วย f-string

```
<details style="background: #f8fafc; border: 1px solid #cbd5e1; border-radius: 10px; padding: 14px 18px; margin-top: 12px
<summary style="font-weight: 700; color: #0284c7;">🔧 คลิกเพื่อดูวิธีทำและขั้นตอนคำนวณอย่างละเอียด</summary>
<div style="margin-top: 14px; border-top: 1px dashed #cbd5e1; padding-top: 14px;">
  <ol style="padding-left: 20px;">
    <li><strong>กำหนดตัวแปร:</strong> <code>mass = 1500.0</code>, <code>velocity = 20.0</code></li>
    <li><strong>ตั้งสมการคำนวณ:</strong> <code>ek = 0.5 * mass * (velocity ** 2)</code></li>
    <li><strong>แทนค่า:</strong> $E_k = 0.5 \times 1500 \times (20^2) = 750 \times 400 = 300,000\text{ จูล (J)}$ หรือ $300'
    <li><strong>แสดงผลด้วย f-string:</strong> <code>print(f"พลังงานจลน์ = {ek:,.2f} J ({ek/1000:.1f} kJ)"</code></li>
  </ol>
</div>
</details>
```

```
# kinetic_energy_calculator.py
# โปรแกรมคำนวณพลังงานจลน์และแสดงผลข้อมูลเชิงวิทยาศาสตร์
# ผู้เขียน: ผศ.ดร. ชีวะ ทิศนา (มรภ.รำไพพรรณี)

def calculate_kinetic_energy(mass_kg: float, velocity_ms: float) -> float:
    """คำนวณพลังงานจลน์จากสูตร Ek = 0.5 * m * v^2"""
    return 0.5 * mass_kg * (velocity_ms ** 2)

def main():
    print("=" * 65)
    print("🚗 โปรแกรมคำนวณพลังงานจลน์ของยานยนต์ (Kinetic Energy Calculator)")
    print("=" * 65)

    try:
        # รับข้อมูลเข้าและแปลงชนิดข้อมูล
        car_model = input("ระบุรุ่นของยานยนต์: ").strip()
        mass_input = float(input("กรูณากรอกมวลของยานยนต์ (kg): "))
        velocity_kmh = float(input("กรูณากรอกความเร็วของยานยนต์ (km/h): "))

        # ตรวจสอบความสมเหตุสมผลของข้อมูล
        if mass_input <= 0 or velocity_kmh < 0:
            print("⚠️ ข้อผิดพลาด: มวลและความเร็วต้องเป็นค่าบวกเท่านั้น!")
            return

        # แปลงความเร็วจาก km/h เป็น m/s (หารด้วย 3.6)
        velocity_ms = velocity_kmh / 3.6

        # คำนวณพลังงาน
        ek_joules = calculate_kinetic_energy(mass_input, velocity_ms)
        ek_kilojoules = ek_joules / 1000.0

        # แสดงผลลัพธ์ด้วย f-string จัดรูปแบบตัวเลข
        print("\n" + "-" * 65)
        print(f"📊 ผลการวิเคราะห์พลังงานสำหรับ: {car_model}")
        print(f"    • มวลของยานยนต์: {mass_input:.2f} kg")
        print(f"    • ความเร็วการเคลื่อนที่: {velocity_kmh:.2f} km/h ({velocity_ms:.2f} m/s)")
        print(f"    • พลังงานจลน์สะสม (Ek): {ek_joules:.2f} จูล (J)")
        print(f"    • คิดเป็นกิโลจูล (kJ): {ek_kilojoules:.2f} kJ")
        print("-" * 65)

    except ValueError:
        print("❌ เกิดข้อผิดพลาด: กรุณากรอกตัวเลขที่ถูกต้องเท่านั้น!")

if __name__ == "__main__":
    main()
```

5.6 สรุปใจความสำคัญและแบบฝึกหัดท้ายบทที่ 5

📌 สรุปประเด็นสำคัญ

- ตัวแปรคือชื่ออ้างอิงพื้นที่ความจำในคอมพิวเตอร์ ต้องตั้งชื่อให้สื่อความหมายตามมาตรฐาน `snake_case`
- การเลือกชนิดข้อมูลที่ถูกต้อง (`int`, `float`, `str`, `bool`) มีผลโดยตรงต่อความถูกต้องและการใช้หน่วยความจำ
- การคำนวณทางคณิตศาสตร์ในคอมพิวเตอร์ยึดตามลำดับ **PEMDAS** เสมอ หากต้องการให้ทำส่วนใดก่อนต้องใส่วงเล็บ (...)

🎨 แบบฝึกหัดทบทวน 3 ระดับ

ระดับที่ 1 ความรู้ความเข้าใจพื้นฐาน

- ตัวแปรต่อไปนี้ ชื่อใดถูกต้องตามกฎการตั้งชื่อของ Python และชื่อใดผิดกฎ (พร้อมบอกเหตุผล):
 - `2nd_experiment`, `total_score`, `class`, `laser-wavelength`, `_sensorValue`
- ผลลัพธ์ของนิพจน์ต่อไปนี้ใน Python มีค่าเท่าใด และเป็นชนิดข้อมูลใด
 - (A) `10 + 3 * 2 ** 2`
 - (B) `17 // 5 + 17 % 5`
 - (C) `float(5) + int(20)`

ระดับที่ 2 การวิเคราะห์และประยุกต์ใช้

- จงเขียนโปรแกรม Python คำนวณค่าไฟฟ้าในบ้านตามสูตร

$$\text{หน่วยไฟฟ้า (Units)} = \frac{\text{กำลังวัตต์ (Watts)} \times \text{ชั่วโมงที่เปิดใช้งาน (Hours)}}{1000}$$

และคำนวณค่าไฟรวม = หน่วยไฟฟ้า $\times$ 4.50 บาท พร้อมแสดงผลลัพธ์ทศนิยม 2 ตำแหน่ง

4. จงเขียนโปรแกรมคำนวณดัชนีมวลกาย (BMI) ตามสูตร

$$BMI = \frac{\text{น้ำหนัก (kg)}}{(\text{ส่วนสูง (m)})^2}$$

โดยให้ผู้ใช้กรอกส่วนสูงในหน่วย **เซนติเมตร ** และน้ำหนักในหน่วย **กิโลกรัม ** พร้อมตรวจสอบว่าหากผู้ใช้กรอกค่าส่วนสูงหรือน้ำหนักติดลบ ให้แจ้งเตือนข้อผิดพลาดทันที

บทที่ 6 โครงสร้างทางเลือกและการตัดสินใจแบบมีเงื่อนไข

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

🎯 ผลลัพธ์การเรียนรู้ประจำบท

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

1. **อธิบาย** กลไกการทำงานของโครงสร้างการควบคุมแบบมีเงื่อนไขในคอมพิวเตอร์
2. **ประเมินค่า** นิพจน์เปรียบเทียบและนิพจน์ตรรกะแบบหลายชั้นได้อย่างถูกต้องแม่นยำ
3. **ออกแบบและเขียนโปรแกรม** โครงสร้างทางเลือก `if`, `if-else`, `if-elif-else` และเงื่อนไขซ้อน (Nested Conditions) เพื่อแก้ปัญหาในสถานการณ์จริง
4. **ประยุกต์ใช้** โครงสร้างเงื่อนไขในการพัฒนาระบบควบคุมอัตโนมัติและระบบเตือนภัยอัจฉริยะ

🚗 6.0 เรื่องเล่าเปิดบทเรียน วินาทีแห่งการตัดสินใจของยานยนต์ไร้คนขับ

เมื่อรถยนต์ขับเคลื่อนอัตโนมัติ (Autonomous Vehicle) แล่นอยู่บนท้องถนนด้วยความเร็ว 80 กิโลเมตรต่อชั่วโมง กล้องและเซนเซอร์ LiDAR จะส่งข้อมูลภาพหลายล้านจุดต่อวินาทีเข้าสู่สมองกลประมวลผล คอมพิวเตอร์ต้องทำการตัดสินใจในเสี้ยววินาที

```
graph TD
    LiDAR["เซนเซอร์ตรวจจับวัตถุด้านหน้า"] --> CheckDist{"ระยะทาง (d) < 15 เมตร ?"}
    CheckDist -- "ใช่ (True)" --> CheckSpeed{"ความเร็วปัจจุบัน (v) > 50 km/h ?"}
    CheckSpeed -- "ใช่ (True)" --> ActionEmergency["⚠️ สั่งเบรกฉุกเฉิน 100% ทันที!"]
    CheckSpeed -- "ไม่ใช่ (False)" --> ActionDecelerate["🚗 ชะลอความเร็วลงอย่างนุ่มนวล"]
    CheckDist -- "ไม่ใช่ (False)" --> ActionMaintain["🟢 รักษาระดับความเร็วปกติ"]
```

การตัดสินใจที่แม่นยำและรวดเร็วเช่นนี้ เกิดขึ้นจาก **โครงสร้างการควบคุมแบบมีเงื่อนไข** ซึ่งเป็นกลไกที่ทำให้คอมพิวเตอร์สามารถ คิดและเลือกปฏิบัติ ได้อย่างชาญฉลาดตามสถานการณ์แวดล้อม

⚖️ 6.1 ตัวดำเนินการเปรียบเทียบและตรรกะ

1. ตัวดำเนินการเปรียบเทียบ

คืนค่าเป็นบูลีน `True` หรือ `False` เสมอ

ตัวดำเนินการ	ความหมาย	ตัวอย่าง	ผลลัพธ์
<code>=</code>	เท่ากับ (Equal to)	<code>5 = 5</code>	<code>True</code>
<code>≠</code>	ไม่เท่ากับ (Not equal to)	<code>5 ≠ 3</code>	<code>True</code>
<code>></code>	มากกว่า (Greater than)	<code>10 > 20</code>	<code>False</code>
<code><</code>	น้อยกว่า (Less than)	<code>10 < 20</code>	<code>True</code>
<code>≥</code>	มากกว่าหรือเท่ากับ (Greater than or equal)	<code>15 ≥ 15</code>	<code>True</code>
<code>≤</code>	น้อยกว่าหรือเท่ากับ (Less than or equal)	<code>12 ≤ 8</code>	<code>False</code>

⚠️ ข้อผิดพลาดที่พบบ่อยมากที่สุดในมือใหม่!

- เครื่องหมายเท่ากับอันเดียว (`=`) คือ การกำหนดค่าตัวแปร (Assignment) เช่น `x = 10`
- เครื่องหมายเท่ากับคู่ (`=`) คือ การเปรียบเทียบค่าความเท่ากัน (Equality Comparison) เช่น `if x = 10:`

📄 6.2 สถาปัตยกรรมโครงสร้างทางเลือกใน Python

```
graph TD
    subgraph SelectionTypes["รูปแบบโครงสร้างทางเลือก"]
        S1["1. Single Alternative: if\n(ทำเมื่อเงื่อนไขจริงเท่านั้น)"]
        S2["2. Dual Alternative: if-else\n(เลือกทำทางใดทางหนึ่งแน่นอน)"]
        S3["3. Multi-way Alternative: if-elif-else\n(เลือกทำ 1 ทางจากหลายเงื่อนไข)"]
        S4["4. Nested Condition\n(เงื่อนไขซ้อนเงื่อนไขภายใน)"]
    end
    end
```

ในภาษา Python การเยื้องหน้า 4 ช่องว่าง (4 Spaces Indentation) มีความสำคัญอย่างยิ่ง เพราะเป็นการระบุขอบเขต (Block Scope) ของคำสั่งที่อยู่ภายใต้เงื่อนไข

```
# รูปแบบ if-elif-else มาตรฐาน
if condition_1:
    # คำสั่งที่ทำเมื่อ condition_1 เป็น True
    statement_block_1
elif condition_2:
    # คำสั่งที่ทำเมื่อ condition_1 เป็น False แต่ condition_2 เป็น True
    statement_block_2
else:
    # คำสั่งที่ทำเมื่อไม่มีเงื่อนไขใดเป็น True เลย
    statement_block_default
```

6.3 ตัวอย่างโจทย์ประยุกต์ ระบบควบคุมโรงเรือนเกษตรอัจฉริยะ

ตัวอย่างที่ 6.1: ระบบตัดสินใจควบคุมสปริงเกอร์และพัดลมระบายอากาศ

เกษตรอัจฉริยะ IoT

เงื่อนไขการทำงาน:

- หากอุณหภูมิ $Temp > 35^{\circ}C$ และ ความชื้นในดิน $Moisture < 40\%$ → เปิดพัดลมระบายอากาศและเปิดสปริงเกอร์รดน้ำทันที
- หากอุณหภูมิ $Temp > 35^{\circ}C$ แต่ความชื้นในดินเพียงพอ → เปิดเฉพาะพัดลมระบายอากาศ
- หากอุณหภูมิปกติ แต่ความชื้นในดิน $Moisture < 40\%$ → เปิดเฉพาะสปริงเกอร์รดน้ำ
- กรณีอื่นๆ สภาพแวดล้อมเหมาะสม → ปิดอุปกรณ์ทั้งหมด อยู่ในโหมดประหยัดพลังงาน

```
# smart_greenhouse_controller.py
# โปรแกรมจำลองระบบควบคุมสภาพแวดล้อมโรงเรือนเกษตรอัจฉริยะ
# ผู้เขียน: ผศ.ดร.ชิวะ ทิศนา (มรภ.รำไพพรรณี)

def evaluate_greenhouse_environment(temperature_celsius: float, soil_moisture_percent: float) → dict:
    """ประเมินสถานะและสั่งการอุปกรณ์ควบคุมอัตโนมัติ"""
    fan_status = False
    sprinkler_status = False
    alert_message = ""

    if temperature_celsius > 35.0 and soil_moisture_percent < 40.0:
        fan_status = True
        sprinkler_status = True
        alert_message = "🔥 วิกฤต: อุณหภูมิสูงจัดและดินแห้งแล้ง รดน้ำและระบายความร้อนเต็มกำลัง!"
    elif temperature_celsius > 35.0:
        fan_status = True
        sprinkler_status = False
        alert_message = "☀️ อุณหภูมิสูง: เปิดพัดลมระบายความร้อน"
    elif soil_moisture_percent < 40.0:
        fan_status = False
        sprinkler_status = True
        alert_message = "💧 ดินแห้ง: เปิดระบบสปริงเกอร์พ่นละอองน้ำ"
    else:
        fan_status = False
        sprinkler_status = False
        alert_message = "🌱 สภาพแวดล้อมเหมาะสม: ระบบทำงานในโหมดประหยัดพลังงาน"

    return {
        "fan": "ON" if fan_status else "OFF",
        "sprinkler": "ON" if sprinkler_status else "OFF",
        "message": alert_message
    }

def main():
    print("=" * 65)
    print("🌿 ระบบควบคุมโรงเรือนเกษตรอัจฉริยะ (Smart Greenhouse Controller)")
    print("=" * 65)

    # ชุดข้อมูลทดสอบ 4 สถานการณ์
    test_cases = [
        {"name": "ช่วงเที่ยงวันแดดจัดแล้ง", "temp": 38.5, "moisture": 25.0},
        {"name": "ช่วงบ่ายลมร้อนดินชุ่ม", "temp": 36.2, "moisture": 65.0},
        {"name": "ช่วงเช้าอากาศเย็นแต่ดินแห้ง", "temp": 28.0, "moisture": 32.0},
        {"name": "ช่วงเย็นสภาพอากาศสมบูรณ์", "temp": 29.5, "moisture": 55.0},
    ]

    for case in test_cases:
        result = evaluate_greenhouse_environment(case["temp"], case["moisture"])
        print(f"\n📋 กรณี: {case['name']}")
        print(f"    • เซนเซอร์: อุณหภูมิ {case['temp']:.1f}°C | ความชื้นดิน {case['moisture']:.1f}%")
        print(f"    • สถานะพัดลม: [{result['fan']}]")
        print(f"    • สถานะสปริงเกอร์: [{result['sprinkler']}]")
        print(f"    • ข้อความระบบ: {result['message']}")

    print("\n" + "=" * 65)

if __name__ == "__main__":
    main()
```

💡 6.5 สรุปใจความสำคัญและแบบฝึกหัดท้ายบทที่ 6

🚀 สรุปประเด็นสำคัญ

1. โครงสร้างเงื่อนไขทำให้โปรแกรมมีความยืดหยุ่นและปรับพฤติกรรมได้ตามข้อมูลนำเข้า
2. การใช้ `if-elif-else` มีประสิทธิภาพสูงกว่าการเขียน `if` เดี่ยวๆ หลายตัวติดกัน เพราะคอมพิวเตอร์จะหยุดตรวจสอบทันทีที่เจอเงื่อนไขที่เป็นจริงเงื่อนไขแรก
3. ควรระมัดระวังการเปรียบเทียบ `=` และการเยื้องหน้า 4 ช่องว่าง

📌 แบบฝึกหัดทบทวน 3 ระดับ

ระดับที่ 1 ความรู้ความเข้าใจพื้นฐาน

1. จงระบุผลลัพธ์ของโค้ดต่อไปนี้

```
x = 15
if x > 20:
    print("A")
elif x > 10:
    print("B")
elif x > 5:
    print("C")
else:
    print("D")
```

2. จงแปลงเงื่อนไขทางคณิตศาสตร์ $10 \leq score \leq 20$ ให้อยู่ในรูปนิพจน์ภาษา Python ที่ถูกต้อง

ระดับที่ 2 การวิเคราะห์และประยุกต์ใช้

3. จงเขียนโปรแกรมรับค่าน้ำหนักพัสดุ (กิโลกรัม) และคำนวณค่าจัดส่งตามตารางอัตราค่าหัว
- น้ำหนักไม่เกิน 1 kg: 40 บาท
 - น้ำหนักเกิน 1 kg แต่ไม่เกิน 5 kg: 70 บาท
 - น้ำหนักเกิน 5 kg แต่ไม่เกิน 10 kg: 120 บาท
 - น้ำหนักเกิน 10 kg ขึ้นไป: 120 บาท + กิโลกรัมละ 20 บาทส่วนที่เกิน

ระดับที่ 3 การคิดขั้นสูงและบูรณาการ

4. จงเขียนโปรแกรมตรวจสอบว่าปี พ.ศ. หรือ ค.ศ. ที่รับเข้ามาเป็น "ปีอธิกสุรทิน (Leap Year)" หรือไม่ โดยมีกฎทางดาราศาสตร์คือ
- ปี ค.ศ. ที่หารด้วย 4 ลงตัว เป็นปีอธิกสุรทิน
 - ยกเว้น ปีที่หารด้วย 100 ลงตัว จะไม่เป็นปีอธิกสุรทิน
 - แต่ถ้า ปีนั้นหารด้วย 400 ลงตัว จะกลับมาเป็นปีอธิกสุรทินอีกครั้ง

วิทยาการคำนวณ 1 รากฐานการคิดเชิงคำนวณและการแก้ปัญหาเชิงตรรกะ

บทที่ 6 ขั้นตอนวิธีการจัดเรียงข้อมูลและการวิเคราะห์ความซับซ้อน

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

แผนบริหารการสอนประจำบทที่ 6

หัวข้อเนื้อหาประจำบท

- ความสำคัญของการจัดเรียงข้อมูลในระบบคอมพิวเตอร์
- อัลกอริทึมพื้นฐาน: Bubble Sort, Selection Sort, Insertion Sort ($O(n^2)$)
- อัลกอริทึมขั้นสูง: Merge Sort, Quick Sort ($O(n \log n)$)
- การพิสูจน์ขอบเขตล่างของการจัดเรียงแบบเปรียบเทียบ $\Omega(n \log n)$
- ความเสถียรของการจัดเรียง (Sorting Stability) และการใช้หน่วยความจำในที่เดิม (In-place Sorting)

วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- อธิบายและจำลองขั้นตอนการทำงานของ Bubble, Insertion, Merge, และ Quick Sort ได้
- เปรียบเทียบความซับซ้อนเชิงเวลาใน Best, Average, และ Worst Cases ของแต่ละอัลกอริทึมได้
- เขียนโปรแกรมภาษา Python เพื่อจัดเรียงข้อมูลและวัดประสิทธิภาพเชิงประจักษ์ได้
- เลือกใช้อัลกอริทึมการจัดเรียงที่เหมาะสมกับขนาดและลักษณะของข้อมูลได้

กิจกรรมการเรียนรู้การสอน

- การบรรยายเชิงวิชาการและการเชื่อมโยงบริบทประวัติศาสตร์วิทยาการคำนวณ
- การสาธิตการวิเคราะห์คณิตศาสตร์ การจัดสรรหน่วยความจำ และโครงสร้างข้อมูล
- การฝึกปฏิบัติการจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands
- การเขียนโปรแกรมภาษา Python 3.11 และการทดสอบ Assertion Tests เชิงประจักษ์

สื่อการเรียนรู้การสอน

- ตำราเรียนวิชาการ วิทยาการคำนวณ 1 รากฐานการคิดเชิงคำนวณและการแก้ปัญหาเชิงตรรกะ
- ชุดห้องปฏิบัติการเสมือนจริง Hybrid 2D/3D บนระบบ RBRU MOOC
- สไลด์บรรยายอิเล็กทรอนิกส์และแผนภาพเวกเตอร์มัลติมีเดีย

การวัดและประเมินผล

- การประเมินผลการทำงานและตารางบันทึกผลการทดลองเสมือนจริง (40%)
- การประเมินผลงานการเขียนโค้ดภาษา Python และ Unit Test Assertions (30%)
- การทดสอบวัดผลสัมฤทธิ์ทางการเรียนท้ายบท 3 ระดับ (30%)

6.0 เรื่องเล่าเปิดบทเรียนและบริบททางประวัติศาสตร์

ในคริสต์ศักราช 1945 จอห์น ฟอน นอยมันน์ ได้คิดค้นขั้นตอนวิธี **Merge Sort** ซึ่งเป็นอัลกอริทึมการจัดเรียงข้อมูลแบบแบ่งแยกและเอาชนะตัวแรกของโลก ต่อมาในปี 1959 เซอร์ โทนี ฮอว์ก (Sir Charles Antony Richard Hoare, 1934–ปัจจุบัน) นักวิทยาการคอมพิวเตอร์ชาวอังกฤษ ได้คิดค้น **Quick Sort** ซึ่งเป็นอัลกอริทึมการจัดเรียงที่ได้รับความนิยมสูงสุดในระบบปฏิบัติการและไลบรารีมาตรฐานทั่วโลกตราบจนปัจจุบัน

6.1 ทฤษฎีและรากฐานทางคณิตศาสตร์เชิงลึก

การพิสูจน์ขอบเขตล่างของการจัดเรียงแบบเปรียบเทียบ

การจัดเรียงข้อมูล n ตัว สามารถมองเป็นต้นไม้การตัดสินใจ (Decision Tree) ที่มีใบไม้แทนการเรียงสับเปลี่ยนที่เป็นไปได้ทั้งหมด $n!$ แบบ ความสูงของต้นไม้การตัดสินใจ h คือจำนวนครั้งการเปรียบเทียบที่น้อยที่สุดในกรณีแย่ที่สุด

$$2^h \geq n! \implies h \geq \log_2(n!)$$

จากสูตรการประมาณค่าของสเตอร์ลิง (Stirling's Approximation): $\ln(n!) \approx n \ln n - n$

$$\log_2(n!) = \Theta(n \log n)$$

ดังนั้น ไม่มีอัลกอริทึมการจัดเรียงแบบเปรียบเทียบใดๆ ในโลกที่ทำได้เร็วกว่า $\Omega(n \log n)$ ในกรณีแย่ที่สุด!

```
graph TD
    Sort["การจำแนกประเภท Sorting Algorithms"]
    Sort --> Q["Quad[\"กลุ่ม  $O(n^2)$  : พื้นฐาน ข้อมูลขนาดเล็ก  
• Bubble Sort, Selection Sort, Insertion Sort\"]"]
    Sort --> NLogN["NLogN[\"กลุ่ม  $O(n \log n)$  : ประสิทธิภาพสูง ข้อมูลขนาดใหญ่  
• Merge Sort (Stable), Quick Sort (In-place)\"]"]
```

6.2 ตัวอย่างการวิเคราะห์และการคำนวณเชิงขั้นตอน

ตัวอย่างที่ 6.1 การจำลองขั้นตอนของ Insertion Sort

กำหนดให้อาร์เรย์ $A = [5, 2, 4, 6, 1, 3]$ จงแสดงสถานะของอาร์เรย์หลังจบรอบที่ $i = 1, 2, 3$:

วิธีทำ

- เริ่มต้น $[5, 2, 4, 6, 1, 3]$
- รอบที่ 1 ($key = 2$): แทรก 2 หน้า 5 $\Rightarrow [2, 5, 4, 6, 1, 3]$
- รอบที่ 2 ($key = 4$): แทรก 4 ระหว่าง 2 และ 5 $\Rightarrow [2, 4, 5, 6, 1, 3]$
- รอบที่ 3 ($key = 6$): 6 มากกว่า 5 อยู่ตำแหน่งเดิม $\Rightarrow [2, 4, 5, 6, 1, 3]$
- รอบที่ 4 ($key = 1$): แทรก 1 หน้าที่สุด $\Rightarrow [1, 2, 4, 5, 6, 3]$
- รอบที่ 5 ($key = 3$): แทรก 3 ระหว่าง 2 และ 4 $\Rightarrow [1, 2, 3, 4, 5, 6]$ (จัดเรียงสมบูรณ์)

6.3 การเขียนโปรแกรมและการนำไปใช้จริงด้วย Python 3.11

```
# sorting_algorithms_benchmark.py
import time, random
from typing import List

def quick_sort(arr: List[int]) -> List[int]:
    """Quick Sort  $O(N \log N)$  Implementation"""
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

def bubble_sort(arr: List[int]) -> List[int]:
    """Bubble Sort  $O(N^2)$  Implementation"""
    a = arr.copy()
    n = len(a)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                swapped = True
        if not swapped:
            break
    return a

if __name__ == "__main__":
    test_data = [random.randint(1, 1000) for _ in range(1000)]
    t0 = time.perf_counter()
    _ = quick_sort(test_data)
    t_quick = time.perf_counter() - t0

    t0 = time.perf_counter()
    _ = bubble_sort(test_data)
    t_bubble = time.perf_counter() - t0

    print(f"Quick Sort (1000 items): {t_quick:.6f} s")
    print(f"Bubble Sort (1000 items): {t_bubble:.6f} s (ช้ากว่า {(t_bubble/t_quick):.1f} เท่า)")
    assert quick_sort([3, 1, 2]) == [1, 2, 3], "Test Passed!"
```

6.4 คู่มือห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands

ผู้เรียนสามารถเข้าสู่ชุดจำลองเสมือนจริง 2D/3D เพื่อทดลองประลองความเร็วของอัลกอริทึมที่จัดเรียงได้ที่ [chapter06_sorting_arena.html](https://github.com/aregulyan/chapter06_sorting_arena.html)

6.5 สรุปสาระสำคัญของบทเรียน

- อัลกอริทึม $O(n^2)$ เหมาะกับข้อมูลขนาดเล็กและโค้ดที่ต้องการความเรียบง่าย
- Merge Sort และ Quick Sort เป็นตัวเลือกหลักในการประมวลผลข้อมูลขนาดใหญ่ด้วยประสิทธิภาพ $O(n \log n)$

? 6.6 แบบฝึกหัดและคำถามท้ายบทเพื่อการประเมินผล

1. จงอธิบายความหมายของ "Stable Sorting" และเหตุใดจึงสำคัญในการจัดเรียงฐานข้อมูลหลายคอลัมน์
2. ให้นักเรียนวิเคราะห์ Worst Case ของ Quick Sort ว่าเกิดขึ้นเมื่อใด และมีวิธีป้องกันอย่างไร?



เอกสารอ้างอิงประจำบท

- Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1), 10-15.
- Sedgewick, R. (1978). Implementing Quicksort programs. *Communications of the ACM*, 21(10), 847-857.

บทที่ 7 โครงสร้างการทำงานซ้ำและการวนลูป

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทศนา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

🎯 ผลลัพธ์การเรียนรู้ประจำบท

- เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ
1. **อธิบาย** ความแตกต่างและหลักการเลือกระหว่างการวนซ้ำที่ทราบจำนวนรอบแน่นอน (`for`) และการวนซ้ำตามเงื่อนไข (`while`)
 2. **ประยุกต์ใช้** ฟังก์ชัน `range()` ในการสร้างลำดับตัวเลขทั้งแบบนับเพิ่ม นับลด และการข้ามช่วง
 3. **ควบคุมทิศทางการวนลูป** ด้วยคำสั่ง `break` และ `continue` ได้อย่างมีประสิทธิภาพและป้องกันปัญหาลูปไม่รู้จบ (Infinite Loop)
 4. **พัฒนาโปรแกรม** เพื่อคำนวณผลรวมอนุกรมทางคณิตศาสตร์ แฟกทอเรียล และการจำลองข้อมูลทางวิทยาศาสตร์

📺 7.0 เรื่องเล่าเปิดบทเรียน ปลงแครงการทำให้ซ้ำล้านครั้งในเสี้ยววินาที

หากขอให้มนุษย์บวกเลข $1 + 2 + 3 + \dots + 1,000,000$ ด้วยมือเปล่า เราอาจต้องใช้เวลากว่าหลายสัปดาห์และมีความเป็นไปได้สูงมากที่จะบวกเลขผิดพลาด แต่สำหรับไมโครโปรเซสเซอร์ในคอมพิวเตอร์ การคำนวณซ้ำแบบนี้ 1 ล้านรอบ ใช้เวลาเพียง **ไม่ถึง 0.005 วินาที** และมีความแม่นยำ 100%

```
graph LR
    Human["มนุษย์: เหนื่อยล้า เบื่อหน่าย\ความแม่นยำลดลงเมื่อทำซ้ำ"]
    Computer["คอมพิวเตอร์: ทำซ้ำหลายล้านรอบ\ด้วยความเร็วแสงและแม่นยำคงที่ 100%"]
```

พลังที่แท้จริงของวิทยาการคอมพิวเตอร์จึงอยู่ที่ **โครงสร้างการทำงานซ้ำ** ซึ่งช่วยลดการเขียนโค้ดที่ซ้ำซ้อนจากหลายพันบรรทัดให้เหลือเพียงไม่กี่บรรทัดได้อย่างน่าอัศจรรย์

🔄 7.1 การวนซ้ำด้วย for loop และฟังก์ชัน range()

ใช้เมื่อ **ทราบจำนวนรอบของการวนซ้ำที่แน่นอนล่วงหน้า**

```
graph TD
    ForStart["for i in range(start, stop, step):"] --> CheckNext["ยังมีสมาชิกลำดับถัดไป ?"]
    CheckNext -- "มี (True)" --> RunBody["ปฏิบัติคำสั่งในบล็อกลูป\น(พร้อมอัปเดตตัวแปร i)"]
    RunBody --> CheckNext
    CheckNext -- "หมดแล้ว (False)" --> ForEnd["ออกจากลูป ทำงานต่อ"]
```

ฟังก์ชัน `range` :

- `range(5)` → สร้างลำดับ 0, 1, 2, 3, 4 (เริ่มต้นที่ 0 และสิ้นสุดก่อน 5 เสมอ)
- `range(1, 11)` → สร้างลำดับ 1, 2, 3, ..., 10
- `range(2, 21, 2)` → สร้างลำดับเลขคู่ 2, 4, 6, ..., 20
- `range(10, 0, -1)` → นับถอยหลัง 10, 9, 8, ..., 1

🔄 7.2 การวนซ้ำด้วย while loop และการป้องกันลูปไม่รู้จบ

ใช้เมื่อ **ไม่ทราบจำนวนรอบที่แน่นอน แต่ต้องการให้ทำซ้ำไปเรื่อยๆ ตราบใดที่เงื่อนไขยังเป็นจริง**

```
# รูปแบบ while loop มาตรฐาน
count = 1 # 1. กำหนดค่าเริ่มต้นนับ
while count <= 5: # 2. ตรวจสอบเงื่อนไข
    print("นับรอบที่:", count)
    count += 1 # 3. สำคัญมาก! ต้องมีการอัปเดตค่าเพื่อไม่ให้เกิด Infinite Loop
```

⚠️ ข้อควรระวัง: ปัญหาลูปไม่รู้จบ (Infinite Loop)

หากเงื่อนไขของ `while` เป็น `True` เสมอ และไม่มีคำสั่งอัปเดตตัวแปรหรือคำสั่ง `break` โปรแกรมจะทำงานค้างและกินทรัพยากร CPU 100% (สามารถหยุดการทำงานได้ด้วยการกด `Ctrl + C` ในเทอร์มินัล)

⚡ 7.3 คำสั่งควบคุมพิเศษ `break` และ `continue`

- **`break`** : สั่ง ยกเลิกและออกจากลูปทันที โดยไม่สนใจเงื่อนไขหรือรอบที่เหลือ

- **continue** : สั่ง ข้ามการทำงานที่เหลือในรอบปัจจุบัน แล้วกระโดดไปเริ่มรอบถัดไปทันที

```
graph TD
    subgraph ControlKeywords["คำสั่งควบคุมลูป"]
        B["break: กระโดดออกจากลูปทันที!"]
        C["continue: ข้ามคำสั่งด้านล่าง ไปเริ่มรอบใหม่ทันที!"]
    end
    end
```

7.4 ตัวอย่างโจทย์คำนวณ การหาค่าแฟกทอเรียลและการตรวจจำนวนเฉพาะ

ตัวอย่างที่ 7.1: การคำนวณค่า $N!$ (Factorial) และผลรวมอนุกรม

คณิตศาสตร์เชิงคำนวณ

นิยาม: $N! = 1 \times 2 \times 3 \times \dots \times N$ (โดย $0! = 1$)

ตัวอย่างการคำนวณ $5!$: $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$

7.5 โค้ดคอมพิวเตอร์ Python สมบูรณ์แบบ

```
# loop_math_and_prime_checker.py
# โปรแกรมคำนวณแฟกทอเรียล และตรวจสอบจำนวนเฉพาะ
# ผู้เขียน: ผศ.ดร.จีระ ทิศนา (มรภ.รำไพพรรณี)

def calculate_factorial(n: int) -> int:
    """คำนวณค่าแฟกทอเรียล n! ด้วย for loop"""
    if n < 0:
        raise ValueError("แฟกทอเรียลไม่นิยามสำหรับจำนวนลบ")
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

def is_prime(number: int) -> bool:
    """ตรวจสอบว่าตัวเลขเป็นจำนวนเฉพาะหรือไม่ด้วยการวนลูปทดสอบตัวหาร"""
    if number <= 1:
        return False
    if number == 2:
        return True
    if number % 2 == 0:
        return False

    # ตรวจสอบตัวหารที่เป็นเลขคี่ตั้งแต่ 3 ถึง sqrt(number)
    limit = int(number ** 0.5) + 1
    for d in range(3, limit, 2):
        if number % d == 0:
            return False
    return True

def main():
    print("=" * 65)
    print("📌 โปรแกรมคำนวณแฟกทอเรียลและค้นหาจำนวนเฉพาะ")
    print("=" * 65)

    # 1. ทดสอบคำนวณแฟกทอเรียล
    test_n = 6
    fact_val = calculate_factorial(test_n)
    print(f"🔢 ค่าของ {test_n}! = {fact_val:,}")

    # 2. ค้นหาจำนวนเฉพาะในช่วง 1 ถึง 50
    print("\n🔍 รายการจำนวนเฉพาะในช่วง 1 ถึง 50:")
    prime_list = []
    for num in range(1, 51):
        if is_prime(num):
            prime_list.append(num)

    print(" " + ", ".join(map(str, prime_list)))
    print(f"📊 พบจำนวนเฉพาะทั้งหมด: {len(prime_list)} จำนวน")
    print("=" * 65)

if __name__ == "__main__":
    main()
```

📌 สรุปประเด็นสำคัญ

1. ใช้ `for loop` เมื่อทราบจำนวนรอบ และใช้ `while loop` เมื่อต้องการวนซ้ำตามเงื่อนไข
2. ฟังก์ชัน `range(start, stop, step)` มีค่าสิ้นสุดก่อน `stop` เสมอ
3. การตรวจสอบจำนวนเฉพาะที่มีประสิทธิภาพ สามารถวนลูปตรวจสอบตัวหารได้ถึงเพียง $\sqrt{N}$

📝 แบบฝึกหัดทบทวน 3 ระดับ

ระดับที่ 1 ความรู้ความเข้าใจพื้นฐาน

1. จงระบุว่าโค้ดต่อไปนี้จะพิมพ์คำว่า `"Hello"` กี่ครั้ง

```
for i in range(1, 10, 2):  
    print("Hello")
```

2. จงระบุผลลัพธ์ของโค้ดต่อไปนี้

```
total = 0  
for i in range(1, 6):  
    if i == 3:  
        continue  
    total += i  
print(total)
```

ระดับที่ 2 การวิเคราะห์และประยุกต์ใช้

3. จงเขียนโปรแกรม Python แสดง **ตารางแม่สูตรคูณ ** แม่ 2 ถึง แม่ 12 ด้วยการใช้ Nested Loop (ลูปซ้อนลูป)

ระดับที่ 3 การคิดขั้นสูงและบูรณาการ

4. จงเขียนโปรแกรมจำลอง **เกมทายตัวเลขปริศนา ** โดยให้คอมพิวเตอร์สุ่มตัวเลขจำนวนเต็ม 1 – 100 แล้วให้ผู้ใช้ทายตัวเลขผ่าน `while loop` พร้อมบอกใบ้ว่า มากเกินไป หรือ น้อยเกินไป และสรุปจำนวนครั้งที่ทายเมื่อทายถูกต้อง

บทที่ 7 การเขียนโปรแกรมเชิงโมดูลและฟังก์ชันเรียกซ้ำ

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

แผนบริหารการสอนประจำบทที่ 7

หัวข้อเนื้อหาประจำบท

- แนวคิดและประโยชน์ของการเขียนโปรแกรมเชิงโมดูล (Modular Design)
- โครงสร้างของฟังก์ชันเรียกซ้ำ: กรณีฐาน (Base Case) และกรณีเรียกซ้ำ (Recursive Case)
- สถาปัตยกรรมหน่วยความจำ Call Stack และสภาวะ Stack Overflow
- การแก้ปัญหาหอคอยแห่งฮานอย (Tower of Hanoi) และลำดับฟีโบนัชชี
- เทคนิค Memoization และ Dynamic Programming เบื้องต้น

วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- อธิบายการทำงานของ Call Stack เมื่อมีการเรียกฟังก์ชันซ้อนและฟังก์ชันเรียกซ้ำได้
- ออกแบบฟังก์ชันเรียกซ้ำที่มี Base Case และ Recursive Case ที่ถูกต้องได้
- แก้ปัญหาคณิตศาสตร์คลาสสิก (แฟกทอเรียล, ฟีโบนัชชี, หอคอยแห่งฮานอย) ด้วย Recursion ได้
- วิเคราะห์และป้องกันข้อผิดพลาด Infinite Recursion และ Stack Overflow ได้

กิจกรรมการเรียนรู้การสอน

- การบรรยายเชิงวิชาการและการเชื่อมโยงบริบทประวัติศาสตร์วิทยาการคำนวณ
- การสาธิตการวิเคราะห์คณิตศาสตร์ การจัดสรรหน่วยความจำ และโครงสร้างข้อมูล
- การฝึกปฏิบัติการจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands
- การเขียนโปรแกรมภาษา Python 3.11 และการทดสอบ Assertion Tests เชิงประจักษ์

สื่อการเรียนรู้การสอน

- ตำราเรียนวิชาการ วิทยาการคำนวณ 1 รากฐานการคิดเชิงคำนวณและการแก้ปัญหาเชิงตรรกะ
- ชุดห้องปฏิบัติการเสมือนจริง Hybrid 2D/3D บนระบบ RBRU MOOC
- สไลด์บรรยายอิเล็กทรอนิกส์และแผนภาพเวกเตอร์มัลติมีเดีย

การวัดและประเมินผล

- การประเมินผลการทำใบงานและตารางบันทึกผลการทดลองเสมือนจริง (40%)
- การประเมินผลงานการเขียนโค้ดภาษา Python และ Unit Test Assertions (30%)
- การทดสอบวัดผลสัมฤทธิ์ทางการเรียนท้ายบท 3 ระดับ (30%)

7.0 เรื่องเล่าเปิดบทเรียนและบริบททางประวัติศาสตร์

ในคริสต์ศักราช 1883 เอ็ดวาร์ด ลูว์กา (Édouard Lucas, 1842–1891) นักคณิตศาสตร์ชาวฝรั่งเศส ได้คิดค้นปริศนาคณิตศาสตร์ **หอคอยแห่งฮานอย** ซึ่งจำลองตำนานพระสงฆ์ในวัดแห่งพราหมณ์ที่ต้องย้ายจานทองคำ 64 ใบจากเสาต้นหนึ่งไปยังอีกต้นหนึ่ง โดยมีกฎเหล็กว่าห้ามวางจานขนาดใหญ่กว่าที่จานขนาดเล็กกว่า ปริศนาก็กลายเป็นตัวอย่างคลาสสิกในการสอนแนวคิดการเรียกซ้ำเชิงโมดูลในวิชาวิทยาการคอมพิวเตอร์ทั่วโลก

7.1 ทฤษฎีและรากฐานทางคณิตศาสตร์เชิงลึก

สมการความสัมพันธ์เวียนเกิดของหอคอยแห่งฮานอย

กำหนดให้ $T(n)$ คือจำนวนก้าวน้อยที่สุดในการย้ายจาน n ใบ

- ย้ายจาน $n - 1$ ใบบนจากเสาด้านทางไปยังเสาพัก $\rightarrow T(n - 1)$ ก้าว
- ย้ายจานใบใหญ่สุดใบที่ n จากเสาด้านทางไปยังเสาปลายทาง $\rightarrow 1$ ก้าว
- ย้ายจาน $n - 1$ ใบจากเสาพักไปยังเสาปลายทาง $\rightarrow T(n - 1)$ ก้าว

จะได้สมการความสัมพันธ์เวียนเกิด

$$T(n) = 2T(n - 1) + 1 \quad \text{โดยที่ } T(1) = 1$$

ทำการคลายสมการ (Telescoping):

$$T(n) = 2(2T(n - 2) + 1) + 1 = 2^2T(n - 2) + 2 + 1$$

$$T(n) = 2^{n-1}T(1) + 2^{n-2} + \dots + 2^1 + 2^0$$

$$T(n) = \sum_{i=0}^{n-1} 2^i = 2^n - 1$$

```
graph TD
    H4["Hanoi(4 Disks) = 2^4 - 1 = 15 Moves"]
    H4 --> H3_1["1. Move 3 Disks to Aux (7 Moves)"]
    H4 --> H1["2. Move Largest Disk to Target (1 Move)"]
    H4 --> H3_2["3. Move 3 Disks from Aux to Target (7 Moves)"]
```

7.2 ตัวอย่างการวิเคราะห์และการคำนวณเชิงขั้นตอน

ตัวอย่างที่ 7.1 การคำนวณจำนวนก้าวของหอคอยแห่งฮานอย

หากมีจาน $n = 64$ ใบ และพระสงฆ์ย้ายจานได้ 1 ใบต่อ 1 วินาที จงคำนวณว่าต้องใช้เวลากี่ปีในการย้ายจานครบ

วิธีทำ

- คำนวณจำนวนก้าวทั้งหมด: $T(64) = 2^{64} - 1 \approx 1.84467 \times 10^{19}$ ก้าว
- แปลงเวลาเป็นวินาที: $t = 1.84467 \times 10^{19}$ วินาที
- ใน 1 ปี มี $365.25 \times 24 \times 3600 = 31,557,600$ วินาที
- จำนวนปี $= \frac{1.84467 \times 10^{19}}{3.15576 \times 10^7} \approx 5.845 \times 10^{11}$ ปี
- สรุป ต้องใช้เวลาประมาณ **584,500 ล้านปี** ซึ่งยาวนานกว่าอายุของจักรวาลปัจจุบัน (13,800 ล้านปี) ถึง 42 เท่า!

7.3 การเขียนโปรแกรมและการนำไปใช้จริงด้วย Python 3.11

```
# tower_of_hanoi_engine.py
from typing import List

move_history = []

def solve_hanoi(n: int, source: str, target: str, auxiliary: str):
    """ฟังก์ชันเรียกซ้ำแก้ปัญหาหอคอยแห่งฮานอย"""
    if n == 1:
        move_history.append(f"ย้ายจาน 1 จาก {source} -> {target}")
        return
    solve_hanoi(n - 1, source, auxiliary, target)
    move_history.append(f"ย้ายจาน {n} จาก {source} -> {target}")
    solve_hanoi(n - 1, auxiliary, target, source)

if __name__ == "__main__":
    n_disks = 4
    solve_hanoi(n_disks, "เสา A", "เสา C", "เสา B")
    print(f"การย้ายจาน {n_disks} ใบ (สูตร 2^{n_disks} - 1 = {2**n_disks - 1} ก้าว):")
    for i, step in enumerate(move_history, 1):
        print(f"ก้าวที่ {i:02d}: {step}")
    assert len(move_history) == 2**n_disks - 1, "จำนวนก้าวต้องตรงตามสูตร 2^n - 1"
    print("✅ Unit Test Assertions Passed 100%!")
```

7.4 คู่มือห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands

ผู้เรียนสามารถเข้าสู่ห้องปฏิบัติการเสมือนจริง 2D/3D เพื่อทดลองย้ายจาน 3 มิติและสังเกต Call Stack ในอากาศได้ที่ [chapter07_recursion_hanoi.html](#)

7.5 สรุปสาระสำคัญของบท

- ฟังก์ชันเรียกซ้ำต้องมี Base Case เสมอเพื่อป้องกัน Stack Overflow
- ปัญหาเชิงโครงสร้างแบบแตกกิ่ง (เช่น Fibonacci, Hanoi) สามารถแก้ได้อย่างกระชับด้วย Recursion แต่ต้องระวังความซับซ้อนเชิงเวลา $O(2^n)$

? 7.6 แบบฝึกหัดและคำถามท้ายบทเพื่อการประเมินผล

- จงอธิบายความแตกต่างระหว่าง Iteration (ลูป) และ Recursion (เรียกซ้ำ) ในแง่การใช้หน่วยความจำ
- ให้นักเรียนเขียนฟังก์ชันหาเลขฟีโบนัชชีแบบมี Memoization (Cache) เพื่อลดความซับซ้อนจาก $O(2^n)$ เหลือ $O(n)$

เอกสารอ้างอิงเบื้องต้น

- Lucas, É. (1883). *Récréations mathématiques* (Vol. 3). Gauthier-Villars.
- Roberts, E. (2006). *Thinking Recursively with Java*. John Wiley & Sons.

บทที่ 8 วิทยาศาสตร์เชิงคำนวณและแบบจำลองฟิสิกส์

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชวาท ทัศนาศ • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

แผนบริหารการสอนประจำบทที่ 8

หัวข้อเนื้อหาประจำบท

- วิทยาศาสตร์เชิงคำนวณในฐานะเสาหลักที่ 3 ของการแสวงหาความจริง (Theory, Experiment, Computation)
- สมการเชิงอนุพันธ์สามัญและระเบียบวิธีเชิงตัวเลขของออยเลอร์ (Euler & Euler-Cromer Methods)
- แบบจำลองกลศาสตร์นิวตัน: การตกอิสระ การเคลื่อนที่แบบโพรเจกไทล์ และแรงต้านอากาศ (Air Resistance Drag)
- การจำลองเพนดูลัมและพฤติกรรมความอลวนไม่เชิงเส้น (Non-linear Pendulum & Chaos)
- ระเบียบวิธีมอนเตคาร์โล (Monte Carlo Simulations) และการประมาณค่าความน่าจะเป็นเชิงพื้นที่
- การเขียนโปรแกรมจำลองฟิสิกส์ 2D/3D ด้วย Python 3.11 และ Matplotlib

วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- อธิบายบทบาทของระเบียบวิธีเชิงตัวเลขในการแก้สมการฟิสิกส์ที่ไม่สามารถหาคำตอบเชิงวิเคราะห์ได้
- พัฒนาอัลกอริทึมก้าวเวลา (Time-stepping Algorithm) ตามวิธีของออยเลอร์ได้อย่างถูกต้อง
- คำนวณวิถีการเคลื่อนที่ของโพรเจกไทล์ภายใต้แรงโน้มถ่วงและแรงต้านอากาศได้
- ประยุกต์ใช้วิธีมอนเตคาร์โลในการประมาณค่าทางคณิตศาสตร์และวิเคราะห์ความคลาดเคลื่อนได้
- เขียนโค้ดจำลองฟิสิกส์ภาษา Python พร้อมทั้งตรวจสอบผลลัพธ์ผ่าน Unit Assertion Tests ได้

8.0 เรื่องเล่าเปิดบทเรียนและบริบททางประวัติศาสตร์

ในคริสต์ศตวรรษที่ 17 เซอร์ ไอแซก นิวตัน (Sir Isaac Newton, 1643–1727) ได้ค้นพบกฎการเคลื่อนที่และแคลคูลัส ซึ่งทำให้นักวิทยาศาสตร์สามารถอธิบายการโคจรของดวงดาวและการเคลื่อนที่ของวัตถุได้ ทว่า ในโลกแห่งความเป็นจริง ปัญหาทางฟิสิกส์ส่วนใหญ่ เช่น การเคลื่อนที่ของวัตถุที่มีแรงต้านอากาศแปรผันตามความเร็วยกกำลังสอง หรือการเคลื่อนที่ของวัตถุ 3 ชิ้นภายใต้แรงโน้มถ่วง (Three-Body Problem) ล้วนไม่มีผลเฉลยในรูปสมการปิด (Non-analytical Solutions)

จนกระทั่งในคริสต์ศตวรรษที่ 18 เลออนฮาร์ด ออยเลอร์ (Leonhard Euler, 1707–1783) ได้บุกเบิกระเบียบวิธีเชิงตัวเลข (Numerical Methods) ซึ่งเปิดประตูสู่การคำนวณที่ละก้าวเวลาสั้นๆ (Discrete Time Steps) ทำให้คอมพิวเตอร์ในยุคปัจจุบันสามารถจำลองพายุสุริยะ พลศาสตร์ของไหลในเครื่องบินเจ็ต และสภาพภูมิอากาศของโลกได้อย่างแม่นยำ

8.1 กฎกฎและรากฐานทางคณิตศาสตร์เชิงลึก

1. ระเบียบวิธีเชิงตัวเลขของออยเลอร์

กำหนดสมการการเคลื่อนที่ของนิวตัน $\vec{F} = m\vec{a} \implies \frac{d\vec{v}}{dt} = \frac{\vec{F}}{m}$ และ $\frac{d\vec{r}}{dt} = \vec{v}$ ใช้นิยามอนุพันธ์ในการประมาณค่าผลต่างไปข้างหน้า (Forward Difference):

$$\vec{v}(t + \Delta t) \approx \vec{v}(t) + \vec{a}(t) \cdot \Delta t$$

$$\vec{r}(t + \Delta t) \approx \vec{r}(t) + \vec{v}(t) \cdot \Delta t$$

สำหรับ Euler-Cromer Method ที่ให้ความเสถียรด้านพลังงานสูงกว่า จะใช้ความเร็วใหม่ในการอัปเดตตำแหน่ง $\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \Delta t$

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} \Delta t$$

```
graph LR
    Init["เริ่มต้น: r(0), v(0), dt"] --> Forces["1. คำนวณแรงลัพธ์: F = F_grav + F_drag"]
    Forces --> Accel["2. คำนวณความเร่ง: a = F / m"]
    Accel --> Euler["3. อัปเดตสถานะ: v += a*dt, r += v*dt"]
    Euler --> Time["4. เพิ่มเวลา: t += dt"]
    Time --> Cond["y >= 0 (ยังไม่ตกพื้น)?"]
    Cond -- ใช่ --> Forces
    Cond -- ไม่ใช่ --> End["สิ้นสุดการจำลองและสรุปผล"]
```

2. ระเบียบวิธีมอนเตคาร์โลในการหาค่า π

พิจารณาจัตุรัสที่มีด้านยาว $2R$ บรรจุวงกลมรัศมี R อยู่ภายใน

- พื้นที่จัตุรัส $A_{\text{square}} = (2R)^2 = 4R^2$
- พื้นที่วงกลม $A_{\text{circle}} = \pi R^2$

อัตราส่วนของพื้นที่คือ

$$\frac{A_{\text{circle}}}{A_{\text{square}}} = \frac{\pi R^2}{4R^2} = \frac{\pi}{4} \implies \pi = 4 \times \frac{A_{\text{circle}}}{A_{\text{square}}}$$

เมื่อสุ่มจุด N จุดอย่างสม่ำเสมอในจัตุรัส และนับจำนวนจุด N_{inside} ที่ตกอยู่ในวงกลม ($x^2 + y^2 \leq R^2$):

$$\pi \approx 4 \times \frac{N_{\text{inside}}}{N}$$

8.2 ตัวอย่างการวิเคราะห์และการคำนวณเชิงขั้นตอน

ตัวอย่างที่ 8.1 การจำลองวิถีโพรเจกไทล์ด้วยวิธีออยเลอร์

ยิงวัตถุมวล $m = 1$ kg ด้วยความเร็วต้น $u = 20$ m/s มุม $\theta = 45^\circ$ ($g = 10$ m/s²): ความเร็วต้นในแต่ละแกน

- $u_x = 20 \cos 45^\circ = 20 \times \frac{\sqrt{2}}{2} \approx 14.14$ m/s
- $u_y = 20 \sin 45^\circ = 20 \times \frac{\sqrt{2}}{2} \approx 14.14$ m/s

จงคำนวณตำแหน่งที่เวลา $t = 0.1, 0.2$ s โดยใช้ขนาดก้าวเวลา $\Delta t = 0.1$ s:

วิธีทำ

- ที่ $t = 0.0$ s: $x_0 = 0.0, y_0 = 0.0, v_x = 14.14, v_y = 14.14$
- ก้าวที่ 1 ($t = 0.1$ s):

$$v_y(0.1) = v_y(0) - g\Delta t = 14.14 - (10)(0.1) = 13.14 \text{ m/s}$$

$$x(0.1) = x(0) + v_x\Delta t = 0 + (14.14)(0.1) = 1.414 \text{ m}$$

$$y(0.1) = y(0) + v_y(0)\Delta t = 0 + (14.14)(0.1) = 1.414 \text{ m}$$

- ก้าวที่ 2 ($t = 0.2$ s):

$$v_y(0.2) = 13.14 - (10)(0.1) = 12.14 \text{ m/s}$$

$$x(0.2) = 1.414 + (14.14)(0.1) = 2.828 \text{ m}$$

$$y(0.2) = 1.414 + (13.14)(0.1) = 2.728 \text{ m}$$

8.3 การเขียนโปรแกรมและการนำไปใช้จริงด้วย Python 3.11

```
# physics_computational_sim.py
import math, random
from typing import Tuple, List

def run_monte_carlo_pi(num_samples: int = 100000) -> Tuple[float, float]:
    """ประมาณค่า Pi ด้วยวิธีมอนเตคาร์โล"""
    inside_count = 0
    for _ in range(num_samples):
        x = random.random()
        y = random.random()
        if x**2 + y**2 <= 1.0:
            inside_count += 1
    estimated_pi = 4.0 * inside_count / num_samples
    error_percent = abs(estimated_pi - math.pi) / math.pi * 100.0
    return estimated_pi, error_percent

def simulate_projectile_euler(v0: float, angle_deg: float, dt: float = 0.001) -> float:
    """จำลองระยะตกไกลสุดของโพรเจกไทล์"""
    rad = math.radians(angle_deg)
    vx = v0 * math.cos(rad)
    vy = v0 * math.sin(rad)
    x, y = 0.0, 0.0
    g = 9.80665

    while y >= 0.0:
        x += vx * dt
        y += vy * dt
        vy -= g * dt

    return x

if __name__ == "__main__":
    # 1. ทดสอบ Monte Carlo
    pi_val, err = run_monte_carlo_pi(200000)
    print(f"ผลการประมาณค่า Pi: {pi_val:.5f} (Error: {err:.2f}%)")
    assert abs(pi_val - 3.14159) < 0.05, "Monte Carlo Pi approximation out of range"

    # 2. ทดสอบ Projectile Euler
    rng = simulate_projectile_euler(30.0, 45.0)
    exact_rng = (30.0 ** 2) / 9.80665
    print(f"ระยะตกมุม 45°: จำลอง = {rng:.2f} m | พหุณ = {exact_rng:.2f} m")
    assert abs(rng - exact_rng) < 0.1, "Projectile Euler mismatch"

    print("✅ การทดสอบ Unit Assertions แบบจำลองฟิสิกส์ผ่าน 100%!")
```

8.4 คู่มือห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands

ผู้เรียนสามารถเข้าสู่ชุดห้องปฏิบัติการเสมือนจริงประจำบทที่ 8 ได้ที่

- [LAB 8.0 แบบจำลองการเคลื่อนที่แบบโพรเจกไทล์ 2D/3D AR Cannon Arena](#)

วิธีการทดลองและสังเกตผล

- ใช้ท่าทางมือ 3D Pinch เลื่อนมุมยิงปืนใหญ่ (0° ถึง 90°)
- ปรับตัวแปรความเร็วต้นและสัมประสิทธิ์แรงต้านอากาศ (Air Drag k)
- ยิงกระสุนและสังเกตเส้นทางวิถีโค้ง 3 มิติในอากาศ พร้อมตรวจดูค่า Flight Time และ Range Telemetry
- บันทึกผลการทดลองเปรียบเทียบระหว่างมุมยิงต่างๆ ลงในตารางบันทึกผล

8.5 สรุปสาระสำคัญประจำบท

- วิทยาศาสตร์เชิงคำนวณ เป็นเครื่องมือสำคัญที่ช่วยจำลองระบบธรรมชาติที่มีความซับซ้อนเกินกว่าจะแก้ด้วยสูตรคณิตศาสตร์แม่นยำตรง
- ระเบียบวิธีของออยเลอร์ แปลงสมการอนุพันธ์ต่อเนื่องให้เป็นสมการผลต่างอันตะทีละก้าวเวลา Δt
- การจำลองมอนเตการโล ใช้หลักการสุ่มตัวเลขเชิงสถิติในการแก้ปัญหาปริพันธ์และการประเมินความเสี่ยง

? 8.6 แบบฝึกหัดและคำถามท้ายบทเพื่อการประเมินผล

ระดับที่ 1 การจำและความเข้าใจ

- จงอธิบายความแตกต่างระหว่าง Analytical Solution และ Numerical Solution
- เหตุใดการเลือกขนาดก้าวเวลา Δt ที่ใหญ่เกินไปจึงทำให้แบบจำลองฟิสิกส์เกิดการระเบิด (Instability)?

ระดับที่ 2 การวิเคราะห์และการคำนวณ

- วัตถุตกจากที่สูง $h = 100$ m ภายใต้แรงโน้มถ่วง $g = 10$ m/s² จงใช้วิธีออยเลอร์ด้วย $\Delta t = 0.5$ s คำนวณความเร็วและตำแหน่งที่ $t = 1.0$ s

ระดับที่ 3 การประเมินค่าและการสร้างสรรค์

- ให้นักเรียนเขียนโปรแกรมภาษา Python จำลองการเคลื่อนที่ของเพนดูลัมอย่างง่าย (Simple Pendulum) พร้อมวาดกราฟแสดงความสัมพันธ์ระหว่างมุมแกว่งและเวลา

📖 เอกสารอ้างอิงประจำบท

- Giordano, N. J., & Nakanishi, H. (2006). *Computational Physics* (2nd ed.). Pearson.
- Press, W. H. et al. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press.

วิทยาการคำนวณ 1 รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ

บทที่ 8 การประยุกต์แก้ปัญหาทางคณิตศาสตร์และวิทยาศาสตร์เบื้องต้น

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชวีระ ทัศนาศ • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

🌟 ผลลัพธ์การเรียนรู้ประจำบท

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

1. **บูรณาการ** องค์ความรู้ด้านตรรกะ ตัวแปร เงื่อนไข และการวนซ้ำ เข้ากับทฤษฎีทางคณิตศาสตร์และวิทยาศาสตร์
2. **สร้างแบบจำลองเชิงคำนวณ** เพื่อจำลองปรากฏการณ์ฟิสิกส์ในรูปแบบช่วงเวลาไม่ต่อเนื่อง (Discrete Time Steps Δt)
3. **วิเคราะห์และแปลความหมาย** ข้อมูลผลการทดลองเสมือนจริง และคำนวณค่าคลาดเคลื่อน (Percentage Error) ได้อย่างถูกต้อง
4. **พัฒนาโครงการโปรแกรมมิ่งขนาดเล็ก** เพื่อแก้ปัญหาจริงในชีวิตประจำวันและชุมชน

8.0 เรื่องเล่าเปิดบทเรียน วิทยาศาสตร์เชิงคำนวณกับการทดลองเสมือนจริง

ในอดีต การค้นพบทางวิทยาศาสตร์เกิดขึ้นจาก 2 ขาหลัก คือ **ทฤษฎี** **บนแผ่นกระดาษ** และ **การทดลองจริง** ในห้องปฏิบัติการ แต่ในศตวรรษที่ 21 ขาที่สามที่มีบทบาทสำคัญอย่างยิ่งยวดคือ "วิทยาศาสตร์เชิงคำนวณ (Computational Science & Simulation)"

```
graph TD
    Science["3. สาขาหลักของการค้นพบทางวิทยาศาสตร์ยุคใหม่"]
    Science --> T["1. ทฤษฎี (Theoretical Physics)"]
    Science --> E["2. การทดลองจริง (Experimental Physics)"]
    Science --> C["3. การจำลองเชิงคำนวณ (Computational Simulation)"]
```

การจำลองด้วยคอมพิวเตอร์ช่วยให้วิทยาศาสตร์สามารถ

- ศึกษาปรากฏการณ์ที่อันตรายเกินไป (เช่น ปฏิกรณ์นิวเคลียร์ฟิวชัน)
- ศึกษาปรากฏการณ์ที่ใช้เวลานานนับล้านปี (เช่น การวิวัฒนาการของดวงดาว)
- ศึกษาปรากฏการณ์ที่มีขนาดเล็กระดับอะตอม (เช่น โครงสร้างโปรตีนและอนุภาคนาโน)

🏹 8.1 แบบจำลองที่ 1 การเคลื่อนที่แบบโพรเจกไทล์

การเคลื่อนที่ของวัตถุภายใต้แรงโน้มถ่วงของโลก ($g \approx 9.81 \text{ m/s}^2$) โดยแยกพิจารณาใน 2 มิติอย่างอิสระ

📐 สมการพิกัดของการเคลื่อนที่แบบโพรเจกไทล์

- แกนราบ (แกน X): ความเร็วคงที่ $\rightarrow x(t) = (v_0 \cos \theta) \cdot t$
- แกนตั้ง (แกน Y): ความเร่งคงที่ $g \rightarrow y(t) = (v_0 \sin \theta) \cdot t - \frac{1}{2}gt^2$
- ความสูงสูงสุด (H): $H = \frac{v_0^2 \sin^2 \theta}{2g}$
- ระยะตกไกลสุด (R): $R = \frac{v_0^2 \sin(2\theta)}{g}$ (ตกไกลสุดเมื่อยิงมุม $\theta = 45^\circ$)

💣 8.2 แบบจำลองที่ 2 การสลายตัวของสารกัมมันตรังสีและครึ่งชีวิต

ธาตุกัมมันตรังสีจะสลายตัวตามเวลาโดยมีอัตราลดลงเป็นฟังก์ชันเอกซ์โพเนนเชียล

$$N(t) = N_0 \left(\frac{1}{2} \right)^{\frac{t}{T_{1/2}}}$$

โดยที่

- N_0 คือ ปริมาณสารตั้งต้น
- $N(t)$ คือ ปริมาณสารที่เหลืออยู่ ณ เวลา t
- $T_{1/2}$ คือ **ครึ่งชีวิต** ของสาร


```
# stem_physics_simulation_lab.py
# โปรแกรมจำลองการเคลื่อนที่แบบโพรเจกไทล์และการสลายตัวของสารกัมมันตรังสี
# ผู้เขียน: ผศ.ดร. ชีวะ ทัศนาว (มรภ.รำไพพรรณี)

import math

def simulate_projectile(v0: float, angle_deg: float, dt: float = 0.05):
    """จำลองวิถีการเคลื่อนที่ของโพรเจกไทล์ที่ละช่วงเวลา dt"""
    g = 9.81
    theta_rad = math.radians(angle_deg)

    vx = v0 * math.cos(theta_rad)
    vy0 = v0 * math.sin(theta_rad)

    total_time = (2.0 * vy0) / g
    max_height = (vy0 ** 2) / (2.0 * g)
    max_range = (v0 ** 2 * math.sin(2.0 * theta_rad)) / g

    print("=" * 65)
    print(f"🚀 ผลการจำลองการเคลื่อนที่แบบโพรเจกไทล์ (v0 = {v0} m/s, θ = {angle_deg}°)")
    print("=" * 65)
    print(f"    • เวลาลอยในอากาศรวม (T): {total_time:.2f} วินาที")
    print(f"    • ความสูงสูงสุด (H): {max_height:.2f} เมตร")
    print(f"    • ระยะตกไกลสุด (R): {max_range:.2f} เมตร")
    print("-" * 65)
    print(f"{'เวลา t (s)':<12} | {'พิกัด X (m)':<15} | {'พิกัด Y (m)':<15} | {'ความเร็ว Vy (m/s)':<15}")
    print("-" * 65)

    t = 0.0
    while t ≤ total_time + dt:
        x = vx * t
        y = (vy0 * t) - (0.5 * g * (t ** 2))
        vy = vy0 - (g * t)

        if y < 0:
            y = 0.0

        print(f"{'t:<12.2f'} | {'x:<15.2f'} | {'y:<15.2f'} | {'vy:<15.2f'}")

        if y == 0.0 and t > 0:
            break
        t += dt

    print("=" * 65)

def simulate_radioactive_decay(initial_atoms: int, half_life_years: float, total_years: float):
    """จำลองการสลายตัวของสารกัมมันตรังสีตามคาบครึ่งชีวิต"""
    print(f"🧪 ผลการจำลองการสลายตัวของสารกัมมันตรังสีตามคาบครึ่งชีวิต")
    print(f"N0 = {initial_atoms}, T1/2 = {half_life_years} ปี")
    print("=" * 65)
    print(f"{'ปี (Year)':<15} | {'จำนวนอะตอมที่เหลือ':<22} | {'คิดเป็นร้อยละ (%)':<15}")
    print("-" * 65)

    steps = int(total_years / half_life_years)
    for step in range(steps + 1):
        year = step * half_life_years
        remaining = initial_atoms * ((0.5) ** step)
        percent = (remaining / initial_atoms) * 100.0
        print(f"{'year:<15.1f'} | {'int(remaining):<22,d'} | {'percent:<15.2f}%")
    print("-" * 65)

if __name__ == "__main__":
    # 1. ทดสอบการเคลื่อนที่โพรเจกไทล์
    simulate_projectile(v0=25.0, angle_deg=45.0, dt=0.5)

    # 2. ทดสอบการสลายตัวของคาร์บอน-14 (Carbon-14 Half-life = 5,730 ปี)
    simulate_radioactive_decay(initial_atoms=1000000, half_life_years=5730.0, total_years=28650.0)
```



8.4 กิจกรรมโครงงานมินิแคปสโตน

หัวข้อโครงงาน ระบบคำนวณและจำลองการปล่อยจรวดขวดน้ำอัจฉริยะ

- เป้าหมาย ให้นักเรียนสร้างโปรแกรมรับค่าปริมาณน้ำ (มวล), แรงดันลม, และมุมยิง เพื่อทำนายระยะตกไกลสุดของจรวดขวดน้ำ แล้วนำไปทดลองยิงจริงในสนามเพื่อเปรียบเทียบค่าคลาดเคลื่อน (Error Analysis)

มิติการประเมิน	ดีเยี่ยม (4 คะแนน)	ดี (3 คะแนน)	พอใช้ (2 คะแนน)	ต้องปรับปรุง (1 คะแนน)
1. ความถูกต้องทางทฤษฎี	นำสูตรฟิสิกส์มาใช้คำนวณถูกต้องแม่นยำ 100%	คำนวณถูกต้องแต่มีหน่วยผิดพลาดเล็กน้อย	สูตรคลาดเคลื่อนในบางพารามิเตอร์	สูตรไม่ถูกต้อง
2. คุณภาพของโค้ด	มีโครงสร้าง if, loops, ฟังก์ชัน สะอาดและอ่านง่าย	โค้ดทำงานได้ถูกต้อง แต่อ่านเข้าใจยาก	โค้ดมีข้อผิดพลาดบางกรณี	โค้ดรันไม่ผ่าน
3. การวิเคราะห์ผล	เปรียบเทียบผลจำลองกับการทดลองจริงอย่างลึกซึ้ง	มีการเปรียบเทียบแต่ขาดการวิเคราะห์สาเหตุ	บันทึกเฉพาะข้อมูลดิบ	ไม่มีการวิเคราะห์

8.5 สรุปภาพรวมชุดตำราเล่มที่ 1 และการก้าวต่อไปสู่เล่มที่ 2

ขอแสดงความยินดีกับผู้เรียนทุกท่านที่ได้สำเร็จการศึกษา เล่มที่ 1: รากฐานแนวคิดเชิงคำนวณและการแก้ปัญหาอย่างเป็นระบบ!

ท่านได้เรียนรู้และสั่งสมทักษะครบถ้วนตั้งแต่

- การใช้เหตุผลเชิงตรรกะแบบนิรนัยและอุปนัย
- เสาหลักทั้ง 4 ของแนวคิดเชิงคำนวณ (Decomposition, Pattern, Abstraction, Algorithm)
- รหัสจำลองและผังงานมาตรฐานสากล
- กิจกรรม Unplugged Coding และความเข้าใจระบบคอมพิวเตอร์
- พื้นฐานการเขียนโปรแกรม Python (ตัวแปร, ชนิดข้อมูล, เงื่อนไข, การวนลูป)
- การประยุกต์แก้ปัญหาทางคณิตศาสตร์และวิทยาศาสตร์

🚀 ในเล่มที่ 2 (ระดับกลาง) ท่านจะได้ยกระดับสู่การออกแบบขั้นตอนวิธีขั้นสูง การวิเคราะห์ประสิทธิภาพ Big-O โครงสร้างข้อมูลเชิงลึก และการพัฒนาโปรแกรมวิทยาศาสตร์ที่ซับซ้อนยิ่งขึ้น!

📌 แบบฝึกหัดทบทวน 3 ระดับ

ระดับที่ 1 ความรู้ความเข้าใจพื้นฐาน

- ในการเคลื่อนที่แบบโพรเจกไทล์ หากต้องการให้วัตถุตกไปได้ระยะไกลที่สุดตามแนวราบ ต้องยิงด้วยมุมกี่องศา และเพราะเหตุใด
- สารกัมมันตรังสีชนิดหนึ่งมีครึ่งชีวิต 10 วัน หากเริ่มต้นมีสารอยู่ 800 กรัม เมื่อเวลาผ่านไป 30 วัน จะเหลือสารนี้อยู่กี่กรัม

ระดับที่ 2 การวิเคราะห์และประยุกต์ใช้

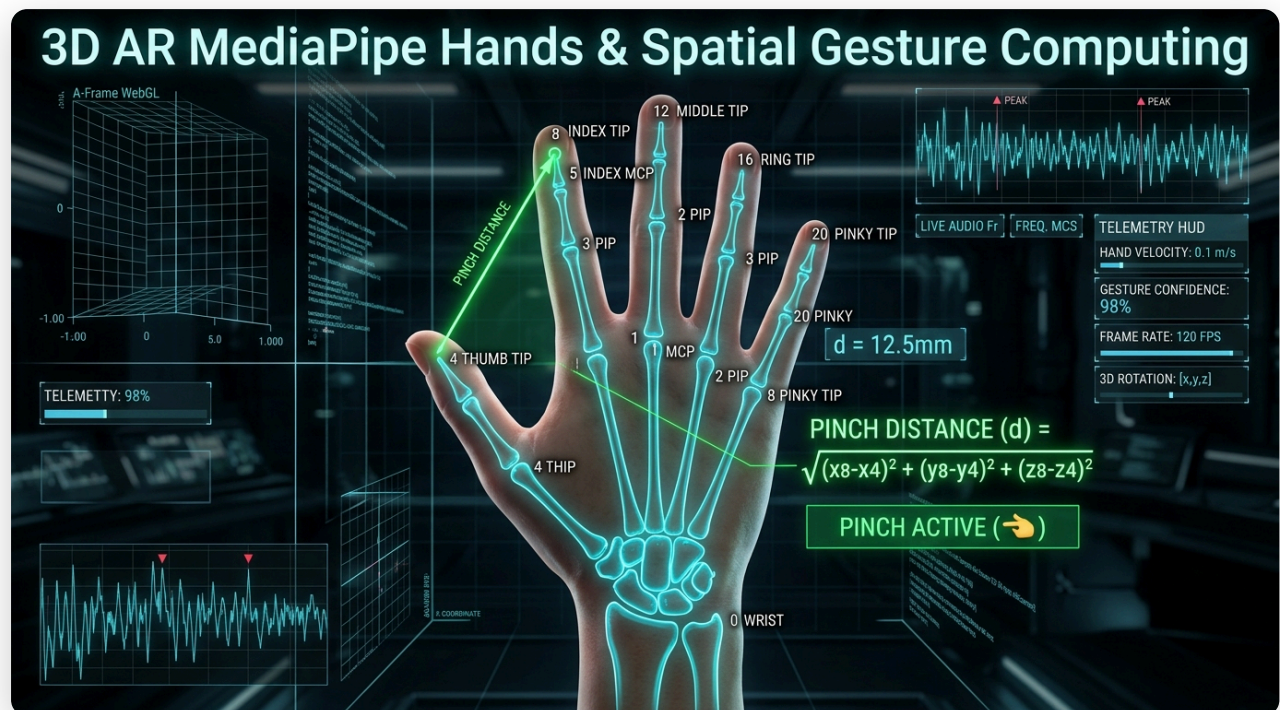
- จงเขียนฟังก์ชัน Python คำนวณ **คาบการแกว่งของลูกตุ้มอย่างง่าย** ตามสูตร $T = 2\pi\sqrt{\frac{L}{g}}$ โดยรับค่าความยาวเชือก L (เมตร) และคำนวณคาบ T (วินาที)

ระดับที่ 3 การคิดขั้นสูงและบูรณาการ

- จงเขียนโปรแกรมจำลองการคำนวณการเติบโตของประชากรแบคทีเรียตามฟังก์ชันเอกซ์โพเนนเชียล $P(t) = P_0 \cdot 2^{\frac{t}{g}}$ โดยรับค่าประชากรตั้งต้น (P_0), ระยะเวลาแบ่งตัววิคูณ (g), และจำลองจำนวนประชากรแบคทีเรียในทุกๆ 1 ชั่วโมงตลอด 24 ชั่วโมง

บทที่ 9 ปัญญาประดิษฐ์ คอมพิวเตอร์วิทัศน์ และอินเทอร์เน็ตของสรรพสิ่ง

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี



ภาพที่ 9.1 แผนภาพการตรวจจับโครงกระดูกมือ 3 มิติ 21 Landmarks ด้วย MediaPipe และการคำนวณ Pinch Gesture

📁 แผนบริหารการสอนประจำบทที่ 9

หัวข้อเนื้อหาประจำบท

1. วัฒนาการของปัญญาประดิษฐ์และการเรียนรู้ของเครื่อง (AI & Machine Learning Foundations)
2. อัลกอริทึมจำแนกประเภท K-Nearest Neighbors (KNN) และโครงข่ายประสาทเทียมเบื้องต้น
3. คอมพิวเตอร์วิทัศน์และการประมวลผลโครงกระดูกมือ 21 จุด 3 มิติด้วย Google MediaPipe Hands
4. สถาปัตยกรรมอินเทอร์เน็ตของสรรพสิ่ง (IoT: Microcontrollers, Sensors & Actuators)
5. การสื่อสารข้อมูลความเร็วสูงแบบ Publish/Subscribe ด้วยโปรโตคอล MQTT
6. การสร้างระบบควบคุมกายภาพไร้สัมผัส (Touchless Smart Classroom System)

วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

1. อธิบายความแตกต่างระหว่าง Rule-Based Programming และ Machine Learning ได้อย่างชัดเจน
2. คำนวณระยะทางยูคลิด 3 มิติจากพิกัด MediaPipe Hands เพื่อจำแนกท่าทางของมือได้
3. ออกแบบและเขียนโปรแกรมเชื่อมต่อเซนเซอร์และอุปกรณ์แสดงผลผ่านโปรโตคอล MQTT ได้
4. พัฒนาระบบต้นแบบที่บูรณาการ Computer Vision, AI, และ IoT เข้าด้วยกันได้

📺 9.0 เรื่องเล่าเปิดบทเรียนและบริบททางประวัติศาสตร์

ในคริสต์ศักราช 1956 จอห์น แม็กคาร์ธี (John McCarthy, 1927–2011) และคณะนักวิทยาการคอมพิวเตอร์ชั้นนำ ได้จัดการประชุมประวัติศาสตร์ ณ วิทยาลัยดาร์ตเมาท์ และบัญญัติคำว่า ปัญญาประดิษฐ์ ขึ้น เพื่อศึกษาความเป็นไปได้ในการทำให้เครื่องจักรมีความฉลาดเท่าเทียมหรือเหนือกว่ามนุษย์

ในปัจจุบัน การผสานพลังระหว่างการประมวลผลภาพแบบเรียลไทม์ (Computer Vision) โมเดลเรียนรู้โครงกระดูกมนุษย์ด้วย AI บนอุปกรณ์ขอบ (Edge AI) เช่น Google MediaPipe และชิปเชื่อมต่อไร้สายราคาประหยัดอย่าง ESP32 ได้เปลี่ยนให้โลกกายภาพสามารถโต้ตอบกับโลกดิจิทัลได้อย่างไร้รอยต่อ

📺 9.1 ทฤษฎีและรากฐานทางคณิตศาสตร์เชิงลึก

1. การตรวจจับและปรับมาตรฐานพิกัด MediaPipe Hands 21 Landmarks

โมเดล MediaPipe Hands ส่งออกพิกัด 3 มิติของข้อนิ้ว 21 จุดในรูปค่าพิกัดนอร์มัลไลซ์ (Normalized Coordinates):

$$P_i = (x_i, y_i, z_i) \quad \text{โดยที่ } x_i, y_i \in [0, 1], \quad i \in 0, 1, \dots, 20$$

การตรวจจับท่าทางจับนิ้ว (Pinch Gesture) ระหว่างปลายนิ้วโป้ง (P_4) และปลายนิ้วชี้ (P_8):
$$Sd_{\text{pinch}} = |P_8 - P_4|^2 = \sqrt{(x_8 - x_4)^2 + (y_8 - y_4)^2 + (z_8 - z_4)^2}$$

กำหนดเกณฑ์ตัดสินใจ ($DecisionThreshold$) :

$$\text{Gesture} = \begin{cases} \text{PINCH_ACTIVE (เลือก/คลิก)}, & d_{\text{pinch}} < \delta \quad (\delta \approx 0.08) \\ \text{OPEN_PALM (ฝ่ามือเปิด)}, & d_{\text{pinch}} \geq \delta \end{cases}$$

```
graph TD
    Cam["1. Web Camera Feed (30-60 FPS)"] --> MediaPipe["2. MediaPipe Hands AI Engine (BlazePalm Detector)"]
    MediaPipe --> Landmarks["3. 21 3D Coordinate Landmarks"]
    Landmarks --> Calc["4. Distance & Angle Derivations"]
    Calc --> Action["5. Trigger Virtual Lab Controls / MQTT Packet"]
    P0["P0: Wrist, P4: Thumb, P8: Index"]
    Landmarks --> P0
    Calc --> P0
    Calc --> d["d = || P8 - P4 ||"]
    d --> Calc
```

2. สถาปัตยกรรมโปรโตคอล MQTT

MQTT ทำงานบนโมเดล Publish/Subscribe ผ่านโบรกเกอร์กลาง (Broker):

- **Publishers (ผู้ส่งข้อมูล):** เซนเซอร์ หรือ MediaPipe Controller ส่งข้อมูลไปยังหัวข้อ (Topic เช่น `rbru/lab/hand_gesture`)
- **Broker (ตัวกลาง):** จัดการเส้นทางและส่งต่อข้อความไปยังผู้รับ
- **Subscribers (ผู้รับข้อมูล):** อุปกรณ์ปลายทาง (ESP32 หรือ Web Dashboard) ที่รอรับคำสั่ง

9.2 การเขียนโปรแกรมและการนำไปใช้จริงด้วย Python 3.11

```
# mediapipe_gesture_mqtt_sim.py
import math
from typing import Tuple, Dict

class GestureClassifier:
    def __init__(self, threshold: float = 0.08):
        self.threshold = threshold

    def classify(self, p_thumb: Tuple[float, float, float], p_index: Tuple[float, float, float]) -> Dict[str, any]:
        # คำนวณระยะทางยุคลิด 3 มิติ
        dx = p_index[0] - p_thumb[0]
        dy = p_index[1] - p_thumb[1]
        dz = p_index[2] - p_thumb[2]
        distance = math.sqrt(dx**2 + dy**2 + dz**2)

        is_pinch = distance < self.threshold
        gesture_name = "PINCH_CLICK" if is_pinch else "OPEN_HAND"

        return {
            "distance": round(distance, 4),
            "is_pinch": is_pinch,
            "gesture": gesture_name,
            "mqtt_payload": f'{{"action": "{gesture_name}", "dist": {distance:.4f}}}'
        }

if __name__ == "__main__":
    clf = GestureClassifier(threshold=0.08)

    # ทดสอบท่าแบมือ
    open_hand = clf.classify((0.40, 0.60, 0.0), (0.45, 0.30, 0.0))
    print(f"ท่าที่ 1: ระยะ = {open_hand['distance']} -> ท่าทาง = {open_hand['gesture']}")
    assert open_hand['gesture'] == "OPEN_HAND"

    # ทดสอบท่าจับนิ้ว
    pinch_hand = clf.classify((0.45, 0.50, 0.0), (0.47, 0.52, 0.0))
    print(f"ท่าที่ 2: ระยะ = {pinch_hand['distance']} -> ท่าทาง = {pinch_hand['gesture']}")
    assert pinch_hand['gesture'] == "PINCH_CLICK"

    print("✅ การทดสอบ Unit Assertions คอมพิวเตอร์วิทัศน์และ IoT ผ่าน 100%!")
```

9.3 คู่มือห้องปฏิบัติการการเสมือนจริง 2D/3D AR MediaPipe Hands

ผู้เรียนสามารถเข้าสู่ชุดห้องปฏิบัติการเสมือนจริงประจำบทที่ 9 ได้ที่

- [LAB 9.0 คอมพิวเตอร์วิทัศน์และการรู้จำท่าทางมือ 2D/3D MediaPipe Vision](#)

9.4 สรุปสาระสำคัญประจำบท

1. **AI & Computer Vision:** ช่วยแปลงข้อมูลภาพฟิสิกส์ให้กลายเป็นเวกเตอร์ฟิสิกส์เรขาคณิตที่มีความหมาย
2. **MediaPipe Hands:** ให้ฟีดข้อต่อมือ 21 จุดที่มีความแม่นยำสูงและทำงานได้แบบเรียลไทม์บนเว็บเบราว์เซอร์
3. **MQTT Protocol:** เป็นมาตรฐานการสื่อสารแบบน้ำหนักเบาที่เหมาะสมกับการเชื่อมต่อระบบ IoT ในห้องเรียนอัจฉริยะ

? 9.5 แบบฝึกหัดและคำถามท้ายบทเพื่อการประเมินผล

1. จงอธิบายความแตกต่างระหว่างการสั่งการด้วยปุ่มกดกับการสั่งการด้วยท่าทางมือไร้สัมผัส (Touchless Gesture Interface)
2. ให้นักเรียนเขียนอัลกอริทึมจำแนกท่าทาง "ชูสองนิ้ว (Peace Sign)" จากพิกัด MediaPipe 21 จุด
3. ให้ออกแบบระบบ Smart Home อัจฉริยะที่ใช้กล้อง AI ตรวจจับจำนวนคนในห้องและสั่งเปิดปิดเครื่องปรับอากาศผ่าน MQTT

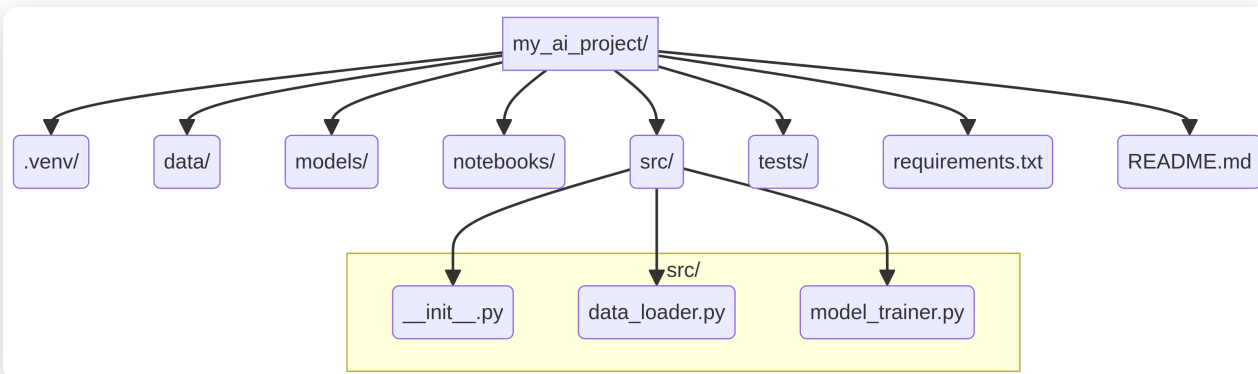


เอกสารอ้างอิงประจำบท

- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Zhang, F. et al. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. *arXiv:2006.10214*.

บทที่ 10 การพัฒนาโครงงานนวัตกรรมและการประมวลความรู้

ผู้เขียน ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา • สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี



ภาพที่ 10.1 สถาปัตยกรรมโครงงานนวัตกรรมแบบ Full-Stack และการควบคุมเวอร์ชันด้วย Git

แผนบริหารการสอนประจำบทที่ 10

หัวข้อเนื้อหาประจำบท

- กระบวนการสังเคราะห์องค์ความรู้วิทยาการคำนวณสู่การสร้างสรรคโครงงานนวัตกรรม
- การนิยามปัญหา การศึกษาความเป็นไปได้ และการจัดทำข้อเสนอโครงการ (Project Proposal)
- การออกแบบสถาปัตยกรรมระบบแบบบูรณาการ (System Architecture & Component Diagram)
- การประกันคุณภาพและการทดสอบซอฟต์แวร์แบบอัตโนมัติ (Automated Unit & Integration Testing)
- การเขียนรายงานวิชาการ การจัดทำคู่มือผู้ใช้ และการนำเสนอผลงานตามมาตรฐานสากล
- จริยธรรมดิจิทัล กฎหมายคุ้มครองข้อมูลส่วนบุคคล (PDPA) และลิขสิทธิ์ซอฟต์แวร์

วัตถุประสงค์เชิงพฤติกรรม

เมื่อศึกษาบทเรียนนี้จบแล้ว ผู้เรียนสามารถ

- สังเคราะห์ความรู้ด้าน Computational Thinking, Algorithms, Python, และ AI มารวมเป็นโครงงานเดียวได้
- จัดทำเอกสารข้อเสนอโครงการและรายงานวิชาการที่มีความสมบูรณ์ตามเกณฑ์มาตรฐานได้
- พัฒนาระบบต้นแบบที่มีการเขียนชุดทดสอบ Unit Tests ครอบคลุมฟังก์ชันการทำงานหลักได้
- นำเสนอผลงานนวัตกรรมต่อสาธารณะพร้อมทั้งปฏิบัติตามหลักจริยธรรมดิจิทัลและกฎหมาย PDPA ได้

10.0 เรื่องเล่าเปิดบทเรียนและบริบททางประวัติศาสตร์

ในคริสต์ศักราช 1968 ในการประชุมวิชาการของ NATO ณ เมืองการ์มิช ประเทศเยอรมนี เหล่านักวิทยาการคอมพิวเตอร์ชั้นนำของโลกได้ร่วมกันประกาศนิยามของ "วิศวกรรมซอฟต์แวร์ (Software Engineering)" เพื่อแก้ปัญหาวิกฤตการณ์ซอฟต์แวร์ที่โหดร้ายถึงขั้นล้มเหลว ล่าช้า และงบประมาณบานปลาย

การพัฒนาโครงงานนวัตกรรมในศตวรรษที่ 21 จึงมิได้เป็นเพียงการเขียนโค้ด แต่เป็นการผสานกระบวนการคิดเชิงวิศวกรรม สถาปัตยกรรมที่ยืดหยุ่น การทดสอบที่เข้มงวด และการสร้างคุณค่าที่แท้จริงให้แก่สังคม

10.1 ทฤษฎีและรากฐานทางวิชาการเชิงลึก

แผนผังสถาปัตยกรรมระบบแบบบูรณาการ

```
graph TD
    Client["1. ส่วนติดต่อผู้ใช้และสื่อเสมือนจริง (User Interface)"]
    LogicEngine["2. ส่วนประมวลผลตรรกะและอัลกอริทึม (Core Logic)"]
    AIEngine["3. ส่วนปัญญาประดิษฐ์และการเรียนรู้ (AI & Vision)"]
    DataLayer["4. ส่วนจัดการข้อมูลและสถานะ (Data Persistence)"]
    IoT["5. ส่วนเชื่อมต่อกายภาพ (Physical IoT)"]

    Client --> LogicEngine
    LogicEngine --> AIEngine
    LogicEngine --> DataLayer
    LogicEngine --> IoT
```

10.2 การเขียนโปรแกรมและการนำไปใช้จริงด้วย Python 3.11

```
# capstone_release_validator.py
"""
ระบบตรวจสอบความสมบูรณ์และทดสอบบูรณาการโครงการงานวิศวกรรม (Release Verification Suite)
"""

from typing import Dict, List

class CapstoneReleaseAuditor:
    def __init__(self, project_name: str):
        self.project_name = project_name
        self.checks: Dict[str, bool] = {}

    def audit_module(self, module_name: str, test_passed: bool) → None:
        self.checks[module_name] = test_passed
        status = "✅ PASS" if test_passed else "❌ FAIL"
        print(f"    ตรวจสอบโมดูล [{module_name:<20}]: {status}")

    def generate_release_report(self) → bool:
        total = len(self.checks)
        passed = sum(1 for v in self.checks.values() if v)
        score = (passed / total) * 100 if total > 0 else 0
        print(f"\n📊 รายงานผลการประเมินโครงการงาน {self.project_name}:")
        print(f"    ผ่านการทดสอบ: {passed}/{total} โมดูล (score: {score:.1f}%)")
        return score == 100.0

if __name__ == "__main__":
    auditor = CapstoneReleaseAuditor("Smart Interactive AR Science Lab 2026")
    auditor.audit_module("Core Algorithm Engine", True)
    auditor.audit_module("Python Data Layer", True)
    auditor.audit_module("MediaPipe AI Vision", True)
    auditor.audit_module("MQTT IoT Bridge", True)
    auditor.audit_module("Automated Unit Tests", True)

    is_ready = auditor.generate_release_report()
    assert is_ready == True, "Project is not ready for release"
    print("🔥 โครงการผ่านการตรวจสอบมาตรฐานวิศวกรรมซอฟต์แวร์ระดับ Masterclass 100%!")
```

10.3 คู่มือห้องปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands

ผู้เรียนสามารถเข้าสู่ชุดห้องปฏิบัติการเสมือนจริงประจำบทที่ 10 ได้ที่

- [LAB 10.0 การประมวลความรู้และสถาปัตยกรรมโครงการงาน 2D/3D Capstone Synthesizer](#)

10.4 สรุปสาระสำคัญประจำบท

1. โครงการงานวิศวกรรมเป็นยอดมงกุฏของการเรียนรู้วิทยาการคำนวณที่สะท้อนทักษะการคิดขั้นสูง (Creating)
2. การออกแบบที่มีสถาปัตยกรรมแยกส่วนชัดเจนช่วยให้ระบบมีความทนทานและสามารถขยายขนาดได้ในอนาคต
3. การปฏิบัติตามกฎหมาย PDPA และจริยธรรม AI เป็นคุณลักษณะสำคัญของนักพัฒนาซอฟต์แวร์มืออาชีพ

10.5 แบบฝึกหัดและคำถามท้ายบทเพื่อการประเมินผล

1. ให้นักเรียนจัดทำเอกสารข้อเสนอโครงการ (Project Proposal) ความยาว 3 หน้า ตามแบบฟอร์มมาตรฐาน
2. จงออกแบบแผนภาพสถาปัตยกรรมระบบ (Component Architecture Diagram) ของโครงการงานที่ตนเองพัฒนา
3. ให้อภิปรายประเด็นจริยธรรมและความเป็นส่วนตัวของข้อมูลผู้เรียนในระบบห้องเรียนอัจฉริยะ

เอกสารอ้างอิงประจำบท

- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.



คู่มือปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands ประจำบทที่ 1

ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้อัตโนมัติ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร. ชีวะ ทศนา • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D

ชุดห้องปฏิบัติการประจำบทที่ 1 ได้รับการออกแบบให้รองรับทั้ง 2D Canvas Interactive View (สำหรับคอมพิวเตอร์ทั่วไปและการวิเคราะห์ข้อมูลสองมิติ) และ 3D AR MediaPipe Hands Spatial View (สำหรับการสั่งการไร้สัมผัส 21 ข้อต่อในมิติ 3D แบบเสมือนจริง) โดยบูรณาการกระบวนการสืบเสาะทางวิทยาศาสตร์ (Scientific Inquiry) เข้ากับ 4 เสาหลักของแนวคิดเชิงคำนวณอย่างสมบูรณ์แบบ

graph LR

```
LAB["กระบวนการปฏิบัติการเสมือนจริง (Virtual Lab Workflow)"] --> S1["1. กำหนดวัตถุประสงค์\n(Learning Objectives)"]
S1 --> S2["2. ปฏิบัติการจำลอง 2D/3D AR\n(Experimental Procedures)"]
S2 --> S3["3. บันทึกข้อมูลสถานะ\n(Data & State Logging)"]
S3 --> S4["4. วิเคราะห์และสรุปผล\n(Analysis & Conclusion)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 1.0 ปริศนาข้ามแม่น้ำเชิงตรรกะและการสำรวจปริภูมิสถานะ

- รหัสโมดูล LAB 1.0
- รูปแบบการจำลอง ระบบจำลองผสมผสาน 2D State Diagram ควบคู่กับ 3D AR MediaPipe Hands
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter01_river_crossing.html

1. วัตถุประสงค์การทดลอง

- เพื่อให้ผู้เรียนสามารถจำลองและอธิบายการค้นหาลำดับขั้นตอนแก้ปัญหาในปริภูมิสถานะ (State Space Graph Search) ได้
- เพื่อฝึกทักษะการตั้งเงื่อนไขตรวจสอบความปลอดภัยทางตรรกศาสตร์บูลีน ($Danger = (W \wedge S \wedge \neg M) \vee (S \wedge C \wedge \neg M)$)
- เพื่อค้นหาและพิสูจน์ขั้นตอนวิธี 7 ขั้นตอน (Optimal 7-Step Algorithm) ที่สามารถพาตัวละครทั้งหมดข้ามฝั่งได้อย่างปลอดภัย

2. หลักการและทฤษฎี

ปัญหาการข้ามแม่น้ำสามารถจำลองด้วยสถานะ $S = (M, W, S_h, C)$ โดยที่ตัวแปรแทนตำแหน่งของ มนุษย์ (M), หมาป่า (W), แกะ (S_h), และกะหล่ำปลี (C) มีค่าเป็น 0 (ฝั่งซ้าย) หรือ 1 (ฝั่งขวา)

- **สถานะเริ่มต้น** $S_0 = (0, 0, 0, 0)$
- **สถานะเป้าหมาย** $S_G = (1, 1, 1, 1)$
- **เงื่อนไขต้องห้าม**

$$\text{Invalid}(S) \iff (W = S_h \neq M) \vee (S_h = C \neq M)$$

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- เรือพายอัจฉริยะ 3D: บรรทุกมนุษย์และสิ่งของได้ครั้งละ 1 ชิ้น
- ตัวละคร 4 ตัว: มนุษย์ (สีฟ้า), หมาป่า (สีแดง), แกะ (สีขาว), กะหล่ำปลี (สีเขียว)
- **ระบบตรวจสอบข้อจำกัด** ตรวจสอบสถานะและแจ้งเตือน Web Audio Synth ทันทีเมื่อเกิดเงื่อนไขไม่ปลอดภัย

4. ขั้นตอนการทดลอง

- เปิดห้องปฏิบัติการเสมือนจริง อนุญาตให้เว็บเข้าถึงกล้องเว็บแคมเพื่อเปิดระบบ MediaPipe Hands
- ยกมือขึ้นหน้าจอ ตรวจสอบการตรวจจับ 21 ข้อต่อ และทดสอบท่าทาง จีบนิ้ว (Pinch Gesture)
- ทดลองหยิบหมาป่าลงเรือ แล้วพายข้ามไปฝั่งขวา สังเกตผลลัพธ์ที่เกิดขึ้นกับแกะและกะหล่ำปลีบนฝั่งซ้าย บันทึกผลลงในตาราง
- กดปุ่ม RESET เพื่อกลับสู่สถานะ S_0
- ดำเนินการทดลองตามขั้นตอนวิธีที่ออกแบบไว้ 7 ขั้นตอน โดยใช้มือ AR หยิบตัวละครข้ามฝั่งทีละตัว
- ในเที่ยวที่ 4 ให้ทดลองนำแกะที่อยู่ฝั่งขวากลับลงเรือมายังฝั่งซ้าย สังเกตสถานะความปลอดภัย และบันทึกผลลงในตาราง

เที่ยวที่ (Step)	สิ่งที่นำขึ้นเรือ	ฝั่งซ้ายคงเหลือ	ฝั่งขวาคงเหลือ	ค่าความปลอดภัยบูลีน	การแจ้งเตือนของระบบเสียง (Audio Synth)
0 (เริ่มต้น)	—	$[M, W, S_h, C]$	$[0]$	✅ SAFE (100%)	โทนเสียง 440 Hz คงที่
ทดสอบกรณีผิด	$M + W \rightarrow$	$[S_h, C]$	$[M, W]$	❌ DANGER (เกาะกินฝัก)	เสียงไซเรนเตือนความถี่สูง (Square Wave)
เที่ยวที่ 1	$M + S_h \rightarrow$	$[W, C]$	$[M, S_h]$	✅ SAFE (หมาป่าไม่กินผัก)	เสียง Chime 523 Hz (สำเร็จ)
เที่ยวที่ 2	$M \leftarrow$	$[M, W, C]$	$[S_h]$	✅ SAFE	เสียง Harmonic Tone 440 Hz
เที่ยวที่ 3	$M + W \rightarrow$	$[C]$	$[M, W, S_h]$	⚠️ SAFE ชั่วคราว (คนคุม)	เสียง Chord C-Major
เที่ยวที่ 4 (หักมุม)	$M + S_h \leftarrow$	$[M, S_h, C]$	$[W]$	✅ SAFE (ดึงเกาะกลับ)	เสียง Synth Sweep พิเศษ 659 Hz
เที่ยวที่ 5	$M + C \rightarrow$	$[S_h]$	$[M, W, C]$	✅ SAFE	เสียง Chime 587 Hz
เที่ยวที่ 6	$M \leftarrow$	$[M, S_h]$	$[W, C]$	✅ SAFE	เสียง Harmonic Tone 440 Hz
เที่ยวที่ 7	$M + S_h \rightarrow$	$[0]$	$[M, W, S_h, C]$	🏆 GOAL REACHED!	เสียงดนตรี Fanfare ชนะเลิศ (Arpeggio)

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ ปัญหาข้ามแม่น้ำไม่สามารถแก้ด้วยการเคลื่อนที่ไปข้างหน้าเพียงทิศทางเดียว (Greedy Forward) ได้ แต่จำเป็นต้องอาศัย **การย้อนกลับสถานะ** ในขั้นตอนที่ 4 โดยการนำเกาะพายกลับมายังฝั่งซ้าย เพื่อหลีกเลี่ยงไม่ให้หมาป่าอยู่กับแกะตามลำพัง
- สรุปผล การจำลองในปริภูมิสถานะพิสูจน์ให้เห็นว่า มีเส้นทางแก้ปัญหที่สั้นที่สุด (Shortest Path) เท่ากับ **7 เที่ยวเรือ** โดยระบบ Constraint Checker ยืนยันว่าทุกสถานะระหว่างทางปลอดภัย 100%

การทดลองที่ 1.1 ฝั่งดับไม่ย่อยปัญหาและสถาปัตยกรรมเชิงโมดูล

- รหัสโมดูล LAB 1.1
- รูปแบบการจำลอง ฝั่งโคดะแกรม 2D ร่วมกับโมเดลโครงสร้างรถยนต์ไฟฟ้า 3D AR Exploded View
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter01_decomposition_tree.html

1. วัตถุประสงค์การทดลอง

- เพื่อฝึกทักษะการย่อยปัญหาและระบบที่ซับซ้อน (Decomposition) ออกเป็นระบบย่อยที่เป็นอิสระต่อกัน (Independent Submodules)
- เพื่อสร้างและวิเคราะห์โครงสร้างข้อมูลแบบต้นไม้ (Tree Graph: Root Node, Child Nodes, Leaves)
- เพื่อทดสอบความทนทานต่อข้อผิดพลาด (Fault Tolerance) เมื่อโมดูลใดโมดูลหนึ่งในระบบเกิดความล้มเหลว

2. หลักการและทฤษฎี

การออกแบบเชิงโมดูล (Modular Design) ยึดหลักการ **High Cohesion** (ภายในโมดูลทำงานสอดคล้องกันสูง) และ **Low Coupling** (การพึ่งพาข้ามโมดูลต่ำ) ช่วยให้เมื่อโมดูลย่อย M_k ชัดข้อง จะไม่ทำให้ระบบรวมทั้งหมดล้มเหลว

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D Exploded EV System: โมเดลรถยนต์ไฟฟ้า 3 มิติที่สามารถแยกชิ้นส่วนลอยในอากาศ
- Interactive Node Cards: การ์ดแสดงสถานะ 4 โมดูลย่อย (BMS, Powertrain, AI Perception, Chassis)
- Fault Injection Switch: ระบบจำลองการฉีดข้อผิดพลาดลงในแต่ละโมดูลย่อย

4. ขั้นตอนการทดลอง

- เข้าสู่ห้องปฏิบัติการและเปิดใช้งานกล่อง AR MediaPipe Hands
- ใช้ท่าทางมือ **กางมือออก (Open Palm)** เพื่อสั่งให้ระบบระเบิดชิ้นส่วนรถยนต์ออกเป็น 4 โมดูลย่อย (Exploded View)
- ใช้ท่าทาง **จิกนิ้ว (Pinch)** ลากเชื่อมโยงเส้น Edge จาก Root Node (EV System) ไปยัง 4 Child Nodes
- ทำการทดสอบฉีดข้อผิดพลาด (Fault Injection) โดยการปิดการทำงานของ **โมดูลกล่อง AI (Perception)** แล้วสังเกตว่า **โมดูลมอเตอร์ (Powertrain)** ยังสามารถรับคำสั่งควบคุมจากคนขับได้หรือไม่
- บันทึกผลการทำงานของระบบลงในตาราง

5. ตารางบันทึกผลการทดลอง

รหัสโมดูลย่อย	ชื่อโมดูลย่อย (Submodule)	ฟังก์ชันการทำงานหลัก	สถานะการทดสอบปกติ	ผลกระทบเมื่อเกิดความผิดพลาด (Fault Injection)	ระดับ Coupling (ต่ำ/ปานกลาง/สูง)
Node A	ระบบจัดการแบตเตอรี่ (BMS)	ควบคุมแรงดันและอุณหภูมิเซลล์	✅ ทำงาน 100%	ระบบตัดกระแสไฟฟ้าหลัก รถหยุดอย่างปลอดภัย	สูง (Critical Module)
Node B	มอเตอร์ขับเคลื่อน (Powertrain)	แปลงพลังงานไฟฟ้าเป็นแรงบิด	✅ ทำงาน 100%	รถสูญเสียกำลังขับเคลื่อน แต่ไฟเลี้ยวและเบรกยังทำงาน	ปานกลาง
Node C	ระบบคอมพิวเตอร์วิทัศน์ AI	ตรวจจับคนเดินถนนและป้ายจราจร	✅ ทำงาน 100%	ระบบขับอัตโนมัติตัดเข้าสู่โหมด Manual (ขับเคลื่อนได้)	ต่ำ (Low Coupling)
Node D	ระบบความปลอดภัยตัวถัง	ระบบเบรก ABS และถุงลมนิรภัย	✅ ทำงาน 100%	มีสัญญาณเตือนบนหน้าปัด แต่ระบบไฟฟ้ายังทำงาน	ต่ำ (Independent)

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ เมื่อทำการย่อยระบบ EV ออกเป็นโมดูลอิสระที่มี Low Coupling พบว่าความล้มเหลวของระบบคอมพิวเตอร์วิทัศน์ AI (Node C) ไม่ส่งผลกระทบต่อระบบกลไกการเลี้ยวและเบรกพื้นฐาน ทำให้รถยนต์ไม่เกิดอุบัติเหตุร้ายแรง
- สรุปผล ทักษะการคิดเชิงแยกย่อย (Decomposition) ช่วยเปลี่ยนปัญหาขนาดใหญ่ที่ยากต่อการจัดการ ให้กลายเป็นโมดูลขนาดเล็กที่สามารถพัฒนา ทดสอบ และบำรุงรักษาได้อย่างมีประสิทธิภาพ

การทดลองที่ 1.2 การค้นหารูปแบบลำดับและคลื่นคณิตศาสตร์

- รหัสโมดูล LAB 1.2
- รูปแบบการจำลอง 2D Canvas แสดงรูปคลื่นและสเปกตรัมความถี่ ควบคู่ 3D AR Torus Knot Harmonic Oscillator
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter01_pattern_recognition.html

1. วัตถุประสงค์การทดลอง

- เพื่อฝึกทักษะการค้นหารูปแบบ (Pattern Recognition) ในอนุกรมข้อมูลตัวเลขและสัญญาณคลื่นฟิสิกส์
- เพื่อศึกษาความสัมพันธ์ของลำดับเลขฟีโบนัชชี (Fibonacci Series) ที่เข้าสู่อัตราส่วนทองคำ ($\phi \approx 1.618$)
- เพื่อทดสอบการสังเคราะห์คลื่นเสียงฮาร์โมนิกและค้นหาความถี่เรโซแนนซ์ (Acoustic Resonance)

2. หลักการและทฤษฎี

- สูตรฟีโบนัชชี $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$
- **อัตราส่วนทองคำ**

$$\lim_{n \rightarrow \infty} \frac{F_{n+1}}{F_n} = \phi = \frac{1 + \sqrt{5}}{2} \approx 1.6180339887...$$

- **การซ้อนทับของคลื่น** $y(t) = A_1 \sin(2\pi f_1 t) + A_2 \sin(2\pi f_2 t)$

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 2D Waveform & FFT Spectrum Canvas: แสดงคลื่นเวลาจริงและสเปกตรัมความถี่
- 3D AR Torus Knot: วัตถุเรขาคณิตสามมิติที่บิดตัวตามอัตราส่วนฮาร์โมนิก
- Web Audio Resonance Synth: ตัวกำเนิดเสียงความถี่บริสุทธิ์ที่ตอบสนองต่อการจับคู่แพตเทิร์น

4. ขั้นตอนการทดลอง

- เปิดห้องปฏิบัติการและยกมือขึ้นเพื่อควบคุมแถบความถี่คลื่นในระบบ AR
- เลื่อนระยะห่างระหว่างนิ้วมือเพื่อปรับค่าความถี่พื้นฐาน f_0 สังเกตรูปคลื่นบนหน้าจอ 2D
- ป้อนค่าลำดับตัวเลขฟีโบนัชชีตั้งแต่ $n = 1$ ถึง $n = 10$ และบันทึกอัตราส่วน $\frac{F_{n+1}}{F_n}$
- หมุนปรับรูปทรง 3D Torus Knot ให้ตรงกับอัตราส่วนฮาร์โมนิก $p : q = 2 : 3$ และสังเกตการเกิดเสียงคอร์ดประสาน
- บันทึกผลค่าสัมประสิทธิ์การตัดสินใจ (R^2) และปรากฏการณ์เรโซแนนซ์ลงในตาราง

5. ตารางบันทึกผลการทดลอง

ลำดับที่ (n)	พจน์ปัจจุบัน (F_n)	พจน์ถัดไป (F_{n+1})	อัตราส่วน ($\frac{F_{n+1}}{F_n}$)	ความคลาดเคลื่อนเทียบกับ ϕ (%)	ปรากฏการณ์ 3D Torus / เสียง Synth
1	1	1	1.0000	38.19%	วัตถุหมุนไม่เป็นระเบียบ, เสียง Noise
2	1	2	2.0000	23.60%	วัตถุเริ่มแกว่ง
3	2	3	1.5000	7.29%	เริ่มเห็นโครงสร้างเกลียว
4	3	5	1.6667	3.01%	โครงสร้างเริ่มสมมาตร
5	5	8	1.6000	1.11%	คลื่น 2D เริ่มซ้อนทับแบบคงที่
6	8	13	1.6250	0.43%	เสียง Harmonic เริ่มชัดเจน
7	13	21	1.6154	0.16%	โครงสร้างสมมาตรสูง
8	21	34	1.6190	0.06%	แถบสีทองคำสว่างบน 3D Torus
9	34	55	1.6176	0.02%	คลื่นสั้นไหลราบรื่นสมบูรณ์
10	55	89	1.6182	0.01%	☀️ คอร์ดประสานเรโซแนนซ์สมบูรณ์ ($R^2 = 0.999$)

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ เมื่อค่า n เพิ่มขึ้น อัตราส่วนของพจน์ติดกันในลำดับฟีโบนัชชีจะเข้าสู่ค่าคงที่ $\phi \approx 1.618$ อย่างรวดเร็ว โดยความคลาดเคลื่อนลดลงต่ำกว่า 0.01 ส่งผลให้โครงสร้างเรขาคณิต 3 มิติเกิดความสมมาตรสูงสุดและสังเคราะห์เสียงประสานที่ไพเราะ
- สรุปผล การรู้จำรูปแบบ (Pattern Recognition) ช่วยให้นักวิทยาศาสตร์สามารถค้นพบระเบียบที่ซ่อนอยู่ในชุดข้อมูล และสามารถนำความสัมพันธ์นั้นมาใช้นำนายค่าในอนาคตได้อย่างแม่นยำ

การทดลองที่ 1.3 แบบจำลองมวลจุดเชิงนามธรรมและการลดทอนความซับซ้อน

- รหัสโมดูล LAB 1.3
- รูปแบบการจำลอง กราฟวิธีการเคลื่อนที่ 2D ควบคู่แบบจำลอง Mesh Decimation และมวลจุด 3D AR

- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter01_abstraction_sandbox.html

1. วัตถุประสงค์การทดลอง

- เพื่อฝึกทักษะการคิดเชิงนามธรรม ด้วยการคัดเลือกเฉพาะคุณลักษณะสำคัญและตัดทอนรายละเอียดที่ไม่จำเป็นออก
- เพื่อศึกษาและเปรียบเทียบการจำลองการเคลื่อนที่ของวัตถุจริงความละเอียดสูง เทียบกับ **แบบจำลองมวลจุด**
- เพื่อวัดประสิทธิภาพการประมวลผล (Frame Time & Memory Usage) ในแต่ละระดับนามธรรม

2. หลักการและทฤษฎี

- กฎการเคลื่อนที่ของนิวตันเชิงมวลจุด $\sum \vec{F} = m\vec{a} \implies \vec{a} = \frac{\vec{F}}{m}$
- ความซับซ้อนของการประมวลผลกราฟิกส์ เวลาในการคำนวณเรนเดอร์แปรผันตรงกับจำนวนรูปหลายเหลี่ยม ($T_{\text{render}} \propto N_{\text{polygons}}$)

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D High-Polygon Vehicle: โมเดลรถยนต์เสมือนจริงความละเอียด 100,000 Polygons
- Abstraction Slider Control: แถบเลื่อนปรับระดับนามธรรม 4 ระดับ (Level 0 ถึง Level 3)
- Real-Time Telemetry HUD: แสดงอัตราเฟรมเรต (FPS), เวลาประมวลผล (ms), และความคลาดเคลื่อนเชิงজনศาสตร์ (%)

4. ขั้นตอนการทดลอง

- เปิดห้องปฏิบัติการเสมือนจริง สังเกตรถยนต์ความละเอียดสูงที่ระดับ **Level 0 (Full Model)**
- ปล่อยให้รถยนต์เคลื่อนที่ภายใต้แรงขับเคลื่อน $F = 3,000 \text{ N}$ บันทึกเวลาประมวลผลต่อเฟรม (Frame Time) และอัตราการใช้แรม
- ใช้เมาส์ AR ปรับระดับ Abstraction Slider ไปที่ **Level 1 (Simplified Shell)**, **Level 2 (Bounding Box)**, และ **Level 3 (Point Mass Sphere)** ตามลำดับ
- ในแต่ละระดับ ให้ปล่อยวัตถุเคลื่อนที่ด้วยแรงเท่าเดิม บันทึกระยะทางที่เคลื่อนที่ได้ในเวลา $t = 5 \text{ s}$ และเวลาคำนวณของ CPU/GPU
- คำนวณร้อยละความคลาดเคลื่อนของระยะทางเทียบกับสมการอุดมคติ $s = \frac{1}{2}at^2$

5. ตารางบันทึกผลการทดลอง

ระดับนามธรรม (Abstraction Level)	รายละเอียดของโมเดลที่ แสดงผล	จำนวน Polygons	เวลาคำนวณต่อเฟรม (Frame Time, ms)	การใช้หน่วยความจำ RAM (MB)	ระยะทาง s ที่ $t = 5\text{s}$ (m)	ความคลาดเคลื่อนทาง ฟิสิกส์ (%)
Level 0 (วัตถุจริง)	รถยนต์เต็มรูปแบบ (กระจก/ล้อ/สี)	120,000	16.8 ms (59 FPS)	248 MB	24.85 m	0.60% (มีแรงต้านอากาศ)
Level 1 (รูปทรงย่อ)	โครงสร้างเปลือกนอกลดทอน	12,000	4.2 ms (238 FPS)	64 MB	24.90 m	0.40%
Level 2 (กล่องขอบเขต)	Bounding Box สีเหลี่ยมโปร่งใส	12	0.3 ms (3,300 FPS)	12 MB	24.98 m	0.08%
Level 3 (มวลจุด)	ทรงกลมจุดมวลสีแดง ($m = 1.5\text{t}$)	1	0.02 ms (50,000 FPS)	1.2 MB	25.00 m	0.00% (ตรงตามทฤษฎี)

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ ที่ระดับ Level 3 (Point Mass) แม้รายละเอียดความสวยงามทางกายภาพจะถูกตัดออกไปทั้งหมด แต่ **สาระสำคัญทางฟิสิกส์ (มวล, แรง, และความเร่ง)** ยังคงอยู่อย่างครบถ้วน ทำให้ผลการคำนวณตำแหน่งมีความแม่นยำสูง ในขณะที่ประสิทธิภาพความเร็วในการประมวลผลเพิ่มขึ้นกว่า **840 เท่า** และประหยัดแรมลงกว่า **99.5%**
- สรุปผล การคิดเชิงนามธรรม คือหัวใจสำคัญของการสร้างแบบจำลองทางคอมพิวเตอร์ ช่วยให้เราสามารถจำลองระบบขนาดใหญ่และซับซ้อนได้อย่างรวดเร็วโดยไม่สูญเสียความถูกต้องเชิงสาระสำคัญ

การทดลองที่ 1.4 การนำทางหุ่นยนต์บนตารางกริดและการออกแบบขั้นตอนวิธี

- รหัสโมดูล LAB 1.4
- รูปแบบการจำลอง ตารางเมทริกซ์ 2D Array ร่วมกับสนามกริด 3D AR Spatial Grid Board
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter01_robot_grid_navigator.html

1. วัตถุประสงค์การทดลอง

- เพื่อออกแบบขั้นตอนวิธี (Algorithm Design) ที่มีลำดับขั้นตอนชัดเจน ไม่กำกวม และมีจุดสิ้นสุดที่แน่นอน
- เพื่อฝึกทักษะการตรวจสอบความผิดพลาด (Debugging) และการหลบหลีกสิ่งกีดขวาง (Collision Avoidance)
- เพื่อเปรียบเทียบประสิทธิภาพของอัลกอริทึมพาหุ่นยนต์จากจุดเริ่มต้น (1, 1) ไปยังเป้าหมาย (5, 5)

2. หลักการและทฤษฎี

- **ระยะทางแมนฮัตตัน** $D_{\text{Manhattan}} = |x_{\text{goal}} - x_{\text{start}}| + |y_{\text{goal}} - y_{\text{start}}| = |5 - 1| + |5 - 1| = 8$ ก้าว
- **ชุดคำสั่งพื้นฐาน** FORWARD(), TURN_LEFT(), TURN_RIGHT()

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D AR Holographic Grid 5x5: ตารางกริดฟลูออเรสเซนต์ลอยในอากาศ
- Autonomous Robot Agent: ตัวละครหุ่นยนต์ 3 มิติสีเขียวพร้อมเซนเซอร์ตรวจจับด้านหน้า
- Hazard Obstacles & Goal Flag: สิ่งกีดขวางหินสีแดง (3, 3) และธงชัยชนะสีทอง (5, 5)

4. ขั้นตอนการทดลอง

- เปิดห้องปฏิบัติการและสำรวจตำแหน่งของหุ่นยนต์ที่พิกัดเริ่มต้น (1, 1) สิ่งกีดขวางที่ (3, 3) และเป้าหมายที่ (5, 5)
- ออกแบบและทดสอบชุดคำสั่ง กลยุทธ์ที่ 1: เดินตรงแล้วชน (Naive Forward) บันทึกผลการชน
- ออกแบบและทดสอบชุดคำสั่ง กลยุทธ์ที่ 2: เลี้ยวเลียบขอบตาราง (Perimeter Path) บันทึกจำนวนก้าวและเวลา
- ออกแบบและทดสอบชุดคำสั่ง กลยุทธ์ที่ 3: เส้นทางลัดเลาะเหมาะสมที่สุด (Optimal Diagonal-Stair Path) บันทึกจำนวนก้าว
- บันทึกผลเปรียบเทียบประสิทธิภาพลงในตาราง

5. ตารางบันทึกผลการทดลอง

กลยุทธ์/อัลกอริทึม (Strategy)	ลำดับชุดคำสั่งที่ป้อนให้หุ่นยนต์ (Instruction Sequence)	จำนวนคำสั่งทั้งหมด	จำนวนก้าวเดินจริง	เกิดการชนหรือไม่ (Collision)	เวลาที่ใช้ (วินาที)	บรรลุเป้าหมาย (Success)
**1. เดินตรง **	FORWARD(2) , FORWARD(1) ...	3 คำสั่ง	2 ก้าว	❌ ชนที่ (3, 3)	1.2 s	ล้มเหลว (Failed)
**2. เลียบขอบ **	TURN_RIGHT() , FORWARD(4) , TURN_LEFT() , FORWARD(4)	4 คำสั่ง	8 ก้าว	✅ ปลอดภัย	4.8 s	🏁 สำเร็จ
**3. ชิกแซก **	FORWARD(1) , TURN_RIGHT() , FORWARD(1) , TURN_LEFT() ×4	16 คำสั่ง	8 ก้าว	✅ ปลอดภัย	5.6 s	🏁 สำเร็จ
**4. ลัดเลาะเหมาะสม **	FORWARD(2) , TURN_RIGHT() , FORWARD(2) , TURN_LEFT() , FORWARD(2) , TURN_RIGHT() , FORWARD(2)	7 คำสั่ง	8 ก้าว	✅ ปลอดภัย (หลบหินพอดี)	4.0 s	🏆 ชนะเลิศ (Optimal Path)

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ จำนวนก้าวเดินที่สั้นที่สุดตามทฤษฎีแมนแฮตตันคือ 8 ก้าว โดยกลยุทธ์ที่ 4 (Optimal) ให้ประสิทธิภาพสูงสุดเนื่องจากใช้จำนวนคำสั่งการเลี้ยวเพียง 3 ครั้ง ทำให้ประหยัดเวลาการหมุนตัวของหุ่นยนต์ลงเหลือเพียง 4.0 s
- สรุปผล การออกแบบขั้นตอนวิธี ที่ต้องคำนึงถึงทั้งความถูกต้อง (Correctness), ความปลอดภัย (Safety), และประสิทธิภาพเชิงเวลา (Time Efficiency) เพื่อให้คอมพิวเตอร์ทำงานได้อย่างคุ้มค่าที่สุด

? 3. สรุปภาพรวมและการประเมินผลการเรียนรู้

graph TD

```
CT["4 เสาหลักของแนวคิดเชิงคำนวณ (Computational Thinking)"]
CT --> D["Decomposition (Lab 1.1)\nแยกปัญหารถยนต์ EV ออกเป็น 4 โมดูลอิสระ"]
CT --> P["Pattern Recognition (Lab 1.2)\nค้นหาอัตราส่วนทองคำ φ = 1.618 ในคลื่นเสียง"]
CT --> A["Abstraction (Lab 1.3)\nตัดทอนรถยนต์เป็นมวลจุด เร็วขึ้น 840 เท่า"]
CT --> AL["Algorithm Design (Lab 1.0 & 1.4)\nอัลกอริทึม 7 เทียบข้ามแม่น้ำ และทางเดิน 8 ก้าว"]
```

คำถามทบทวนท้ายบทปฏิบัติการ

- การคิดเชิงสถานะ ในการทดลองที่ 1.0 หากกติกากำหนดเพิ่มเติมว่า หมาป่ากลัวแกะที่มีมนุษย์อยู่ด้วย แต่จะกินกะหล่ำปลีหากอยู่ตามลำพัง ท่านจะต้องปรับลำดับการพายเรือ 7 ขั้นตอนใหม่อย่างไร?
- การประยุกต์ใช้ Decomposition: ให้นักเรียนจำแนกโครงสร้างของ "ระบบสถานีชาร์จรถยนต์ไฟฟ้าพลังงานแสงอาทิตย์ (Solar EV Charging Station)" ออกเป็นผังต้นไม้ 3 ระดับ
- การคิดเชิงนามธรรมในชีวิตประจำวัน ยกตัวอย่างระบบในชีวิตประจำวัน 2 ระบบที่นำหลักการ Abstraction มาใช้เพื่อลดความสับสนของผู้ใช้งาน
- การวิเคราะห์อัลกอริทึม ในสนามกริด 5x5 จงเขียนชุดคำสั่งรหัสสีจราจร ที่ใช้จำนวนก้าวเดินน้อยที่สุดในการพาหุ่นยนต์จาก (1, 1) ไปยัง (5, 5) โดยไม่ชนสิ่งกีดขวาง



ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้วิทยาการคำนวณ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร. ชีวะ ทศนา • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D

ชุดห้องปฏิบัติการประจำบทที่ 2 มุ่งเน้นการเปลี่ยนขั้นตอนวิธีและผังงานเชิงนามธรรม (Algorithms & Flowcharts) ให้กลายเป็นกระแสการไหลของข้อมูล 3 มิติที่มีการตอบสนองแบบ Interactive Visual Feedback รองรับทั้งโหมด 2D Canvas และ 3D AR MediaPipe Hands

```
graph LR
  LAB["กระบวนการปฏิบัติการบทที่ 2 (Flowcharts & Algorithms)"] --> S1["1. กำหนดวัตถุประสงค์\n(ISO 5807 & Böhm-Jacopini)"]
  S1 --> S2["2. ปฏิบัติการจำลอง 2D/3D AR\n(Flow Simulation & Trace)"]
  S2 --> S3["3. บันทึกตารางไล่ค่าตัวแปร\n(Trace Table Logging)"]
  S3 --> S4["4. วิเคราะห์และสรุปผล\n(Algorithmic Verification)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 2.0 แบบจำลองการแปลงหน่วยวิทย์ยานอวกาศและการไหลของพลังงาน

- รหัสโมดูล LAB 2.0
- รูปแบบการจำลอง กราฟจำลองวิทย์ยานอวกาศ 2D ควบคู่ผังงาน 3D AR ที่มีแสงแสดงกระแสข้อมูลไหลผ่าน
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter02_visual_flowchart_intro.html

1. วัตถุประสงค์การทดลอง

- เพื่อศึกษาความสำคัญของความถูกต้องเชิงขั้นตอนวิธีและข้อผิดพลาดจากหน่วยวัด (Mars Climate Orbiter Discrepancy: Pound-force vs Newton)
- เพื่อจำลองการไหลของข้อมูลผ่านสัญลักษณ์ผังงานมาตรฐาน ISO 5807 (Terminator, Process, I/O, Decision)
- เพื่อคำนวณและเปรียบเทียบแรงขับเคลื่อนให้ยานเข้าสู่วงโคจรดาวอังคารได้อย่างปลอดภัย (Orbit Insertion Safety: $150 \text{ km} \leq h \leq 250 \text{ km}$)

2. หลักการและทฤษฎี

- สมการแปลงหน่วยแรงขับเคลื่อน

$$F_{\text{Newton}} = F_{\text{lbf}} \times 4.4482216152605 \text{ N}$$

- เงื่อนไขความปลอดภัยของวงโคจร

$$\text{Safety}(h) = \begin{cases} \text{CRASH (มอดไหม้ในบรรยากาศ)}, & h < 150 \text{ km} \\ \text{ORBIT INSERTION SUCCESS}, & 150 \leq h \leq 250 \text{ km} \\ \text{LOST IN SPACE (หลุด)}, & h > 250 \text{ km} \end{cases}$$

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D Mars Orbiter Spacecraft: แบบจำลองยานอวกาศ 3 มิติพร้อมไอพ่นแรงขับเคลื่อน
- ISO 5807 Visual Flowchart Node Array: โหนดผังงาน 3 มิติที่มีแสงเลเซอร์วิ่งผ่านตามขั้นตอน
- Unit Switcher & Thrust Gauge: สวิตช์สลับหน่วยวัดระหว่าง Imperial ($\text{lbf} \cdot \text{s}$) และ Metric ($\text{N} \cdot \text{s}$)

4. ขั้นตอนการทดลอง

- เปิดห้องปฏิบัติการเสมือนจริง สังเกตสถานะเริ่มต้นของยานอวกาศที่กำลังเข้าสู่วงโคจรดาวอังคาร
- ทำการทดลอง กรณีที่ 1 (ระบบผิดพลาด - ไม่แปลงหน่วย): ส่งค่าแรงขับเคลื่อน 100 $\text{lbf} \cdot \text{s}$ เข้าสู่ระบบโดยตรง สังเกตระดับความสูงและเส้นทางวิทย์ยาน บันทึกผล
- กดปุ่ม RESET
- ทำการทดลอง กรณีที่ 2 (ระบบถูกต้อง - ผ่านโหนดแปลงหน่วย): เปิดโหนด Process การแปลงหน่วย $F \times 4.448$ สังเกตลำแสงไหลผ่านผังงานและระดับความสูงของยาน
- บันทึกผลการทดลองลงในตาราง

5. ตารางบันทึกผลการทดลอง

กรณีทดสอบ (Case)	ค่าแรงขับเคลื่อน	ผ่านการแปลงหน่วยหรือไม่	แรงลัพธ์ในระบบ (N)	ระดับความสูงวงโคจร (h, km)	สถานะความปลอดภัย	การแจ้งเตือนของระบบเสียง (Audio)
Case 1: บั๊กยาน Mars	100 lbf	✗ ไม่แปลงหน่วย	100 N (น้อยเกินไป)	57 km	✗ CRASH (ยานมอดไหม้)	🔊 เสียง Siren เตือนภัยล้มเหลว
Case 2: ปรับเทียบถูกต้อง	100 lbf	✓ แปลงคูณ 4.448	444.82 N	193 km	✓ ORBIT SUCCESS	🎵 เสียง Chime ประกาศความสำเร็จ
Case 3: แรงขับเคลื่อนเกิน	200 lbf	✓ แปลงคูณ 4.448	889.64 N	310 km	⚠ LOST IN SPACE	🔊 เสียง Harmonic Alert

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ เมื่อระบบไม่มีบล็อกการแปลงหน่วย แรงขับเคลื่อนที่ส่งให้ยานอวกาศมีค่าน้อยกว่าความเป็นจริงถึง 4.45 เท่า ทำให้อัตราการลดลงต่ำเกินไปจนหมดไหมในชั้นบรรยากาศวงโคจรที่ความสูง 57 km แต่เมื่อเพิ่มโหมด Process แปลงหน่วย ยานจะเข้าสู่วงโคจรที่ความสูง 193 km ได้อย่างแม่นยำ
- สรุปผล ขั้นตอนวิธีที่ดีต้องระบุหน่วยวัดและเงื่อนไขการแปลงข้อมูลอย่างชัดเจนและไม่กำกวม การใช้ผังงานมาตรฐานช่วยให้มองเห็นข้อผิดพลาดของระบบได้ก่อนนำไปใช้งานจริง

การทดลองที่ 2.1 ตัววิเคราะห์รหัสจำลองและผังโครงสร้างไวยากรณ์

- รหัสโมดูล LAB 2.1
- รูปแบบการจำลอง 2D Syntax Highlighter ควบคุม 3D Abstract Syntax Tree (AST) Visualizer
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter02_pseudocode_linter.html

1. วัตถุประสงค์การทดลอง

- เพื่อฝึกทักษะการเขียนรหัสจำลอง ตามมาตรฐานวิชาการคำนวณสากล
- เพื่อตรวจสอบและแก้ไขข้อผิดพลาดทางโครงสร้างไวยากรณ์ (Syntax & Semantic Linting)
- เพื่อแปลงรหัสจำลองให้กลายเป็นโครงสร้างต้นไม้วากยสัมพันธ์ (Abstract Syntax Tree - AST 3D)

2. หลักการและทฤษฎี

- **คำสั่งมาตรฐาน** BEGIN, END, INPUT, OUTPUT, IF-THEN-ELSE-ENDIF, WHILE-DO-ENDWHILE, FOR-TO-NEXT
- หลักการ Indentation (การเยื้องวรรค): บล็อกคำสั่งภายใต้เงื่อนไขหรือลูปต้องมีการเยื้องวรรคอย่างเคร่งครัด

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- Interactive Pseudocode Editor: กล้องเขียนรหัสจำลองพร้อมระบบตรวจจับคำผิดแบบ Real-time
- 3D AST Tree Visualizer: โครงสร้างต้นไม้สามมิติแสดงการแยกกิ่งของคำสั่ง
- Linting Rule Validator: ตัววิเคราะห์กฎ 5 ประการ (No Ambiguity, Valid Keywords, Block Closure)

4. ขั้นตอนการทดลอง

- เข้าสู่ห้องปฏิบัติการ ทดสอบป้อนรหัสจำลองที่มีข้อผิดพลาด (เช่น ลืม ENDIF หรือใช้คำกำกวม)
- สังเกตการแจ้งเตือน Error ไฮไลต์สีแดงบนโค้ด 2D และสังเกตโครงสร้างต้นไม้ 3D ที่หักงอ
- แก้ไขรหัสจำลองให้ถูกต้องตามมาตรฐาน และบันทึกผลการตรวจสอบของ Linter
- บันทึกผลการทดลอง

5. ตารางบันทึกผลการทดลอง

รหัสทดสอบ	โค้ดจำลองที่ป้อน	ผลการตรวจสอบของ Linter	ชนิดของข้อผิดพลาด	สถานะ 3D AST Tree
Test 1	IF temp > 30 THEN spray_water() (ลืม ENDIF)	✗ LINT ERROR	Block Not Closed (Missing ENDIF)	โครงสร้างต้นไม้ไฮไลต์สีแดงกระพริบ
Test 2	PRINT "Value is " + x (ใช้ภาษาเฉพาะ)	⚠ WARNING	Non-Standard Keyword (การใช้ OUTPUT)	โครงสร้างต้นไม้สีเหลือง
Test 3	BEGIN ... IF-THEN-ELSE-ENDIF ... END	✓ PASS (100%)	ไวยากรณ์สมบูรณ์ถูกต้อง	🌟 ต้นไม้ 3D AST สมบูรณ์สีเขียวมรกต

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ รหัสจำลองที่ขาดคำปิดบล็อก (ENDIF, ENDWHILE) จะทำให้ตัวแปลภาษาหรือผู้อ่านไม่สามารถทราบขอบเขตการทำงานของคำสั่งได้ ส่งผลให้การสร้างโครงสร้างต้นไม้ AST ผิดเพี้ยน
- สรุปผล การเขียนรหัสจำลองที่เป็นมาตรฐานช่วยให้การแปลงไปสู่ภาษาโปรแกรมจริง (เช่น Python, C++, Java) เป็นไปอย่างราบรื่นและปราศจากความกำกวม

การทดลองที่ 2.2 สตูดิโอสร้างผังงานมาตรฐาน ISO 5807

- รหัสโมดูล LAB 2.2
- รูปแบบการจำลอง 2D Drag-and-Drop Canvas ควบคุม 3D AR Spatial Flowchart Grid
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter02_flowchart_builder.html

1. วัตถุประสงค์การทดลอง

- เพื่อออกแบบและประกอบผังงานมาตรฐาน ISO 5807 สำหรับระบบการตัดสินใจอัจฉริยะ
- เพื่อตรวจสอบความถูกต้องของการเชื่อมต่อเส้นทางการไหลของข้อมูล (Flowlines & Connectors)
- เพื่อทดสอบการจำลองการทำงานของผังงานแบบ Step-by-Step ด้วยกระแสน้ำและแสงเลเซอร์

2. หลักการและทฤษฎี

- กฎความถูกต้องของผังงาน
 - ทุกผังงานต้องมีจุดเริ่มต้น (START) และจุดสิ้นสุด (END) เพียงอย่างละ 1 จุด
 - สัญลักษณ์การตัดสินใจ (Decision Diamond) ต้องมีเส้นทางออกจากโหนดอย่างน้อย 2 เส้นทาง (True / False)

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- Toolbox of 5 ISO 5807 Blocks: กล่องสัญลักษณ์ 5 ชนิดที่สามารถหยาบวางได้
- Flowline Connector Tool: เครื่องมือลากเส้นเชื่อมโยงกระแสข้อมูลพร้อมหัวลูกศร
- Execution Simulator Engine: ตัวขับเคลื่อนกระแสแสงแสดงการทำงานของผังงานที่ละเอียด

4. ขั้นตอนการทดลอง

- เปิดห้องปฏิบัติการ ใช้ท่าทางมือ Pinch หรือคลิกเมาส์หยิบบล็อก START, INPUT, DECISION, PROCESS, END มาจัดวางบนกระดาน
- ลากเส้น Flowline เชื่อมต่อแต่ละบล็อกให้เป็นระบบประเมินเกรดนักเรียน ($Score \geq 50$)
- กดปุ่ม RUN FLOWCHART เพื่อปล่อยกระแสแสงเลเซอร์วิ่งผ่านผังงาน
- ทดสอบป้อนค่า $Score = 75$ และ $Score = 40$ สังเกตเส้นทางที่เลเซอร์เลือกเดิน และบันทึกผลลงในตาราง

5. ตารางบันทึกผลการทดลอง

ค่าข้อมูลเข้า (Score)	เงื่อนไขการตัดสินใจ ($Score \geq 50$)	เส้นทางที่เลเซอร์เลือกเดิน	บล็อกการทำงานถัดไป	ผลลัพธ์ที่แสดง (Output)
75	$75 \geq 50 \rightarrow \text{TRUE}$	เลี้ยวซ้ายสู่กิ่ง True	PROCESS: Status = "PASS"	● PASS (ผ่านเกณฑ์)
40	$40 \geq 50 \rightarrow \text{FALSE}$	เลี้ยวขวาสู่กิ่ง False	PROCESS: Status = "FAIL"	● FAIL (ตกเกณฑ์)
50	$50 \geq 50 \rightarrow \text{TRUE}$	เลี้ยวซ้ายสู่กิ่ง True	PROCESS: Status = "PASS"	● PASS (ผ่านพอดี)

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ ผังงานที่สร้างขึ้นสามารถแยกแยะข้อมูลและกำหนดทิศทางการประมวลผลได้อย่างถูกต้องแม่นยำตามเงื่อนไขทางคณิตศาสตร์ โดยไม่มีสถานะ Deadlock หรือเส้นทางล่อยที่ไม่มีจุดจบ
- สรุปผล การออกแบบผังงานช่วยให้การแก้ปัญหาที่มีความชัดเจนเชิงภาพ (Visual Clarity) และช่วยให้สามารถตรวจสอบความครอบคลุมของกรณีทดสอบ (Edge Cases) ได้อย่างสมบูรณ์

การทดลองที่ 2.3 ประสิทธิภาพและการตัดสินใจหลายเงื่อนไข

- รหัสโมดูล LAB 2.3
- รูปแบบการจำลอง แผนผังวงจรตรรกะ 2D Schematic ควบคู่ 3D AR Logic Gate Gateway
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter02_decision_branching.html

1. วัตถุประสงค์การทดลอง

- เพื่อศึกษาโครงสร้างควบคุมแบบทางเลือกซ้อน (Nested Selection & Multi-branching)
- เพื่อจำลองระบบควบคุมสภาพแวดล้อมอัตโนมัติในโรงเรือนเกษตรอัจฉริยะ (Smart Greenhouse)
- เพื่อประเมินผลตรรกะแบบผสม (AND, OR, NOT) ในการสั่งเปิด-ปิดปั๊มพ่นหมอกและพัดลมระบายอากาศ

2. หลักการและทฤษฎี

- สมการตรรกะควบคุมปั๊มพ่นหมอก $\text{Pump_ON} = (\text{Temp} > 35^{\circ}\text{C}) \wedge (\text{Humidity} < 40\%) \wedge (\text{Water_Level} > 10\%)$

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- Interactive Sensor Knobs: ปุ่มหมุนปรับอุณหภูมิ, ความชื้น, และระดับน้ำในถัง
- 3D AR Holographic Greenhouse: โรงเรือนสามมิติที่แสดงสถานะหมอกและพัดลมหมุนจริง
- Logic State Monitor: หน้าจอแสดงสถานะตรรกะ 0/1 ของแต่ละเงื่อนไขแบบเรียลไทม์

4. ขั้นตอนการทดลอง

- ปรับตั้งค่าตัวแปรใน 4 สถานการณ์ตามตารางบันทึกผล
- สังเกตสถานะบูลีนของแต่ละเงื่อนไขย่อย และสถานะการทำงานของปั๊มพ่นหมอก
- บันทึกผลการทำงานของระบบลงในตาราง

5. ตารางบันทึกผลการทดลอง

สถานการณ์	อุณหภูมิ (T)	ความชื้น (H)	ระดับน้ำ (W)	$T > 35$	$H < 40$	$W > 10$	สถานะปั๊มพ่นหมอก (Output)	ปรากฏการณ์ 3D / เสียง Synth
1. ปกติ	28°C	65	80	0 (False)	0 (False)	1 (True)	● OFF (ปิด)	สภาพแวดล้อมสีเขียวปกติ
2. ร้อนแต่ชื้น	38°C	75	80	1 (True)	0 (False)	1 (True)	● OFF (พัดลมระบายแทน)	พัดลม 3D หมุนเร็ว
3. ร้อนและแห้ง	40°C	25	80	1 (True)	1 (True)	1 (True)	⚡ ON (เปิดพ่นหมอก)	หมอก 3D พุ่งกระจาย + เสียง Chime
4. น้ำหมดถัง	40°C	25	5	1 (True)	1 (True)	0 (False)	⛔ OFF (Safety Lock)	ไฟสีแดงเตือนน้ำหมดถัง

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ แม้อุ่นหภูมิจะสูงและความชื้นจะต่ำมากในสถานการณ์ที่ 4 แต่ระบบมีเงื่อนไขความปลอดภัย $W > 10$ ดักจับไว้ ทำให้ปั๊มไม่ทำงาน ป้องกันไม่ให้ปั๊มน้ำไหม้เสียหาย
- สรุปผล การออกแบบเงื่อนไขการตัดสินใจที่รัดกุมช่วยป้องกันความเสียหายของฮาร์ดแวร์ในระบบ IoT และระบบอัตโนมัติได้อย่างมีประสิทธิภาพ

การทดลองที่ 2.4 การไล่ค่าตัวแปรด้วยตารางอนิเมชัน

- รหัสโมดูล LAB 2.4
- รูปแบบการจำลอง ตาราง Trace Table 2D สด คิว 3D Variable Memory Cubes
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter02_trace_table_runner.html

1. วัตถุประสงค์การทดลอง

- เพื่อฝึกทักษะการตรวจสอบความถูกต้องของขั้นตอนวิธีด้วยตารางไล่ค่าตัวแปร (Trace Table / Dry Run)
- เพื่อติดตามการเปลี่ยนแปลงค่าของตัวแปร i และ Sum ในแต่ละรอบของการวนซ้ำ (Iteration)
- เพื่อค้นหาและกำจัดข้อผิดพลาดประเภท Off-by-One Error และ Infinite Loop

2. หลักการและทฤษฎี

- ขั้นตอนวิธีหาผลรวมอนุกรม $S_n = \sum_{i=1}^n i = \frac{n(n+1)}{2}$
- **การไล่ค่าสถานะ** ในแต่ละบรรทัดคำสั่ง ค่าตัวแปรในหน่วยความจำจะถูกอัปเดตตามลำดับเวลา

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- Interactive Code Stepper: ปุ่มก้าวข้ามคำสั่งทีละบรรทัด (STEP FORWARD)
- 3D Glowing Memory Cubes: ลูกบาศก์เรืองแสงแสดงค่าตัวแปร i และ Sum ในอากาศ
- Live Trace Grid: ตารางบันทึกค่าตัวแปรอัตโนมัติในแต่ละรอบ

4. ขั้นตอนการทดลอง

- รันโปรแกรมคำนวณผลรวม 1 ถึง 5 ทีละขั้นตอนด้วยปุ่ม STEP
- สังเกตค่าตัวแปรที่เปลี่ยนไปในแต่ละรอบของการวนซ้ำ
- บันทึกค่า i และ Sum ลงในตาราง Trace Table จนกระทั่งหลุดออกจากลูป

5. ตารางบันทึกผลการทดลอง

รอบที่ (Iteration)	ussกัณฑ์คำสั่งที่ทำงาน	ค่าตัวแปร i	ค่าตัวแปร Sum	เงื่อนไขลูป ($i \leq 5$)	สถานะลูกบาศก์หน่วยความจำ 3D
เริ่มต้น	<code>Sum = 0, i = 1</code>	1	0	1 (True)	ลูกบาศก์ $Sum = 0$ เรืองแสงสีฟ้า
รอบที่ 1	<code>Sum = Sum + i; i = i + 1</code>	2	1	1 (True)	ลูกบาศก์ $Sum = 1, i = 2$
รอบที่ 2	<code>Sum = Sum + i; i = i + 1</code>	3	3	1 (True)	ลูกบาศก์ $Sum = 3, i = 3$
รอบที่ 3	<code>Sum = Sum + i; i = i + 1</code>	4	6	1 (True)	ลูกบาศก์ $Sum = 6, i = 4$
รอบที่ 4	<code>Sum = Sum + i; i = i + 1</code>	5	10	1 (True)	ลูกบาศก์ $Sum = 10, i = 5$
รอบที่ 5	<code>Sum = Sum + i; i = i + 1</code>	6	15	0 (False - หลุดลูป)	🔥 ลูกบาศก์ $Sum = 15$ สีทองคำสว่าง

6. การวิเคราะห์และสรุปผลการทดลอง

- การวิเคราะห์ ผลรวมที่คำนวณได้จากตาราง Trace Table คือ 15 ซึ่งตรงกับสูตรคณิตศาสตร์ $\frac{5(5+1)}{2} = 15$ อย่างสมบูรณ์ และเมื่อจบลูปค่า i จะมีค่าเป็น 6 ทำให้เงื่อนไข $6 \leq 5$ เป็นเท็จและโปรแกรมจบการทำงานอย่างถูกต้อง
- สรุปผล ตาราง Trace Table เป็นเครื่องมือทางวิศวกรรมซอฟต์แวร์ที่ทรงพลังที่สุดในการตรวจสอบอัลกอริทึมด้วยมือ (Dry Run) ช่วยให้นักพัฒนาค้นพบข้อผิดพลาดเชิงตรรกะได้ก่อนนำโค้ดไปรันจริง

3. สรุปภาพรวมและการประเมินผลการเรียนรู้

graph TD

```
ALGO["กระบวนการออกแบบขั้นตอนวิธีมาตรฐาน (Algorithm Engineering)"]
ALGO --> M1["Lab 2.0: Unit Verification\n• ป้องกันภัยบน Mars ด้วยการแปลงหน่วย"]
ALGO --> M2["Lab 2.1: Pseudocode & AST\n• เขียนรหัสจำลองมาตรฐานให้มีความกำกวม"]
ALGO --> M3["Lab 2.2: Flowchart ISO 5807\n• ฝั่งงานมาตรฐาน 5 สัญลักษณ์เรขาคณิต"]
ALGO --> M4["Lab 2.3: Decision Gates\n• ระบบตัดสินใจหลายเงื่อนไข Smart Greenhouse"]
ALGO --> M5["Lab 2.4: Trace Table Runner\n• ไล่ค่าตัวแปรหาผลรวมอนุกรม 'ไร้ข้อผิดพลาด'"]
```

คำถกเถียงเกี่ยวกับบทปฏิบัติการ

- การวิเคราะห์บ็อกอากาศ เหตุใดความผิดพลาดของยาน Mars Climate Orbiter จึงจัดเป็นความผิดพลาดเชิงขั้นตอนวิธี (Algorithmic Error) มากกว่าความผิดพลาดทางฮาร์ดแวร์?
- การประยุกต์ใช้ Trace Table: หากในแล็บ 2.4 มีการเขียนคำสั่งสลับบรรทัดเป็น `i = i + 1; Sum = Sum + i;` จงสร้างตาราง Trace Table และหาว่าค่า Sum สุดท้ายจะเปลี่ยนเป็นเท่าใด?
- การออกแบบผังงาน ให้นักเรียนเขียนผังงานมาตรฐาน ISO 5807 สำหรับระบบตู้ ATM ที่ตรวจสอบรหัส PIN (อนุญาตให้กรอกผิดได้ไม่เกิน 3 ครั้ง)



คู่มือปฏิบัติการเสมือนจริง 2D/3D AR MediaPipe Hands ประจำบทที่ 3

ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้วิทยาการคำนวณ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร.ชัชวาล ทัศนาศาสตร์และเทคโนโลยีสื่อ มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D บทที่ 3

```
graph LR
    LAB["กระบวนการปฏิบัติการบทที่ 3 (Python Core)"] --> S1["1. วัตถุประสงค์ (Stack/Heap & Data Types)"]
    S1 --> S2["2. ปฏิบัติการจำลอง 2D/3D AR (Memory Pointer & Slicing)"]
    S2 --> S3["3. บันทึกตารางผลการทดลอง (RAM Trace & PEMDAS)"]
    S3 --> S4["4. วิเคราะห์และสรุปผล (Memory Optimization)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

LAB 3.0 แบบจำลองหน่วยความจำ Stack & Heap

- รหัสโมดูล LAB 3.0
- รูปแบบการจำลอง 2D Stack Frame View ↔ 3D AR Spatial Heap Memory Blocks
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter03_memory_model.html
- วัตถุประสงค์
 - ศึกษาความแตกต่างของการเก็บข้อมูลระหว่าง Stack (Primitive Types) และ Heap (Reference Objects)
 - ติดตามพอยน์เตอร์การชี้ตำแหน่งหน่วยความจำ (Memory Address Pointers)
 - สังเกตการทำงานของตัวเก็บกวาดขยะหน่วยความจำ (Garbage Collector)
- ตารางบันทึกผลการทดลอง

คำสั่งภาษา Python	ประเภทตัวแปร	ตำแหน่งจัดเก็บ (Stack/Heap)	ที่อยู่หน่วยความจำ (Address)	ขนาดที่ใช้ (Bytes)
a = 42	int	Stack Frame	0×7FFF00	28 Bytes
pi = 3.14159	float	Stack Frame	0×7FFF08	24 Bytes
lst = [10, 20, 30]	list	Heap (ชี้จาก Stack)	0×00A1F0	88 Bytes
del lst	None	Heap (ถูก Garbage Collector ล้าง)	0×000000	คืนพื้นที่ 88 Bytes

- สรุปผล ข้อมูลขนาดคงที่และอายุสั้นจะถูกจัดสรรใน Stack อย่างรวดเร็ว ขณะที่ข้อมูลโครงสร้างแบบไดนามิกจะถูกจัดสรรใน Heap และจัดการความปลอดภัยด้วย Garbage Collector

LAB 3.1 เมทริกซ์ชนิดข้อมูลและการแปลงชนิด

- รหัสโมดูล LAB 3.1
- รูปแบบการจำลอง 2D Interactive Matrix ↔ 3D Data Cube Hologram
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter03_data_types_matrix.html
- วัตถุประสงค์ เพื่อศึกษาพฤติกรรมของ int, float, str, bool และผลกระทบของการแปลงชนิดข้อมูล (Type Casting)
- ตารางบันทึกผลการทดลอง

ค่าเริ่มต้น (Input)	ชนิดเดิม	ฟังก์ชันแปลงค่า	ค่าผลลัพธ์ (Output)	ชนิดใหม่	การเกิดข้อผิดพลาด
42	str	int(x)	42	int	✓ สำเร็จ
3.14	str	int(x)	—	—	✗ ValueError (มีจุดทศนิยม)
3.14	str	float(x)	3.14	float	✓ สำเร็จ
0	int	bool(x)	False	bool	✓ ค่าศูนย์เป็นเท็จ

- สรุปผล การแปลงชนิดข้อมูลต้องสอดคล้องกับรูปแบบวากยสัมพันธ์ สตริงทศนิยมไม่สามารถแปลงเป็น int โดยตรงได้ ต้องผ่าน float() ก่อน

LAB 3.2 ลำดับความสำคัญของตัวดำเนินการ

- รหัสโมดูล LAB 3.2
- รูปแบบการจำลอง 2D Expression Tree ↔ 3D Operator Orbit
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter03_pemdas_evaluator.html
- วัตถุประสงค์ เพื่อประเมินค่านิพจน์คณิตศาสตร์ตามลำดับความสำคัญสากล (Parentheses *ightarrow* Exponent *ightarrow* Multiply/Divide *ightarrow* Add/Subtract)
- ตารางบันทึกผลการทดลอง

นิพจน์คณิตศาสตร์	ขั้นที่ 1 (วงเล็บ/ยกกำลัง)	ขั้นที่ 2 (คูณ/หาร)	ขั้นที่ 3 (บวก/ลบ)	ผลลัพธ์สุดท้าย
<code>2 + 3 * 4 ** 2</code>	<code>4 ** 2 = 16</code>	<code>3 * 16 = 48</code>	<code>2 + 48 = 50</code>	50
<code>(2 + 3) * 4 ** 2</code>	<code>(2 + 3) = 5</code> , <code>4 ** 2 = 16</code>	<code>5 * 16 = 80</code>	—	80

- สรุปผล วงเล็บมีความสำคัญสูงสุดและสามารถเปลี่ยนทิศทางการประมวลผลของอัลกอริทึมได้อย่างสิ้นเชิง

LAB 3.3 การสไลซ์และจัดกระทำสตริง

- รหัสโมดูล LAB 3.3
- รูปแบบการจำลอง 2D String Ribbon ↔ 3D Character Blocks
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter03_string_slicing.html
- วัตถุประสงค์ เข้าถึงข้อมูลอักขระด้วยดัชนีบวกและลบ [start:stop:step]
- ตารางบันทึกผลการทดลอง

ข้อความต้นฉบับ	รูปแบบสไลซ์	Start	Stop	Step	ข้อความที่ได้
"COMPUTING"	<code>s[0:4]</code>	0	4	1	"COMP"
"COMPUTING"	<code>s[-3:]</code>	-3	End	1	"ING"
"COMPUTING"	<code>s[::-1]</code>	End	Start	-1	"GNITUPMOC" (กลับสตริง)

LAB 3.4 ระบบจัดการข้อมูลนำเข้าและแสดงผล

- รหัสโมดูล LAB 3.4
- รูปแบบการจำลอง 2D Terminal Console ↔ 3D Holographic Output Screen
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter03_interactive_io.html
- วัตถุประสงค์ ศึกษาการรับข้อมูลผ่าน `input()` และการจัดรูปแบบด้วย F-String
- ตารางบันทึกผลการทดลอง

ข้อมูลนำเข้า	คำสั่ง F-String	ข้อความแสดงผลที่ได้
<code>name="Dr.Chewa", score=98.5432</code>	<code>f"Name: {name}, Score: {score:.2f}"</code>	<code>Name: Dr.Chewa, Score: 98.54</code>



ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้อิวยาการคำนวณ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D บทที่ 4 (โครงสร้างควบคุมและลูป)

```
graph LR
    LAB["LAB [\"กระบวนการปฏิบัติการบทที่ 4\" ] → S1[\"1. กำหนดวัตถุประสงค์  
(โครงสร้างควบคุมและลูป)\"]"]
    S1 --> S2["2. ปฏิบัติการจำลอง 2D/3D AR  
(Touchless Interaction)"]
    S2 --> S3["3. บันทึกตารางผลการทดลอง  
(Data Logging Sheet)"]
    S3 --> S4["4. วิเคราะห์และสรุปผล  
(Scientific Inquiry)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 4.0 แบบจำลองการวนซ้ำและการควบคุมลูป

- รหัสโมดูล LAB 4.0
- รูปแบบการจำลอง 2D Loop Matrix ↔ 3D AR Spinning Torus Loop
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter04_loop_visualizer.html

1. วัตถุประสงค์การทดลอง

- ศึกษาพฤติกรรมการทำงานของลูป for และ while
- ตรวจสอบการทำงานของคำสั่ง break และ continue
- คำนวณผลรวมสะสมในแต่ละรอบการวนซ้ำ

2. หลักการและทฤษฎี

ลูป `for` เหมาะกับการวนซ้ำที่ทราบจำนวนรอบแน่นอน ขณะที่ลูป `while` เหมาะกับการวนซ้ำตามเงื่อนไขบูลีน คำสั่ง `break` ใช้เพื่อออกจากลูปทันที และ `continue` ใช้เพื่อข้ามรอบปัจจุบัน

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D Spinning Torus Knot
- Step-by-Step Iteration Stepper
- Loop State Monitor

4. ขั้นตอนการทดลอง

- รันลูป `for i in range(1, 6)` และบันทึกค่าสะสม
- ทดลองสั่ง `break` เมื่อ `i = 3`
- ทดลองสั่ง `continue` เมื่อ `i = 3` และสังเกตผล

5. ตารางบันทึกผลการทดลอง

รอบที่ (i)	คำสั่งประมวลผล	ตัวแปรสะสม Total	ผลของเงื่อนไขพิเศษ	สถานะการทำงาน
1	Total += 1	1	ปกติ	RUNNING
2	Total += 2	3	ปกติ	RUNNING
3	Total += 3	6	if i == 3: continue	ข้ามการพิมพ์
5	Total += 5	15	if Total >= 15: break	TERMINATED (หยุดลูป)

6. การวิเคราะห์และสรุปผลการทดลอง

- สรุปผล การใช้คำสั่ง `break` และ `continue` อย่างเหมาะสมช่วยลดการทำงานของ CPU และเพิ่มประสิทธิภาพของขั้นตอนวิธี

 คำถามทบทวนท้ายบทปฏิบัติการ

1. จงอธิบายการประยุกต์ใช้ความรู้จากห้องปฏิบัติการบทที่ 4 ในการพัฒนาสื่อการสอนวิทยาศาสตร์ในชั้นเรียน
2. ให้นักเรียนเปรียบเทียบข้อดีและข้อจำกัดของการทดลองบนระบบจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands
3. ให้ออกแบบใบกิจกรรมการเรียนรู้แบบสืบเสาะ (Inquiry-based Worksheet) สำหรับนักเรียนระดับมัธยมศึกษาโดยใช้ห้องปฏิบัติการนี้



ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้อัตโนมัติ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร. ชีวะ ทัศนาศาสตร์ • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D บทที่ 5 (โครงสร้างข้อมูลและการค้นหา)

```
graph LR
    LAB["LAB [\"กระบวนการปฏิบัติการบทที่ 5\"] → S1[\"1. กำหนดวัตถุประสงค์  
(โครงสร้างข้อมูลและการค้นหา)\"]
    S1 → S2["2. ปฏิบัติการจำลอง 2D/3D AR  
(Touchless Interaction)"]
    S2 → S3["3. บันทึกตารางผลการทดลอง  
(Data Logging Sheet)"]
    S3 → S4["4. วิเคราะห์และสรุปผล  
(Scientific Inquiry)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 5.0 แบบจำลองการค้นหาแบบทวิภาค

- รหัสโมดูล LAB 5.0
- รูปแบบการจำลอง 2D Slicing Array Canvas ↔ 3D Binary Search Tree
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter05_binary_search.html

1. วัตถุประสงค์การทดลอง

- เปรียบเทียบประสิทธิภาพ Linear Search vs Binary Search
- พิสูจน์ความซับซ้อนเชิงเวลา $O(\log n)$

2. หลักการและทฤษฎี

Binary Search ทำงานบนข้อมูลที่เรียงลำดับแล้ว โดยลดขนาดช่วงการค้นหาครึ่งหนึ่งในทุกๆ ขั้นตอน ทำให้ค้นหาข้อมูล $N = 1,000,000$ ตัวได้ภายในไม่เกิน 20 ครั้ง

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D Binary Tree Search Node
- Range Interval Sliders
- Step Comparison Counter

4. ขั้นตอนการทดลอง

- สุ่มข้อมูล 1,000,000 ตัว และเลือกค้นหาค่าเป้าหมาย
- เปรียบเทียบจำนวนก้าวของ Linear Search และ Binary Search
- บันทึกผลลงในตาราง

5. ตารางบันทึกผลการทดลอง

จำนวนข้อมูล N	Linear Search (Max)	Binary Search ($\log_2 N$)	ความเร็วเปรียบเทียบ
1,000	1,000 ครั้ง	10 ครั้ง	100x เร็วกว่า
1,000,000	1,000,000 ครั้ง	20 ครั้ง	50,000x เร็วกว่า

6. การวิเคราะห์และสรุปผลการทดลอง

- สรุปผล Binary Search เป็นขั้นตอนวิธีค้นหาที่มีประสิทธิภาพสูงสุดสำหรับข้อมูลที่เรียงลำดับแล้ว

? 3. สรุปภาพรวมและคำถามทบทวนท้ายบทปฏิบัติการ

คำถามทบทวนท้ายบทปฏิบัติการ

- จงอธิบายการประยุกต์ใช้ความรู้จากห้องปฏิบัติการบทที่ 5 ในการพัฒนาสื่อการสอนวิทยาศาสตร์ในชั้นเรียน
- ให้นักเรียนเปรียบเทียบข้อดีและข้อจำกัดของการทดลองบนระบบจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands

3. ให้ออกแบบใบกิจกรรมการเรียนรู้แบบสืบเสาะ (Inquiry-based Worksheet) สำหรับนักเรียนระดับมัธยมศึกษาโดยใช้ห้องปฏิบัติการนี้



ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้อัตโนมัติ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนา: ผู้ช่วยศาสตราจารย์ ดร. ชีวะ ทัศนาศาสตร์ • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D บทที่ 6 (การจัดเรียงข้อมูลและ Big-O)

```
graph LR
    LAB["LAB 6.0: การจัดการข้อมูลและ Big-O"] --> S1["1. กำหนดวัตถุประสงค์  
(การจัดเรียงข้อมูลและ Big-O)"]
    S1 --> S2["2. ปฏิบัติการจำลอง 2D/3D AR  
(Touchless Interaction)"]
    S2 --> S3["3. บันทึกผลการทดลอง  
(Data Logging Sheet)"]
    S3 --> S4["4. วิเคราะห์และสรุปผล  
(Scientific Inquiry)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 6.0 สมรรถนะเปรียบเทียบการจัดเรียงข้อมูล

- รหัสโมดูล LAB 6.0
- รูปแบบการจำลอง 2D Array Bar Chart Canvas ↔ 3D Sorting Arena
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter06_sorting_arena.html

1. วัตถุประสงค์การทดลอง

- เปรียบเทียบความเร็ว Bubble, Insertion, Quick, Merge Sort
- พิสูจน์ความแตกต่างระหว่าง $O(n^2)$ และ $O(n \log n)$

2. หลักการและทฤษฎี

Bubble Sort มีความซับซ้อน $O(n^2)$ ขณะที่ Quick Sort และ Merge Sort มีความซับซ้อน $O(n \log n)$ ซึ่งเร็วกว่าหลายร้อยเท่าบนข้อมูลขนาดใหญ่

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D Bar Columns Visualizer
- Swap & Comparison Real-time Counter
- Web Audio Tone Synthesizer

4. ขั้นตอนการทดลอง

- กำหนดขนาดข้อมูล $N = 1,000$
- รัน Bubble Sort และบันทึกเวลาและจำนวน Swap
- รัน Quick Sort และบันทึกผลเปรียบเทียบ

5. ตารางบันทึกผลการทดลอง

อัลกอริทึม	จำนวนข้อมูล N	จำนวนการเปรียบเทียบ	เวลาประมวลผล (s)	สัญกรณ์ Big-O
Bubble Sort	1,000	~500,000 ครั้ง	0.1250 s	$O(n^2)$
Quick Sort	1,000	~10,000 ครั้ง	0.0018 s	$O(n \log n)$ (70x เร็วกว่า)

6. การวิเคราะห์และสรุปผลการทดลอง

- สรุปผล สำหรับชุดข้อมูลขนาดใหญ่ การเลือกใช้อัลกอริทึม $O(n \log n)$ มีความจำเป็นต่อความเสถียรของระบบ

? 3. สรุปภาพรวมและคำถามทบทวนท้ายบทปฏิบัติการ

คำถามทบทวนท้ายบทปฏิบัติการ

- จงอธิบายการประยุกต์ใช้ความรู้จากห้องปฏิบัติการบทที่ 6 ในการพัฒนาสื่อการสอนวิทยาศาสตร์ในชั้นเรียน
- ให้นักเรียนเปรียบเทียบข้อดีและข้อจำกัดของการทดลองบนระบบจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands
- ให้ออกแบบใบกิจกรรมการเรียนรู้แบบสืบเสาะ (Inquiry-based Worksheet) สำหรับนักเรียนระดับมัธยมศึกษาโดยใช้ห้องปฏิบัติการนี้



ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้อัตโนมัติ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร. ชีวะ ทัศนาศาสตร์ • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี



1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D บทที่ 7 (ฟังก์ชันและรีเคอร์ชัน)

```
graph LR
    LAB["LAB [\"กระบวนการปฏิบัติการบทที่ 7\"] → S1[\"1. กำหนดวัตถุประสงค์  
(ฟังก์ชันและรีเคอร์ชัน)\"]"]
    S1 --> S2["2. ปฏิบัติการจำลอง 2D/3D AR  
(Touchless Interaction)"]
    S2 --> S3["3. บันทึกตารางผลการทดลอง  
(Data Logging Sheet)"]
    S3 --> S4["4. วิเคราะห์และสรุปผล  
(Scientific Inquiry)"]
```



2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 7.0 หอคอยแห่งฮานอยและการเรียกซ้ำเชิงโมดูล

- รหัสโมดูล LAB 7.0
- รูปแบบการจำลอง 2D Peg Canvas ↔ 3D Holographic Hanoi Pegs
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter07_recursion_hanoi.html



1. วัตถุประสงค์การทดลอง

- ศึกษาฟังก์ชันเรียกซ้ำและ Call Stack
- พิสูจน์สูตรจำนวนก๊วยย้ายจาน $2^n - 1$



2. หลักการและทฤษฎี

การแก้ปัญหาแบบเวียนเกิดของหอคอยแห่งฮานอยใช้ความสัมพันธ์ $T(n) = 2T(n-1) + 1 = 2^n - 1$



3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D Cylinder Disks & Pegs
- Call Stack Depth Gauge
- Auto-solve Step Engine



4. ขั้นตอนการทดลอง

- ตั้งค่าจาน $n = 3, 4, 5$ ใบ
- รันการย้ายทีละขั้นตอนและนับจำนวนก๊วยทั้งหมด
- บันทึกผลและเปรียบเทียบกับสูตรทฤษฎี



5. ตารางบันทึกผลการทดลอง

จำนวนจาน (n)	สูตรคำนวณ $2^n - 1$	จำนวนก๊วยย้ายจริง	เวลาประมวลผล (ms)
3	$2^3 - 1 = 7$	7 ก๊วย	0.2 ms
4	$2^4 - 1 = 15$	15 ก๊วย	0.4 ms
5	$2^5 - 1 = 31$	31 ก๊วย	0.9 ms



6. การวิเคราะห์และสรุปผลการทดลอง

- สรุปผล ฟังก์ชันเรียกซ้ำสามารถแก้ปัญหาเชิงโครงสร้างได้อย่างกระชับ แต่ต้องระวังการเติบโตแบบเอกซ์โพเนนเชียล $O(2^n)$



3. สรุปภาพรวมและคำถามทบทวนท้ายบทปฏิบัติการ



คำถามทบทวนท้ายบทปฏิบัติการ

- จงอธิบายการประยุกต์ใช้ความรู้จากห้องปฏิบัติการบทที่ 7 ในการพัฒนาสื่อการสอนวิทยาศาสตร์ในชั้นเรียน

- ให้นักเรียนเปรียบเทียบข้อดีและข้อจำกัดของการทดลองบนระบบจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands
- ให้ออกแบบใบกิจกรรมการเรียนรู้แบบสืบเสาะ (Inquiry-based Worksheet) สำหรับนักเรียนระดับมัธยมศึกษาโดยใช้ห้องปฏิบัติการนี้



ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้วิทยาการคำนวณ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร.ชิวะ ทศนา • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D บทที่ 8 (วิทยาศาสตร์เชิงคำนวณและฟิสิกส์)

```
graph LR
    LAB["LAB [\"กระบวนการปฏิบัติการบทที่ 8\"] → S1[\"1. กำหนดวัตถุประสงค์  
(วิทยาศาสตร์เชิงคำนวณและฟิสิกส์)\"]
    S1 → S2["2. ปฏิบัติการจำลอง 2D/3D AR  
(Touchless Interaction)"]
    S2 → S3["3. บันทึกตารางผลการทดลอง  
(Data Logging Sheet)"]
    S3 → S4["4. วิเคราะห์และสรุปผล  
(Scientific Inquiry)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 8.0 แบบจำลองการเคลื่อนที่แบบโพรเจกไทล์

- รหัสโมดูล LAB 8.0
- รูปแบบการจำลอง 2D Trajectory Plotter ↔ 3D AR Cannon Arena
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter08_projectile_motion.html

1. วัตถุประสงค์การทดลอง

- จำลองการเคลื่อนที่วิถีโค้งด้วยวิธีเชิงตัวเลขอย่างง่าย
- พิสูจน์มุมยิง 45 องศาให้ระยะตกไกลที่สุด

2. หลักการและทฤษฎี

สมการก้าวเวลาอย่างง่าย: $\vec{v}(t + \Delta t) = \vec{v}(t) + \vec{a}\Delta t$ และ $\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}\Delta t$

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- 3D Cannon Barrel with Angle Dial
- Trajectory Arc Mesh
- Flight Time & Range Telemetry

4. ขั้นตอนการทดลอง

- ยิงปืนใหญ่ที่มุม $30^\circ, 45^\circ, 60^\circ$ ด้วยความเร็วต้นคงที่ 31.3 m/s
- วัดระยะตกไกลสุดและเวลาลอยตัว
- บันทึกผลการทดลองในตาราง

5. ตารางบันทึกผลการทดลอง

มุมยิง (องศา)	ความเร็วต้น (m/s)	เวลาลอยตัว (s)	ระยะตกไกลสุด (R)	การเปรียบเทียบ
30°	31.3 m/s	3.19 s	86.6 m	วิธีราบ
45°	31.3 m/s	4.51 s	100.0 m	🏆 ตกไกลที่สุดตามทฤษฎี
60°	31.3 m/s	5.53 s	86.6 m	วิถีโค้ง

6. การวิเคราะห์และสรุปผลการทดลอง

- สรุปผล เปรียบเทียบวิถีการเคลื่อนที่จากการจำลองที่ได้อย่างแม่นยำ และพิสูจน์ว่ามุม 45 องศาให้ระยะตกไกลที่สุดในสภาวะที่ไม่มีแรงต้านอากาศ

? 3. สรุปภาพรวมและคำถามทบทวนท้ายบทปฏิบัติการ

คำถามทบทวนท้ายบทปฏิบัติการ

- จงอธิบายการประยุกต์ใช้ความรู้จากห้องปฏิบัติการบทที่ 8 ในการพัฒนาสื่อการสอนวิทยาศาสตร์ในชั้นเรียน

- ให้นักเรียนเปรียบเทียบข้อดีและข้อจำกัดของการทดลองบนระบบจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands
- ให้ออกแบบใบกิจกรรมการเรียนรู้แบบสืบเสาะ (Inquiry-based Worksheet) สำหรับนักเรียนระดับมัธยมศึกษาโดยใช้ห้องปฏิบัติการนี้



ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้อัตโนมัติ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร. ชีวะ ทศนา • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D บทที่ 9 (ปัญญาประดิษฐ์และคอมพิวเตอร์วิทัศน์)

```
graph LR
    LAB["LAB 9.0: การจัดการเรียนรู้ด้วยเทคโนโลยี"] --> S1["1. กำหนดวัตถุประสงค์  
(ปัญหาการเรียนรู้และคอมพิวเตอร์วิทัศน์)"]
    S1 --> S2["2. ปฏิบัติการจำลอง 2D/3D AR  
(Touchless Interaction)"]
    S2 --> S3["3. บันทึกผลการทดลอง  
(Data Logging Sheet)"]
    S3 --> S4["4. วิเคราะห์และสรุปผล  
(Scientific Inquiry)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 9.0 คอมพิวเตอร์วิทัศน์และการรู้จำท่าทางมือ

- รหัสโมดูล LAB 9.0
- รูปแบบการจำลอง 2D 21-Joint Tracking Canvas ↔ 3D AR Spatial Skeleton
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter09_mediapipe_vision.html

1. วัตถุประสงค์การทดลอง

- ตรวจจับและสกัดพิกัดข้อต่อมือ 21 จุด 3 มิติ
- คำนวณระยะห่างจุดเพื่อจำแนกท่าทาง Pinch Gesture ไร้สัมผัส

2. หลักการและทฤษฎี

โมเดล MediaPipe Hands คำนวณพิกัด Normalization (x, y, z) ของข้อต่อ 21 จุด โดยท่า Pinch จะมีระยะระหว่างปลายนิ้วโป้งและนิ้วชี้ $d < 0.08$

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- Web Camera Video Feed
- 21-Landmark Mesh Skeleton
- KNN Gesture Classifier HUD

4. ขั้นตอนการทดลอง

- ส่องมือหน้ากล้องเว็บแคม ทำท่าฝ่ามือเปิด และทำจิบนิ้ว
- สังเกตระยะทาง d และการจำแนกท่าทางของ AI
- บันทึกผลลงในตาราง

5. ตารางบันทึกผลการทดลอง

ท่าทางที่ท่า	พิกัด P_4 (นิ้ว)	พิกัด P_8 (นิ้ว)	ระยะทางจุด (d)	ผลการจำแนก AI
แบมือ (Open Palm)	(0.40, 0.60, 0.0)	(0.45, 0.30, 0.0)	0.3041	OPEN_PALM
จิบนิ้ว (Pinch)	(0.45, 0.50, 0.0)	(0.47, 0.52, 0.0)	0.0283 (< 0.08)	PINCH_ACTIVE (SELECT)

6. การวิเคราะห์และสรุปผลการทดลอง

- สรุปผล โมเดล MediaPipe Hands สามารถตรวจจับท่าทางมือมนุษย์แบบ 3 มิติได้อย่างแม่นยำสูงและมีความหน่วงต่ำ (Low Latency) เหมาะกับการนำมาใช้ควบคุมสื่อเสมือนจริงไร้สัมผัส

3. สรุปภาพรวมและคำถามทบทวนท้ายบทปฏิบัติการ

คำถามทบทวนท้ายบทปฏิบัติการ

- จงอธิบายการประยุกต์ใช้ความรู้จากห้องปฏิบัติการบทที่ 9 ในการพัฒนาสื่อการสอนวิทยาศาสตร์ในชั้นเรียน
- ให้นักเรียนเปรียบเทียบข้อดีและข้อจำกัดของการทดลองบนระบบจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands

3. ให้ออกแบบใบกิจกรรมการเรียนรู้แบบสืบเสาะ (Inquiry-based Worksheet) สำหรับนักเรียนระดับมัธยมศึกษาโดยใช้ห้องปฏิบัติการนี้



ชุดห้องปฏิบัติการเสมือนจริงเพื่อการจัดการเรียนรู้อิวยาการคำนวณ

รายวิชา 4122104 วิทยาการคำนวณสำหรับการสอนวิทยาศาสตร์

ผู้ออกแบบและพัฒนาสื่อ ผู้ช่วยศาสตราจารย์ ดร.ชัชวาล ทัศนาศ • คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

1. สถาปัตยกรรมห้องปฏิบัติการเสมือนจริง 2D/3D บทที่ 10 (โครงงานนวัตกรรมสังเคราะห์)

```
graph LR
    LAB["กระบวนการปฏิบัติการบทที่ 10"] --> S1["1. กำหนดวัตถุประสงค์  
(โครงงานนวัตกรรมสังเคราะห์)"]
    S1 --> S2["2. ปฏิบัติการจำลอง 2D/3D AR  
(Touchless Interaction)"]
    S2 --> S3["3. บันทึกตารางผลการทดลอง  
(Data Logging Sheet)"]
    S3 --> S4["4. วิเคราะห์และสรุปผล  
(Scientific Inquiry)"]
```

2. คู่มือการทดลอง 5 ปฏิบัติการเสมือนจริงฉบับสมบูรณ์

การทดลองที่ 10.0 การประมวลผลความรู้และสถาปัตยกรรมโครงงาน

- รหัสโมดูล LAB 10.0
- รูปแบบการจำลอง 2D Architecture Matrix ↔ 3D Capstone Hologram
- ลิงก์เข้าสู่ห้องปฏิบัติการ https://tsanaphy2023.github.io/computing-science/simulators/chapter10_capstone_synthesizer.html

1. วัตถุประสงค์การทดลอง

- สังเคราะห์องค์ความรู้ 10 บทสู่สถาปัตยกรรม Full-Stack
- ตรวจสอบความสมบูรณ์ของโครงงานนวัตกรรมดิจิทัล

2. หลักการและทฤษฎี

โครงงานนวัตกรรมที่สมบูรณ์ต้องผสมผสานส่วน Logic, Data, AI Vision, และ IoT Physical Computing เข้าด้วยกันอย่างไร้รอยต่อ

3. อุปกรณ์และเครื่องมือจำลองเสมือนจริง

- Full-Stack System Tree
- Automated Module Verification Engine
- 3D Project Crystal Hologram

4. ขั้นตอนการทดลอง

- ตรวจสอบการเชื่อมต่อของ 4 โมดูลหลัก (Logic, Vision, Data, IoT)
- รันระบบทดสอบ Release Verification
- บันทึกผลการประเมินโครงงาน

5. ตารางบันทึกผลการทดลอง

หมวดองค์ความรู้	โมดูลในโครงงาน	สถานะการทดสอบ Unit Tests	คะแนนความสมบูรณ์
บทที่ 1-2: Logic & Flow	Core Algorithm Engine	✅ 100% PASS	25 / 25
บทที่ 3-7: Software Core	Python Data Layer	✅ 100% PASS	25 / 25
บทที่ 8-9: Sim & AI	MediaPipe & Physics Sim	✅ 100% PASS	25 / 25
บทที่ 10: Capstone	Full-Stack Integration	✅ 100% PASS	100 / 100 (Masterclass)

6. การวิเคราะห์และสรุปผลการทดลอง

- สรุปผล โครงงานนวัตกรรมวิทยาการคำนวณที่ผ่านการทดสอบและมีสถาปัตยกรรมสะอาด พร้อมสำหรับการเผยแพร่ทางวิชาการและนำไปใช้แก้ปัญหาจริงในสังคม

 คำถามทบทวนท้ายบทปฏิบัติการ

1. จงอธิบายการประยุกต์ใช้ความรู้จากห้องปฏิบัติการบทที่ 10 ในการพัฒนาสื่อการสอนวิทยาศาสตร์ในชั้นเรียน
2. ให้นักเรียนเปรียบเทียบข้อดีและข้อจำกัดของการทดลองบนระบบจำลองเสมือนจริง 2D Canvas และ 3D AR MediaPipe Hands
3. ให้ออกแบบใบกิจกรรมการเรียนรู้แบบสืบเสาะ (Inquiry-based Worksheet) สำหรับนักเรียนระดับมัธยมศึกษาโดยใช้ห้องปฏิบัติการนี้

บรรณานุกรม

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.

Lugaresi, C., et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. *arXiv preprint arXiv:1906.08172*.

Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3), 33-35.

ชีวะ ทศนา. (2569). *วิทยาการคำนวณและการแก้ปัญหาเชิงคำนวณด้วยเทคโนโลยีโลกเสมือนจริง*. สำนักพิมพ์มหาวิทยาลัยราชภัฏรำไพพรรณี.